

VOLUME 00 / RUST BACKEND MASTERY

学習ガイド — コンピュータとWebから完成像まで

プログラミング未経験から、Rust で作る Web アプリ「RustNote」の全体像と、全14巻の歩き方をつかむ

第0巻: 学習ガイド — コンピュータとWebから完成像まで

> この巻は、全14巻からなる「Rust バックエンド教材」の**入口**です。プログラミングをしたことがない人を想定して、「コンピュータの中で何が起きているのか」「Web アプリはどうやって動いているのか」を、たとえ話と図で1つずつ説明します。そのうえで、このシリーズを通して作るメモアプリ **RustNote（ラストノート）** の完成像と、14巻をどの順番で読めばよいかを示します。
>
> 難しい文法はまだ覚えなくて大丈夫です。この巻のコードは「動かして、出力を見て、何が起きたかを言葉で説明する」ためのものです。

この巻で身に付けること

到達目標

この巻を読み終えると、次のことができるようになります。

- **コンピュータの部品と OS の役割**を、たとえ話を使って自分の言葉で説明できる
- **ファイル・フォルダ・パス**を区別し、「今いるフォルダ」を意識してコマンドを打てる
- **PowerShell（パワーシェル）**で基本のコマンドを打ち、Rust のプログラムをコンパイルして実行できる
- ブラウザに URL を入れてから画面が出るまでに起きる **HTTP のやりとり**を、順番に説明できる
- **サーバー・ポート・localhost** の意味が分かり、Rust だけで小さな Web サーバーを動かせる
- **データベース・認証・認可**が、なぜ Web アプリに必要なのかを説明できる
- Rust コンパイラの**エラーメッセージを部品ごとに読んで**、自分で直せる
- RustNote の完成像と、全14巻の読み進め方を把握して、学習の計画を立てられる

このシリーズで作るもの: RustNote

このシリーズでは、最終的に **RustNote** という「ログインして使うメモ帳」の Web アプリを作り、インターネットに公開します。最初に完成した姿を見ておくと、各巻で学ぶことが「全体のどこの部品なのか」が分かりやすくなります。

RustNote でできることは次のとおりです。

- メールアドレスとパスワードで**アカウントを作る（登録）**、**ログイン**、**ログアウト**する
- ログインした人だけが、**自分のノート**を一覧・作成・表示・編集・削除できる
- ノートに「済み（done）」の印を付けたり外したりできる
- キーワードで**検索**でき、ノートが多いときは**ページ送り**で少しずつ表示する
- 他の人のノートは、URL を直接打ち込んでも**絶対に見られない**

> **用語: Web アプリ（ウェブアプリ）** … ブラウザ（Chrome や Edge など）で開いて使うアプリのことです。スマホのアプリのようにインストールしなくても、URL を開くだけで使えます。

完成画面のイメージ

次の2枚は、RustNote の完成画面のイメージです（実際の色や配置は第9巻・第10巻で一緒に作りながら決めます）。



図0-1: RustNote のログイン画面（完成イメージ）。表示は GET、送信は POST という別々のリクエストになる



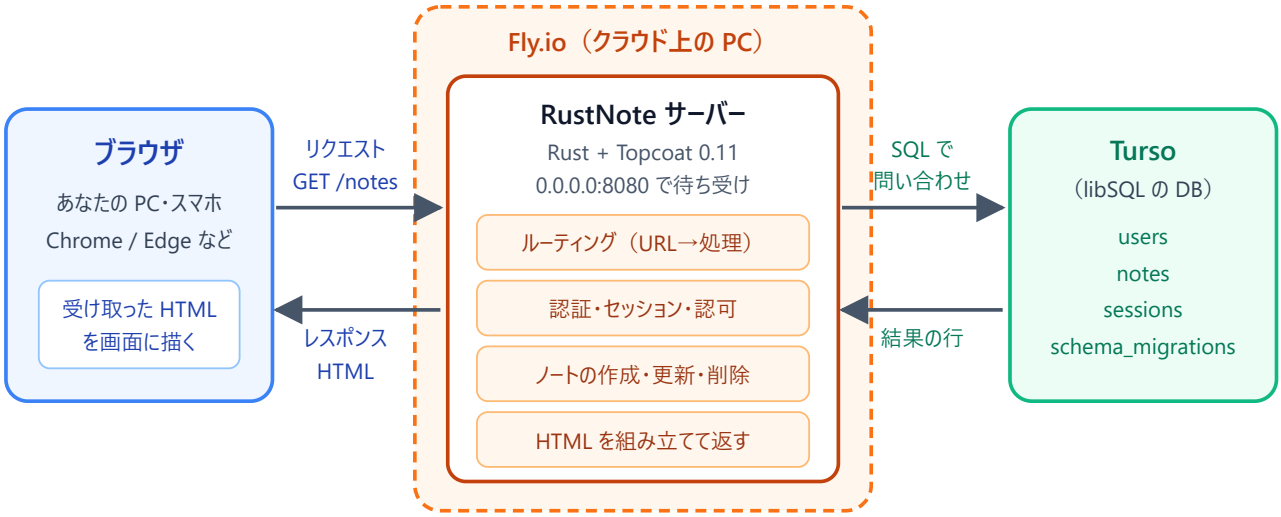
図0-2: ログイン後のノート一覧画面（完成イメージ）。チェック済みのノートは「済み（done）」

画面の部品と、そのとき裏でブラウザが送るリクエスト（お願い）の対応は次のとおりです。今は「部品ごとに決まった URL がある」とだけ分かれば十分です。URL やメソッド（GET・POST）は UNIT 05 で説明します。

画面の部品	ブラウザが送るリクエスト	サーバーがすること
一覧を開く・検索・ページ送り	`GET /notes`（`?q=買い物` で検索）	ログイン中の人のノートだけを探して HTML で返す
「作成」ボタン	`POST /notes`	タイトルを検査して保存し、一覧へ移動させる（303）
タイトルをクリック	`GET /notes/{id}`	そのノートの詳細を返す（他人のなら 404）
「編集」	`GET /notes/{id}/edit` → `POST /notes/{id}/edit`	編集フォームを返す → 変更を保存する
チェックボックス	`POST /notes/{id}/toggle`	済み／未済みを切り替える
「削除」	`POST /notes/{id}/delete`	ノートを削除する
「ログアウト」	`POST /logout`	ログイン状態を終わらせる

全体の構成

RustNote は、次の3つの登場人物でできています。



開発中は同じ組み立てを自分の PC で動かします（http://127.0.0.1:3000、DB はローカルのファイル）

図0-3: RustNote のシステム構成。ブラウザ ⇔ Fly.io 上の Rust サーバー ⇔ Turso データベース

登場人物	役割	たとえると	主に学ぶ巻
ブラウザ	URL を開き、サーバーから届いた HTML を画面に描く	お店に来たお客さん	第0巻・第6巻
RustNote サーバー	リクエストを受け取り、ログインの確認・ノートの処理をして HTML を返す。**Rust** で書き、Web アプリ用の部品集 **Topcoat（トップコート）** を使う	お店の店員	第2〜6巻・第8〜10巻
Turso（ターソ）	ユーザー・ノート・ログイン状態を、消えないように保存しておく**データベース**	お店の帳簿・倉庫	第7巻
Fly.io（フライアイオー）	サーバーのプログラムを、インターネットから届く場所で動かしてくれる**クラウドサービス**	お店を出す貸しテナント	第12巻

> **用語: フレームワーク** … アプリを作るときに毎回必要になる「よくある仕組み」（URL ごとに処理を振り分ける、フォームの値を受け取る、など）をまとめて用意してくれる部品集です。Topcoat は Rust 用の Web フレームワークの1つで、このシリーズでは 0.10 系を使います。まだ若いフレームワークなので、版が変わると書き方が変わることがあります（第1巻・第6巻で詳しく扱います）。

シリーズ全14巻の地図

14巻は、大きく4つのまとまりに分かれています。上から順に読むのが基本です。

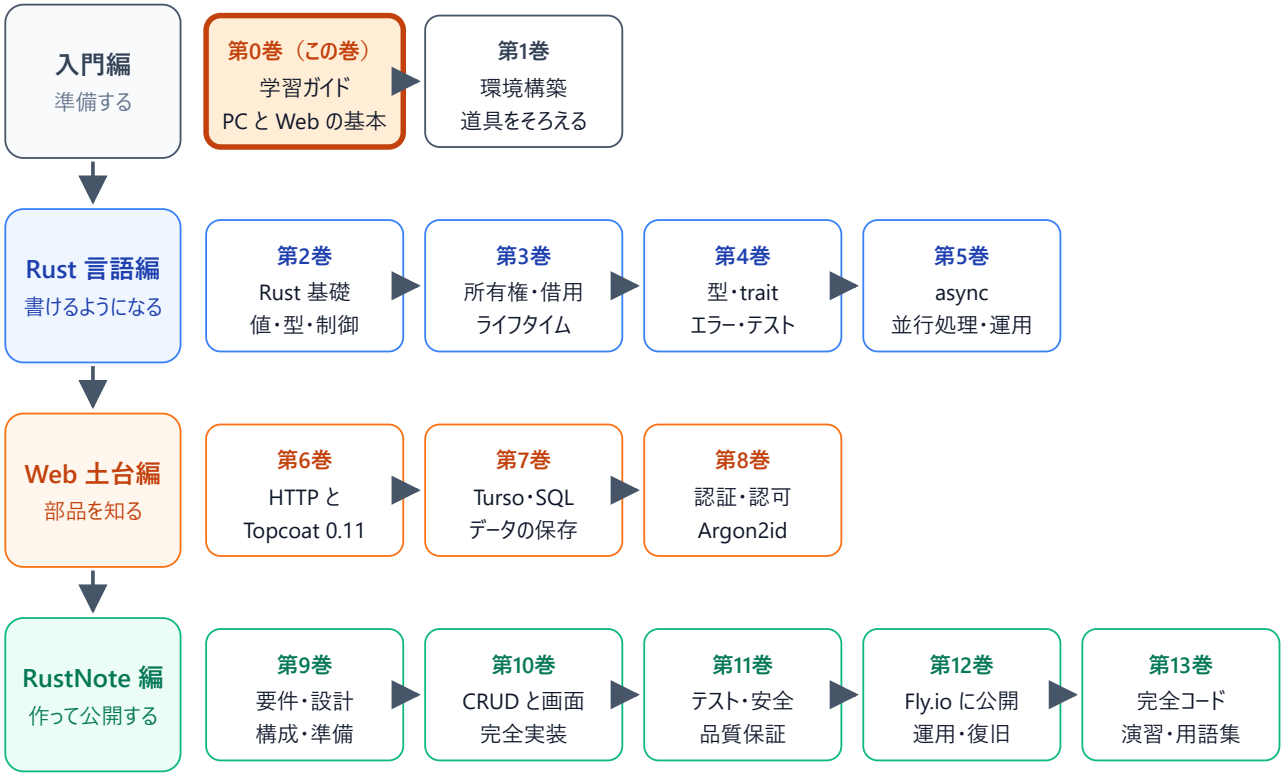


図0-4: シリーズ全14巻の地図。左の4段を上から下へ、各段の中は左から右へ読む

巻	テーマ	読み終えるとできること
第0巻	学習ガイド（この巻）	コンピュータと Web の基本が分かり、全体の地図を持てる
第1巻	環境構築	PowerShell・Git・VS Code・Rust・Topcoat・Turso・Fly.io の道具を自分の PC にそろえられる
第2巻	Rust 基礎	変数・型・関数・条件分岐・くり返し・構造体などで、小さなプログラムを書ける
第3巻	所有権・借用・ライフタイム	Rust 独特の「値の持ち主」のルールを理解し、コンパイラと仲良くなれる
第4巻	型・trait・エラー・テスト	大きめのプログラムを、型とエラー処理とテストで安全に組み立てられる
第5巻	async・並行処理	たくさんのリクエストを同時にさばくための非同期処理が分かる
第6巻	HTTP と Topcoat 0.11	Topcoat で Web ページを返し、フォームを受け取れる
第7巻	Turso・SQL	テーブルを設計し、SQL でデータを保存・検索できる
第8巻	認証・認可	安全な登録・ログイン・ログアウトと、「自分のデータだけ触れる」仕組みを作れる
第9巻	RustNote 設計・準備	要件と画面を決め、プロジェクトの骨組みを作れる
第10巻	RustNote 完全実装	ノートの一覧・作成・編集・削除・検索・ページ送りを完成させられる
第11巻	品質保証	テスト・セキュリティ確認・ログ・性能測定で品質を確かめられる
第12巻	Fly.io 公開・運用	Docker で包んでインターネットに公開し、障害から復旧できる
第13巻	完全コード・演習・用語集	完成したコードの全体を読み通し、総合演習で力を確かめられる

この巻の学習地図

この巻の10の単元（UNIT）は、「コンピュータの基本」→「Web の基本」→「学び方」の順に並んでいます。

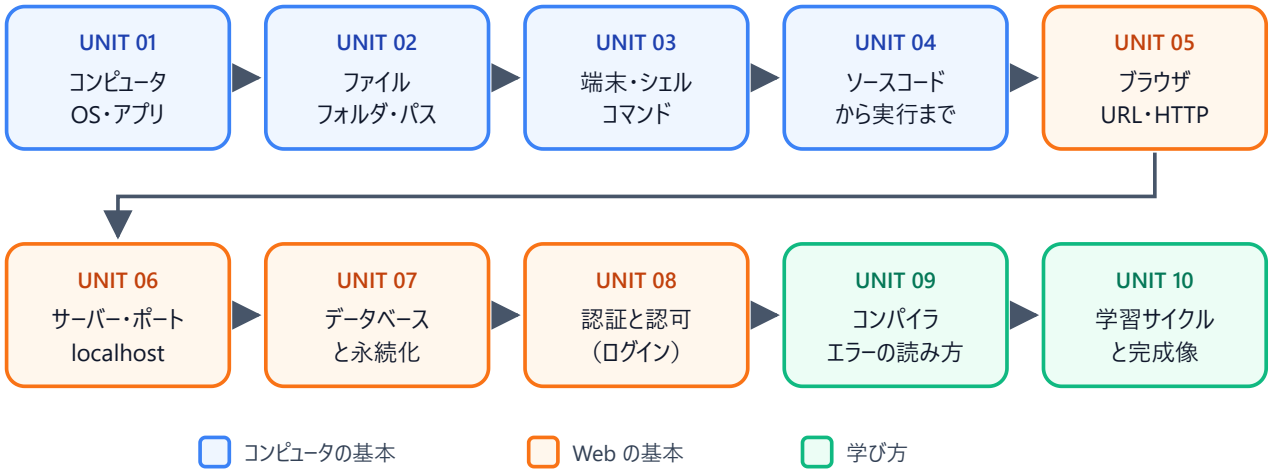


図0-5: この巻の学習地図。青はコンピュータ、橙は Web、緑は学び方のまとまり

単元	一言でいうと	動かすもの
UNIT 01	コンピュータは「料理人・調理台・冷蔵庫・店長」	自分のプロセス番号を表示するプログラム
UNIT 02	ファイルの場所は「住所」で表す	ファイルを書いて読み戻すプログラム
UNIT 03	文字で OS にお願いする窓	PowerShell の基本コマンド
UNIT 04	人の言葉を機械の言葉に翻訳する	`cargo new` → `cargo run`
UNIT 05	Web は「決まった形の手紙」のやりとり	`curl.exe` と、HTTP を手で送るプログラム
UNIT 06	ポートは「部屋番号」	Rust だけで作る小さな Web サーバー
UNIT 07	消えない場所にしまう	ファイルに保存するカウンター
UNIT 08	「誰か」と「何をしてよいか」は別の話	ログインと持ち主チェックのミニ版
UNIT 09	エラーは「直し方つきの診断書」	わざと間違えたコード
UNIT 10	予想→実行→観察→変更→説明	テストで間違いを見つけて直す

前提知識

- **プログラミングの経験は不要**です。
- パソコンの基本操作（ファイルを開く、文字を入力する、コピーと貼り付け）ができれば十分です。
- 英語が苦手でも大丈夫です。コマンドやエラーメッセージに出てくる英単語は、そのつど意味を説明します。

この巻は全14巻の最初に読む巻です。先に読んでおくべき巻はありません。

この巻の進め方

この巻は「読むだけ」でも理解できるように書いていますが、**手を動かすと理解の深さがまったく違います**。おすすめの進め方は次のとおりです。

1. まず UNIT 01 から UNIT 10 まで、コードは眺めるだけで一度通して読む
2. 第1巻で Rust などの道具をインストールする
3. この巻に戻り、コードを実際に動かしながら2回目を読む

コードを動かすときは、練習用のプロジェクト（プログラムをまとめておくフォルダ）を1つだけ作り、例ごとに中身を入れ替えて使います。UNIT 04 で詳しく説明しますが、手順だけ先に書いておきます。

```
# ホームフォルダに rust-study フォルダを作って移動する
mkdir rust-study
cd rust-study
# 練習用プロジェクト practice を作って、その中へ移動する
cargo new practice
cd practice
```

あとは、各例のコードを `src\main.rs` というファイルに**まるごと貼り付けて保存**し、`cargo run` を実行します。

> **ヒント:** `cargo run` は、プログラム自身が表示する文字より先に、次のような3行を表示します。
>
> `Compiling practice v0.1.0 (...)` ／ `Finished ...` ／ `Running ...`
>
> これは Cargo（カーゴ）という道具の「今コンパイルしています」「終わりました」「実行します」という報告です（UNIT 04 で説明します）。この巻の実行結果では、説明に必要なとき以外はこの3行を省き、**プログラム自身の出力だけ**を載せます。`cargo run -q` と打つと、この3行を出さずに実行できます。

> **ヒント:** この巻に出てくる `C:\Users\you\...` の `you` の部分は、あなたの Windows のユーザー名に読み替えてください。プロセス番号や時刻など、実行するたびに変わる値もあります。

使うツールと版

この巻のコードと実行結果は、次の環境で実際に動かして確認しています。

道具	版	この巻での使い道
Windows 11	ARM64 版（x64 版でも同じ手順）	すべての作業
PowerShell	5.1（Windows に最初から入っているもの）	コマンドを打つ（UNIT 03 以降）
Rust（rustc・cargo）	1.99.1、edition 2024（1.99 以上なら同じ手順）	すべての Rust コード
curl.exe	8.x（Windows に最初から入っているもの）	HTTP のやりとりを見る（UNIT 05・06）
ブラウザ	Chrome または Edge	作ったサーバーにアクセスする（UNIT 06）
SQLite のコマンドラインツール	3.50（参考）	SQL の例の確認だけに使用（UNIT 07。読者は見るだけでよい）

> **注意:** この巻ではまだ Topcoat・Turso・Fly.io は使いません。Rust に最初から付いてくる機能（**標準ライブラリ**）だけで、Web の仕組みを体験します。このシリーズで使う Topcoat 0.11 は **Rust 1.99 以上**が必要なので、シリーズ全体で「Rust 1.99 以上、動作確認は 1.99.1」を基準にしています。インストールと版の確かめ方は第1巻で説明します。
>
> WSL（Windows の中で Linux を動かす仕組み）や macOS・Linux を使う人は、コマンドの違いを各単元の「補足」で示します。

UNIT 01: コンピュータ・OS・アプリ

この単元のゴール

- **CPU・メモリ・ストレージ**の役割を、台所のたとえで説明できる
- **OS（オーエス）**が何をしているか、アプリとの関係を説明できる
- 動いているプログラムを **プロセス** と呼ぶことと、プロセスが外へ結果を伝える3つの出口（**標準出力・標準エラー出力・終了コード**）を区別できる
- RustNote のサーバーも「OS の上で動く1つのプロセス」だと分かる

たとえ話でつかむ

コンピュータの中は、**レストランの厨房（ちゅうぼう）**にとてもよく似ています。

- **CPU（シーピーユー）は料理人**です。レシピに書かれた手順を、1つずつ、ものすごい速さでこなします。1秒間に何十億回もの計算ができますが、レシピに書いていないことは一切しません。

- **メモリは調理台**です。今まさに使っている材料や、切りかけの野菜を置いておく場所です。料理人の手がすぐ届くので速く使えますが、広さには限りがあり、**閉店（電源オフ）すると片付けられて何も残りません。**
- **ストレージ（SSD など）は冷蔵庫や食品庫**です。たくさんしまっておけて、**閉店しても中身は残ります。**ただし、取りに行くのは調理台より時間がかかります。
- **OS は店長**です。どの料理人がいつどのコンロを使うか、調理台のどこを誰が使うかを決め、他の人の皿に勝手に触らないように見張っています。Windows・macOS・Linux はどれも OS です。
- **アプリ（プログラム）はレシピ**で、**プロセスはそのレシピで「今まさに作っている最中の1皿」**です。同じレシピ（同じプログラム）から、同時に2皿（2つのプロセス）を作ることもできます。

> **用語: CPU** … Central Processing Unit（中央処理装置）の略。計算と命令の実行を担当する部品です。
>
> **用語: プロセス** … 実行中のプログラムのことです。ファイルとして保存されている「プログラム」は動いていない状態、OS が読み込んで動かし始めたものが「プロセス」です。

図で理解する

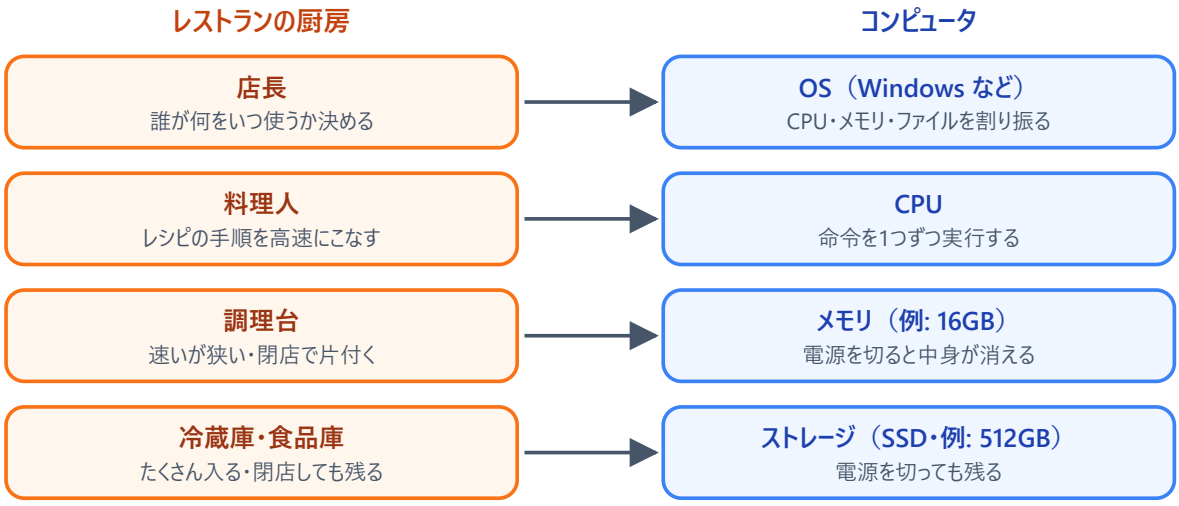


図1-1: 厨房とコンピュータの対応。左のたとえが、右の部品に当たる

次の図は、コンピュータを**層（レイヤー）**で見たものです。いちばん下に機械（ハードウェア）があり、その上に OS、さらにその上でたくさんのプロセスが動いています。プロセスは機械を直接さわらず、**必ず OS にお願いして**メモリやファイルやネットワークを使います。

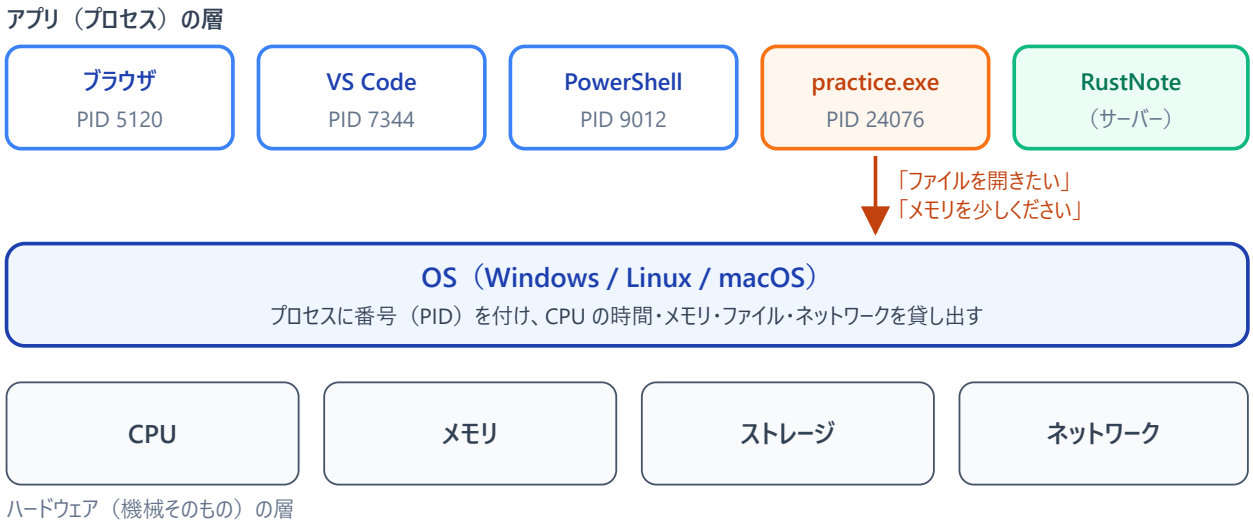


図1-2: コンピュータの層。プロセスは OS を通してだけ機械を使う（PID の数字は例）

まず動かす最小コード

最初のプログラムでは、「自分は OS の上で動く1つのプロセスである」ということを、プログラム自身に確かめさせます。

自分が動いている環境とプロセス番号を表示する

> **▼ このコードがやること（先に日本語で）**：このプログラムが動いている OS の種類と CPU の種類、そして OS がこのプロセスに付けた番号（プロセス ID）を表示します。最後に CPU に1から100までの足し算をさせ、その結果も表示します。

```rust
use std::env;
use std::process;

fn main() {
    // このプログラムが動いている OS の種類（windows / linux / macos など）
    let os = env::consts::OS;
    // CPU の種類（x86_64 / aarch64 など）
    let arch = env::consts::ARCH;
    // OS がこのプロセスに付けた番号（プロセス ID）
    let pid = process::id();

    println!("OS の種類: {os}");
    println!("CPU の種類: {arch}");
    println!("プロセス ID: {pid}");

    // CPU に足し算を 100 回してもらい、途中の値はメモリ上の total に置く
    let mut total = 0;
    for n in 1..=100 {
        total += n;
    }
    println!("1 から 100 までの合計: {total}");
}
```

練習用プロジェクトの `src\main.rs` をこの内容に置き換えて保存し、実行します。

```
cargo run
```

▼ **実行結果**（2回実行した結果です）

```
OS の種類: windows
CPU の種類: aarch64
プロセス ID: 24076
1 から 100 までの合計: 5050
```

```
OS の種類: windows
CPU の種類: aarch64
プロセス ID: 49372
1 から 100 までの合計: 5050
```

同じプログラムなのに、**プロセス ID だけが毎回変わっている**ことに注目してください。プログラムを起動するたびに、OS が新しいプロセスを作り、新しい番号を付けているからです。`aarch64`（エーアーチ64）は ARM 系の CPU の名前です。一般的な Intel や AMD の CPU の PC なら `x86_64` と表示されます。

▼ コードを1つずつ分解して解説

解説1: 道具箱から道具を取り出す（`use`）

```
use std::env;
use std::process;
```

- `std`（エスティーディー、standard の略）は、Rust に最初から付いてくる道具箱で、**標準ライブラリ**と呼びます。
- `use std::env;` は「`std` の中の `env`（環境に関する道具）を、短く `env` と書いて使えるようにする」という宣言です。
- 同じように `process`（プロセスに関する道具）も使えるようにしています。

> **用語: ライブラリ** … 誰かが作ってくれた便利な部品（関数など）の集まりです。自分で一から作らなくても、取り出して使えます。

解説2: プログラムの入り口 `main`

```
fn main() {
    // ...
}
```

- `fn` は function（関数）の略で、「ここから処理のまとまりを書きます」という印です。
- `main`（メイン）という名前の関数は特別で、**プログラムを起動すると最初にここが実行されます。**
- `{` から `}` までが、この関数の中身です。

解説3: 値に名前を付けて表示する

```
let os = env::consts::OS;
let pid = process::id();
println!("OS の種類: {os}");
```

- `let`（レット）は「値に名前を付けて、メモリに置いておく」という意味です。この名前付きの入れ物を **変数** と呼びます。
- `env::consts::OS` は、Rust が教えてくれる「この OS の種類」という文字です。
- `process::id()` は、**OS にこのプロセスの番号を問い合わせる**関数です。`()` は「関数を呼び出す」印です。
- `println!`（プリントライン）は、文字を画面に1行表示します。`"..."` の中の `{os}` の部分に、変数 `os` の中身が埋め込まれます。

解説4: CPU に計算させる

```
let mut total = 0;
for n in 1..=100 {
    total += n;
}
```

- `let mut` の `mut`（ミュート、mutable の略）は「あとで中身を書き換えてよい」という印です。Rust の変数は、`mut` を付けないと書き換えられません（UNIT 09 で、付け忘れたときのエラーを見ます）。
- `for n in 1..=100` は「`n` を 1, 2, 3, ..., 100 と変えながら、`{ }` の中を100回くり返す」という意味です。`1..=100` は「1以上100以下」を表します。
- `total += n;` は「`total` に `n` を足して、`total` を書き換える」という意味です。
- この100回の足し算を実行しているのが CPU、途中の `total` の値を置いているのがメモリです。

もう一歩: 実務に近い形

プロセスは、外の世界に結果を伝える**出口を3つ**持っています。

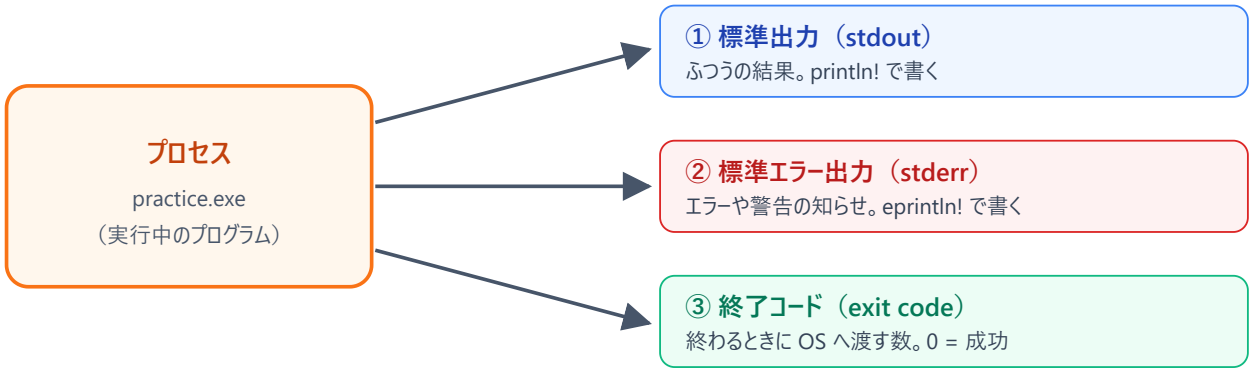


図1-3: プロセスの3つの出口。画面に出る文字は①と②、成功・失敗の合図は③

成功と失敗を、出口を使い分けて伝える

> **▼ このコードがやること（先に日本語で）**：起動するときに名前を1つ受け取り、「こんにちは、○○ さん」と表示します。名前が渡されなかったときは、エラーのメッセージを「エラー用の出口」に書き、**失敗の合図として終了コード 2** を OS に返して終わります。

```rust
use std::env;
use std::process;

fn main() {
    // コマンドラインで渡された 1 つ目の値を取り出す（無ければ None）
    let name = env::args().nth(1);

    match name {
        Some(name) => {
            // 標準出力へ書く（ふつうの結果）
            println!("こんにちは、{name} さん");
        }
        None => {
            // 標準エラー出力へ書く（エラーの知らせ）
            eprintln!("エラー: 名前を 1 つ渡してください（例: cargo run -- Hanako）");
            // 終了コード 2 でプロセスを終える（0 以外は「失敗」の合図）
            process::exit(2);
        }
    }
}
```

`cargo build` でコンパイルだけ行い、できあがった実行ファイル `practice.exe` を PowerShell から直接動かします。`$LASTEXITCODE` は「直前に動かしたプログラムの終了コード」を表示する PowerShell の変数です。

```
cargo build
.\target\debug\practice.exe Hanako
$LASTEXITCODE
.\target\debug\practice.exe
$LASTEXITCODE
```

▼ 実行結果

```
こんにちは、Hanako さん
0
エラー: 名前を 1 つ渡してください（例: cargo run -- Hanako）
2
```

さらに、`>` を使って「標準出力だけをファイルへ送る」と、2つの出口が別物であることがはっきり分かります。

```
.\target\debug\practice.exe > out.txt
Get-Content out.txt
```

▼ 実行結果

```
エラー: 名前を 1 つ渡してください（例: cargo run -- Hanako）
```

エラーのメッセージは画面に出ますが、`out.txt` の中身は空っぽです（`Get-Content` は何も表示しません）。`>` が横取りしたのは①の標準出力だけで、②の標準エラー出力はそのまま画面に流れたからです。

▼ コードを1つずつ分解して解説

解説1: 起動時に渡された値を受け取る

```
let name = env::args().nth(1);
```

- `env::args()` は、プログラムを起動したときに渡された値（**コマンドライン引数**）の並びです。0番目にはプログラム自身の名前が入り、1番目から先が渡した値です。
- `.nth(1)` は「1番目を取り出す」という意味です。
- 値が渡されていないこともあるので、結果は「ある（`Some(値)`）」か「ない（`None`）」のどちらかになります。この「あるかもしれないし、ないかもしれない」を表す型を `Option`（オプション）と呼びます（第2巻で詳しく学びます）。

解説2: 場合分けする `match`

```
match name {
    Some(name) => { /* 値があったとき */ }
    None => { /* 値がなかったとき */ }
}
```

- `match`（マッチ）は、値の形に応じて処理を分ける構文です。
- `Some(name)` の場合は、中身を `name` という名前で取り出して使えます。
- Rust では `Some` と `None` の**両方の場合を必ず書かないとコンパイルが通りません**。「値がなかったとき」を書き忘れる事故を、コンパイラが防いでくれるわけです。

解説3: エラー用の出口と終了コード

```
eprintln!("エラー: 名前を 1 つ渡してください（例: cargo run -- Hanako）");
process::exit(2);
```

- `eprintln!`（イープリントライン）は `println!` のエラー版で、**標準エラー出力**に書きます。
- `process::exit(2)` は、終了コード 2 を OS に渡して、その場でプロセスを終わらせます。
- 終了コードは「0 なら成功、0 以外なら失敗」という世界共通の約束です。どの数を使うかはプログラムが決めます。

終了コード	よくある意味
`0`	成功
`1`	一般的な失敗（Rust で `main` がエラーを返したときもこれ）
`2`	使い方の間違い（引数が足りない など）によく使われる
`101`	Rust のプログラムが**パニック**（次の節で説明）して止まった

> **ヒント:** `cargo run -- Hanako` のように、`--` の後ろに書いた値がプログラムへの引数になります。`--` より前は Cargo への指示、後ろはプログラムへの値、という区切りです。

よくあるエラーと読み方

実行中に止まる「パニック」

存在しない場所のデータを読もうとすると、Rust のプログラムは**パニック**して止まります。

```
fn main() {
    // 3 人分の点数をメモリに並べる
    let scores = vec![80, 95, 70];
    // 11 人目（番号 10）を取り出そうとする
    let index = 10;
    println!("{}", scores[index]);
}
```

▼ **実行結果**（`cargo run` の出力をそのまま載せています）

```
   Compiling practice v0.1.0 (C:\Users\you\rust-study\practice)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.28s
     Running `target\debug\practice.exe`

thread 'main' (50284) panicked at src\main.rs:6:26:
index out of bounds: the len is 3 but the index is 10
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
error: process didn't exit successfully: `target\debug\practice.exe` (exit code: 101)
```

何が起きたか: 1行ずつ読むと、次のことが書いてあります。

出力の部分	意味
`thread 'main' (50284) panicked`	メインの処理（スレッド）がパニックした。括弧の数字はスレッドの番号
`at src\main.rs:6:26`	場所は `src\main.rs` の **6行目・26文字目**
`index out of bounds: the len is 3 but the index is 10`	「範囲外の番号。長さは 3 なのに、番号 10 を指定した」
`note: run with RUST_BACKTRACE=1 ...`	「もっと詳しい経路を見たければ、この設定で実行して」という補足
`(exit code: 101)`	Cargo からの報告。プロセスは終了コード 101（パニック）で終わった

直し方: 番号は 0 から数えるので、3人分なら使える番号は `0`・`1`・`2` だけです。`let index = 2;` のように範囲内の番号にすれば動きます。

> **つまずきポイント:** パニックは**コンパイルは通ったのに、動かしてみて初めて起きる失敗**です（UNIT 04 で「コンパイル時の失敗」と「実行時の失敗」を区別します）。Rust は、範囲外の場所をこっそり読んでデタラメな値を返す代わりに、**その場で止めて場所を教えてくれます**。止まるのは不親切に見えますが、間違った値のまま処理が進むより、ずっと安全です。

RustNote ではここで使う

- RustNote のサーバーも、Fly.io というクラウドの PC の上で動く**1つのプロセス**です。違いは、`practice.exe` がすぐ終わるのに対して、サーバーは**止めるまでずっと動き続ける**ことです。
- サーバーの動作記録（**ログ**）は、標準出力・標準エラー出力に書きます。Fly.io はそれを集めて、`fly logs` というコマンドで見られるようにしてくれます（第12巻）。
- 起動に必要な設定が足りないときは、RustNote は分かりやすいメッセージを出して**0 以外の終了コード**で止まります。「どこかおかしいまま動き続ける」より「すぐ止まって理由を言う」ほうが、原因を早く見つけられるからです。

練習問題

問題1（予想してから実行）：最小コードの `for n in 1..=100` を `for n in 1..=10` に変えると、最後の行には何と表示されるでしょうか。実行する前に予想してから、確かめてください。

問題2: 「もう一歩」のコードで、名前が無いときの終了コードを `2` から `7` に変えてください。PowerShell で `$LASTEXITCODE` を表示して、7 になることを確かめましょう。

問題3: 次の a〜d は、厨房のたとえでいうと「料理人（CPU）」「調理台（メモリ）」「冷蔵庫（ストレージ）」のどれに当たるでしょうか。

- a. RustNote のサーバーが今まさに処理している、送られてきたノートのタイトル

- b. データベースに保存されたノート
- c. タイトルが 1〜80 文字かどうかを数えて判定すること
- d. `cargo build` で作られた `practice.exe` というファイル

解答と解説

問題1の解答: `1 から 100 までの合計: 55` と表示されます（文の「100」は `println!` の中の文字なので変わりません）。くり返しが1〜10になるので、1+2+…+10 = 55 です。文字の部分も直すなら、次のようにします。

```
fn main() {
    let mut total = 0;
    for n in 1..=10 {
        total += n;
    }
    println!("1 から 10 までの合計: {total}");
}
```

▼ 実行結果

```
1 から 10 までの合計: 55
```

問題2の解答: `process::exit(2);` を `process::exit(7);` に書き換えて、`cargo build` してから実行します。

```
cargo build
.\target\debug\practice.exe
$LASTEXITCODE
```

▼ 実行結果

```
エラー: 名前を 1 つ渡してください（例: cargo run -- Hanako）
7
```

ポイントは、`cargo build` を**し直すこと**です。ソースコードを書き換えても、コンパイルし直さなければ `practice.exe` は古いままです（UNIT 04 で詳しく見ます）。

問題3の解答:

- a. **メモリ**（処理中の値は調理台の上にある）
- b. **ストレージ**（データベースの中身は、最終的にストレージ上のファイルに保存される）
- c. **CPU**（数えて比べるのは計算＝料理人の仕事）

- d. **ストレージ**（ファイルはストレージに保存される。実行すると、OS がその中身をメモリに読み込んでプロセスにする）

まとめ

- CPU は計算する料理人、メモリは速いが消える調理台、ストレージは遅いが残る冷蔵庫、OS はそれらを割り振る店長です。
- 動いているプログラムを**プロセス**と呼び、OS が番号（PID）を付けて管理します。
- プロセスは**標準出力・標準エラー出力・終了コード**の3つの出口で結果を伝えます。0 は成功、Rust のパニックは 101 です。

UNIT 02: ファイル・フォルダ・パス

この単元のゴール

- **ファイル**と**フォルダ**（**ディレクトリ**）の関係を、木の形（ツリー）で説明できる
- **絶対パス**と**相対パス**の違いを説明し、正しく書ける
- 「今いるフォルダ（**カレントディレクトリ**）」によって、相対パスの指す場所が変わることを体験する
- Rust でファイルを書いて読み戻し、「ファイルが見つからない」エラーを読める

たとえ話でつかむ

ファイルの場所を表す**パス**は、**住所**とそっくりです。

- 「東京都 千代田区 千代田 1-1」のように、**いちばん大きな区切りから順に全部書いた住所**は、日本中どこにいる人に渡しても同じ場所を指します。これが**絶対パス**です。Windows なら `C:\Users\you\rust-study\practice\Cargo.toml` のように、ドライブ名 `C:` から書き始めます。
- 一方、「この角を曲がって2軒目」という**道案内**は、**今どこに立っているか**によって指す家が変わります。これが**相対パス**です。`src\main.rs` は「今いるフォルダの中の `src` フォルダの中の `main.rs`」という意味で、どこに立っているかで行き先が変わります。
- **フォルダ**は「棚」や「引き出し」です。引き出しの中に小さな引き出し（フォルダ）や書類（ファイル）を入れられます。

> **用語: ファイル** … ストレージに保存された、名前の付いたデータのかたまり。文章、画像、プログラムなど、中身は何でもかまいません。
>
> **用語: フォルダ（ディレクトリ）** … ファイルや別のフォルダを入れておく入れ物。Windows では「フォルダ」、コマンドの世界では「ディレクトリ」と呼ぶことが多いですが、同じものです。
>
> **用語: 拡張子（かくちょうし）** … ファイル名の最後の `.` から後ろの部分（`main.rs` の `rs`、`Cargo.toml` の `toml`）。中身の種類の目印です。`.rs` は Rust のソースコードを表します。

図で理解する

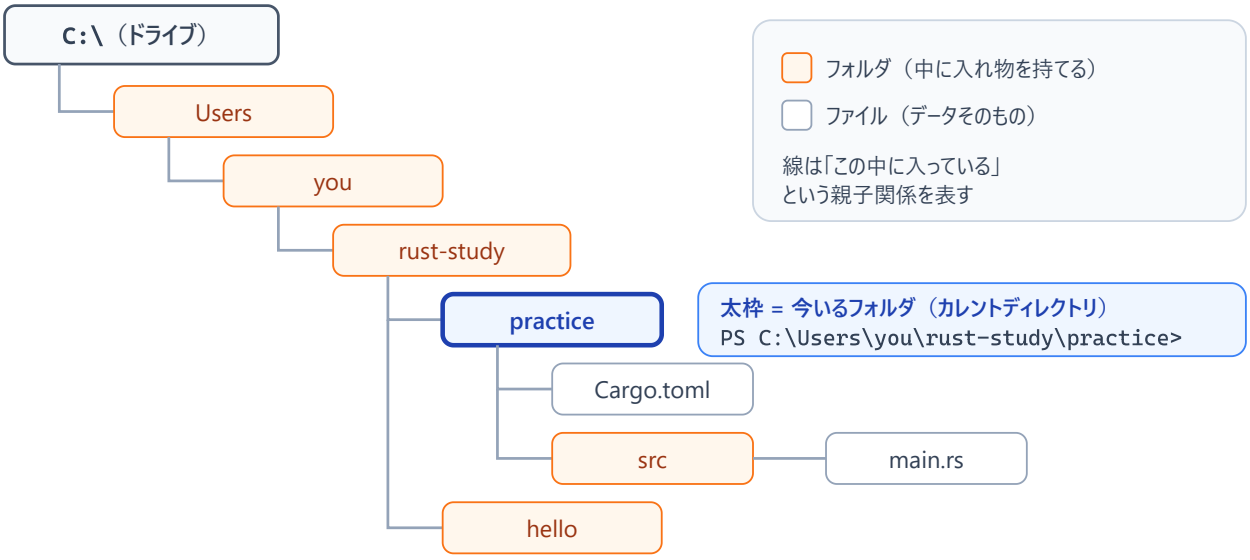


図2-1: フォルダの木（ツリー）。上が「親」、線でつながった下が「子」

同じ `main.rs` を、絶対パスと相対パスでどう書くかを比べます。今いるフォルダが `practice` のとき、次の図の3つの書き方は**どれも同じファイル**を指します。

絶対パス（どこにいても同じ場所）

```
C:\Users\you\rust-study\practice\src\main.rs
```

相対パス（今いる practice から見た道順）

```
src\main.rs      または      .\src\main.rs
```

道順に使う特別な名前

`.`	今いるフォルダ自身（「ここ」）	`..\hello` →	隣の hello フォルダ
`..`	1つ上の親フォルダ（例: practice から見た .. は rust-study）		

図2-2: 絶対パスと相対パス。相対パスは「今いるフォルダ」が出発点

> **ヒント:** Windows ではパスの区切りに `\`（円記号またはバックスラッシュ）を使い、Linux や macOS では `/`（スラッシュ）を使います。Rust のプログラムの中では、Windows でも `/` で書いたパスがたいてい通じます。日本語キーボードでは `\` が `¥` と表示されることがありますが、同じ文字です。

まず動かす最小コード

ファイルを書いて、読み戻す

> **▼ このコードがやること（先に日本語で）**：今いるフォルダの場所を表示してから、`memo` というフォルダを作り、その中に `today.txt` というファイルを書き込みます。書いたファイルをすぐに読み戻して、相対パス・絶対パス・中身を表示します。

```rust
use std::env;
use std::fs;
use std::path::Path;

fn main() -> std::io::Result<()> {
    // 今いるフォルダ（カレントディレクトリ）を調べて表示する
    let here = env::current_dir()?;
    println!("今いるフォルダ: {}", here.display());

    // memo フォルダを作る（すでにあっても OK）
    fs::create_dir_all("memo")?;

    // 「memo フォルダの中の today.txt」という相対パスを組み立てる
    let relative = Path::new("memo").join("today.txt");

    // ファイルに文字を書き込む（無ければ作り、あれば上書き）
    fs::write(&relative, "Rust の勉強を始めた\n")?;

    // 同じファイルを読み戻す
    let text = fs::read_to_string(&relative)?;

    println!("相対パス: {}", relative.display());
    println!("絶対パス: {}", here.join(&relative).display());
    print!("中身: {text}");
    Ok(())
}
```

```
cargo run
```

▼ 実行結果

```
今いるフォルダ: C:\Users\you\rust-study\practice
相対パス: memo\today.txt
絶対パス: C:\Users\you\rust-study\practice\memo\today.txt
中身: Rust の勉強を始めた
```

実行したあと、エクスプローラーで `practice` フォルダを開くと、`memo` フォルダと `today.txt` ができているはずです。

▼ コードを1つずつ分解して解説

解説1: 失敗するかもしれない処理と `?`

```
fn main() -> std::io::Result<()> {
    let here = env::current_dir()?;
    // ...
    Ok(())
}
```

- ファイルの操作は、**失敗する可能性がいつもある**処理です（ファイルが無い、書き込みが禁止されている、ディスクがいっぱい…）。
- `-> std::io::Result<()>` は「この `main` は、成功（`Ok`）か失敗（`Err`）のどちらかを返します」という宣言です。
- 処理の後ろに付けた `?`（はてな）は、「**失敗だったら、ここで止めて失敗を報告する。成功なら中身を取り出して先へ進む**」という意味の記号です。
- 最後の `Ok(())` は「最後まで来たので成功です」という報告です。
- この仕組みは第2巻・第4巻で詳しく学びます。今は「`?` は失敗したらそこで止まる印」と覚えておけば十分です。

解説2: 今いるフォルダを調べる

```
let here = env::current_dir()?;
println!("今いるフォルダ: {}", here.display());
```

- `env::current_dir()` は、**このプロセスのカレントディレクトリ**を OS に問い合わせます。
- `cargo run` で起動したプログラムは、**`cargo run` を打ったときに PowerShell がいたフォルダ**をカレントディレクトリとして受け継ぎます。
- `.display()` は、パスを画面に表示できる形にする道具です。

解説3: パスを組み立てる

```
let relative = Path::new("memo").join("today.txt");
```

- `Path::new("memo")` は「`memo` というパス」を作ります。
- `.join("today.txt")` は、その後ろに区切り文字をはさんで `today.txt` をつなぎます。
- Windows なら `memo\today.txt`、Linux なら `memo/today.txt` と、**OS に合った区切り文字**を自動で使ってくれるのが利点です。
- 区切り文字を自分で `"memo\\today.txt"` のように書くより、`join` を使うほうが安全です。

解説4: 書く・読む

```
fs::create_dir_all("memo")?;
fs::write(&relative, "Rust の勉強を始めた\n")?;
let text = fs::read_to_string(&relative)?;
```

- `fs`（ファイルシステム）は、ファイルやフォルダを扱う道具箱です。
- `create_dir_all` はフォルダを作ります。途中のフォルダが無ければそれも作り、すでにあっても失敗しません。
- `write` はファイルに書き込みます（**上書き**なので、前の中身は消えます）。
- `read_to_string` はファイル全体を文字列として読み込みます。
- `&relative` の `&` は「パスを貸すだけで、手放さない」という印です（第3巻で詳しく学びます）。
- `"\n"` は**改行**を表す特別な書き方です。

もう一歩: 実務に近い形

パスを部品に分け、「ここから見て」存在するか調べる

> **▼ このコードがやること（先に日本語で）：** `src/main.rs` というパスを「ファイル名」「拡張子」「親フォルダ」に分解して表示します。続いて、いくつかの相対パスについて「今いるフォルダから見て存在するか」を調べ、最後に今いるフォルダの中身を一覧表示します。

```rust
use std::fs;
use std::path::Path;

fn main() -> std::io::Result<()> {
    // 1 つのパスを部品に分けてみる
    let path = Path::new("src/main.rs");
    println!("パス全体  : {}", path.display());
    println!("ファイル名: {:?}", path.file_name());
    println!("拡張子    : {:?}", path.extension());
    println!("親フォルダ: {:?}", path.parent());
    println!();

    // いろいろな相対パスが「今いるフォルダから見て」存在するか調べる
    let candidates = [
        "Cargo.toml",
        "src/main.rs",
        "main.rs",
        "./src/../Cargo.toml",
    ];
    for candidate in candidates {
        let exists = Path::new(candidate).exists();
        println!("{candidate:<22} -> 存在する? {exists}");
    }
    println!();

    // 今いるフォルダの中身を一覧表示する
    println!("今いるフォルダの中身:");
    for entry in fs::read_dir(".")? {
        let entry = entry?;
        let kind = if entry.path().is_dir() {
            "フォルダ"
        } else {
            "ファイル"
        };
        println!("  [{kind}] {}", entry.file_name().display());
    }
    Ok(())
}
```

▼ **実行結果**（`practice` フォルダで `cargo run` した場合）

```
パス全体   : src/main.rs
ファイル名: Some("main.rs")
拡張子     : Some("rs")
親フォルダ: Some("src")

Cargo.toml              -> 存在する? true
src/main.rs             -> 存在する? true
main.rs                 -> 存在する? false
./src/../Cargo.toml     -> 存在する? true

今いるフォルダの中身:
  [フォルダ] .git
  [ファイル] .gitignore
  [ファイル] Cargo.lock
  [ファイル] Cargo.toml
  [フォルダ] memo
  [フォルダ] src
  [フォルダ] target
```

▼ コードを1つずつ分解して解説

解説1: パスの部品

```
println!("ファイル名: {:?}", path.file_name());
println!("拡張子     : {:?}", path.extension());
println!("親フォルダ: {:?}", path.parent());
```

- `file_name()` は最後の部品（`main.rs`）、`extension()` は拡張子（`rs`）、`parent()` は1つ上（`src`）を返します。
- 結果に `Some(...)` が付いているのは、「部品が無い場合もある」からです（例えば `C:\` には親がありません）。UNIT 01 で見た `Option` と同じ考え方です。
- `{:?}` は「プログラマー向けの形で表示する」という指定で、`Some(...)` や `"` もそのまま見せてくれます。

解説2: 相対パスは「今いるフォルダ」が出発点

```
let exists = Path::new(candidate).exists();
```

- `exists()` は、そのパスにファイルかフォルダが**本当にあるか**を OS に確かめます。
- `main.rs` が `false` なのは、`main.rs` が `practice` の直下ではなく `src` の中にあるからです。
- `./src/../Cargo.toml` は「ここ（`.`）→ `src` に入る → 1つ上（`..`）に戻る → `Cargo.toml`」という遠回りの道順で、結局 `Cargo.toml` を指すので `true` です。

解説3: フォルダの中身を一覧する

```
for entry in fs::read_dir(".")? {
    let entry = entry?;
    // ...
}
```

- `fs::read_dir(".")` は、今いるフォルダ（`.`）の中身を1つずつ取り出します。
- `.git` や `.gitignore` のように `.` で始まる名前は、Git というツールが使う設定です。エクスプローラーでは隠れていることがあります（第1巻で説明します）。
- `target` フォルダは、Cargo がコンパイルした結果を置く場所です（UNIT 04）。

よくあるエラーと読み方

ファイルが見つからない（NotFound）

> **▼ このコードがやること（先に日本語で）**：今いるフォルダにある `config.txt` を読み込んで表示しようとします。ただし、このファイルはまだ作っていません。

```
use std::fs;

fn main() -> std::io::Result<()> {
    // config.txt を読む（今いるフォルダに無ければ失敗する）
    let text = fs::read_to_string("config.txt")?;
    println!("設定: {text}");
    Ok(())
}
```

▼ 実行結果

```
Error: Os { code: 2, kind: NotFound, message: "指定されたファイルが見つかりません。" }
error: process didn't exit successfully: `target\debug\practice.exe` (exit code: 1)
```

何が起きたか:

- 1行目は、プログラム自身が出したエラーです。`?` で止まった `main` が失敗を返したので、Rust がその中身を表示しました。
- `Os { code: 2, ... }` … OS が返してきたエラーで、Windows のエラー番号は 2
- `kind: NotFound` … 種類は「見つからない」
- `message: "..."` … OS が用意した説明文（日本語版 Windows なので日本語）
- 2行目は Cargo の報告で、プロセスが終了コード 1（失敗）で終わったことを伝えています。

直し方:「ファイルが本当にあるか」と「**今いるフォルダはどこか**」の2つを確かめます。`config.txt` を作るか、ファイルのある場所で実行するか、パスを正しく書き直します。

実行する場所が変わると、相対パスの行き先も変わる

最小コード（`memo\today.txt` を書くプログラム）を、今度は `target\debug` フォルダに移動してから直接実行してみます。

```
cd target\debug
.\practice.exe
```

▼ 実行結果

```
今いるフォルダ: C:\Users\you\rust-study\practice\target\debug
相対パス: memo\today.txt
絶対パス: C:\Users\you\rust-study\practice\target\debug\memo\today.txt
中身: Rust の勉強を始めた
```

エラーにはなりませんが、`memo` フォルダが**予想と違う場所**（`target\debug` の中）にできてしまいました。プログラムは同じでも、カレントディレクトリが変わると相対パスの行き先が変わるからです。

> **つまずきポイント:**「さっきは動いたのに、今は `NotFound` になる」「保存したはずのファイルが見当たらない」というトラブルの多くは、**カレントディレクトリの取り違え**が原因です。困ったら、まず PowerShell の `PS C:\...>` の部分（今いるフォルダ）を確認しましょう。プログラム側で `env::current_dir()` を表示するのも有効です。

RustNote ではここで使う

- RustNote のプロジェクトも、`Cargo.toml`・`src` フォルダ（Rust のコード）・`migrations` フォルダ（データベースの表を作る SQL）・`tests` フォルダ（テスト）などの**決まった形のフォルダ構成**を持ちます。詳しい中身は第9巻で作ります。
- 開発中のデータベースは、**ローカルのファイル**として保存します。そのファイルの場所を相対パスで書くと、「どこで `cargo run` したか」によって別のファイルを見てしまうことがあります。RustNote では、場所を設定（環境変数）で決めるようにして、この事故を防ぎます（第9巻）。

練習問題

問題1: `memo` フォルダの中に `today.txt` と `tomorrow.txt` の2つのファイルを書き込み、`memo` フォルダの中身を一覧表示するプログラムを書いてください。

問題2: 次のパスは、絶対パスと相対パスのどちらでしょうか。

- a. `C:\Users\you\rust-study`

- b. `src\main.rs`
- c. `..\hello`
- d. `/home/you/rust-study`（Linux のパス）
- e. `.\Cargo.toml`

問題3（予想してから実行）：`practice` フォルダの中で `cd target\debug` してから `cargo run` すると、Cargo はプロジェクトを見つけられるでしょうか。見つけられるなら、最小コードの `memo` フォルダはどこにできるでしょうか。

解答と解説

問題1の解答:

```
use std::fs;
use std::path::Path;

fn main() -> std::io::Result<()> {
    fs::create_dir_all("memo")?;

    // 2 つのファイルを書く
    fs::write(Path::new("memo").join("today.txt"), "今日の予定\n")?;
    fs::write(Path::new("memo").join("tomorrow.txt"), "明日の予定\n")?;

    // memo フォルダの中身を一覧表示する
    for entry in fs::read_dir("memo")? {
        let entry = entry?;
        println!("{}", entry.path().display());
    }
    Ok(())
}
```

▼ 実行結果

```
memo\today.txt
memo\tomorrow.txt
```

`fs::read_dir("memo")` に渡したのが相対パスなので、表示されるパスも `memo\...` という相対パスになります。

問題2の解答: a は**絶対**（ドライブ名 `C:` から始まる）、b は**相対**、c は**相対**（`..` は今いる場所が出発点）、d は**絶対**（Linux では `/` から始まるパスが絶対パス）、e は**相対**（`.` は今いる場所）。

問題3の解答: Cargo は、今いるフォルダに `Cargo.toml` が無ければ**親フォルダを順にさかのぼって**探すので、プロジェクトを見つけて実行できます。ただし、プログラムのカレントディレクトリは `target\debug` のままなので、`memo` は `target\debug` の中にできます。

```
     Running `.\practice.exe`
今いるフォルダ: C:\Users\you\rust-study\practice\target\debug
相対パス: memo\today.txt
絶対パス: C:\Users\you\rust-study\practice\target\debug\memo\today.txt
中身: Rust の勉強を始めた
```

「Cargo がプロジェクトを探す場所」と「プログラムが相対パスを解決する場所」は別物だ、という点がポイントです。

まとめ

- ファイルはデータ、フォルダは入れ物で、全体は `C:\` を根元とする木（ツリー）になっています。
- 絶対パスは「どこからでも同じ場所」、相対パスは「今いるフォルダ（カレントディレクトリ）からの道順」です。`.` は「ここ」、`..` は「1つ上」。
- 「ファイルが無い」と言われたら、ファイルの有無と**今いるフォルダ**の両方を確かめます。

UNIT 03: 端末・シェル・コマンド

この単元のゴール

- **端末（ターミナル）**・**シェル**・**コマンド**の違いを説明できる
- PowerShell で、今いる場所の確認・移動・フォルダ作成・一覧・表示・コピー・削除ができる
- コマンドを「コマンド名・サブコマンド・オプション・引数」の部品に分けて読める
- **コマンドライン引数**と**環境変数**を使って、プログラムに値を渡せる
- PowerShell と bash（WSL・macOS・Linux のシェル）の書き方の違いに気づける

たとえ話でつかむ

コマンドを打つ作業は、**役所の窓口でお願いをする**ことに似ています。

- **端末（ターミナル）は窓口のカウンター**です。あなたが文字を書いて差し出し、返事の文字を受け取る場所です。Windows では「Windows Terminal（ウィンドウズ ターミナル）」というアプリがこの役目をします。
- **シェルは窓口の係員**です。あなたが書いたお願い（コマンド）を読み、「それなら `cargo` という担当に回しますね」と、OS に頼んでプログラムを起動します。Windows の **PowerShell**、Linux や macOS の **bash（バッシュ）** や **zsh（ズィーシェル）** がシェルです。
- **コマンドはお願いの文**です。決まった言い方で書かないと、係員に「そんな担当はいません」と言われてしまいます。
- PowerShell と bash は、**違う方言を話す係員**です。どちらも「フォルダの中身を見せて」というお願いを受け付けますが、細かい言い回しが違います。

マウスでアイコンをクリックする代わりに文字でお願いするのは、最初は不便に感じるかもしれません。しかし文字のお願いは、**そのままコピーして人に渡せる、同じ手順を何度でも正確にくり返せる、記録が残る**という大きな利点があります。開発

の仕事でコマンドを使うのは、このためです。

> **用語: CLI（シーエルアイ）** … Command Line Interface の略。文字のコマンドで操作するやり方のことです。マウスで操作する画面は **GUI（ジーユーアイ）** と呼びます。

図で理解する

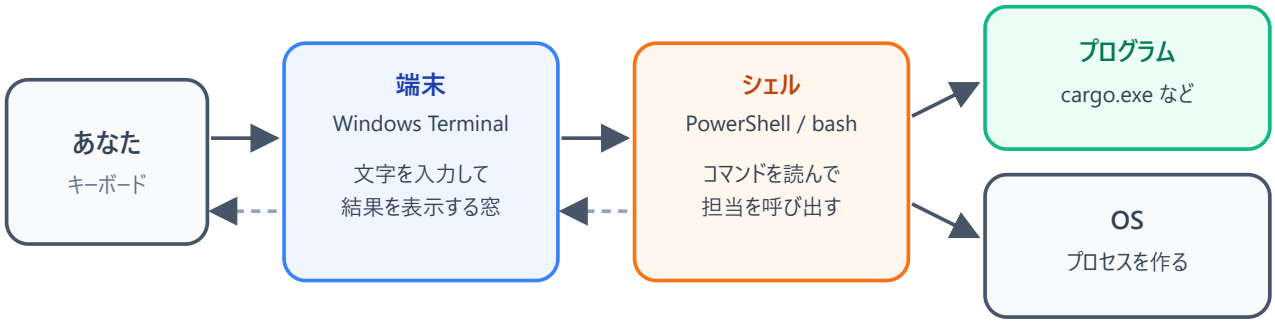


図3-1: 端末・シェル・OS の関係。シェルが OS に頼んでプログラムを起動する

実際の PowerShell の画面は次のようになります。`PS C:\Users\you>` の部分を**プロンプト**と呼び、「今いるフォルダはここで、お願いを待っています」という意味です。

```
Windows PowerShell
PS C:\Users\you> cd rust-study\practice
PS C:\Users\you\rust-study\practice> ls src
   ディレクトリ: C:\Users\you\rust-study\practice\src
Mode            LastWriteTime   Length Name
----            -------------   ------ ----
-a----    2026/10/17     11:30      689 main.rs
PS C:\Users\you\rust-study\practice> cargo --version
cargo 1.99.1 (5f94df478 2026-08-27)
PS C:\Users\you\rust-study\practice> cargo run -q -- Hanako
こんにちは、Hanako さん
PS C:\Users\you\rust-study\practice> _
```

図3-2: PowerShell の画面イメージ。黄色がプロンプト（今いるフォルダ）、白が入力と結果

コマンドは、決まった順番の部品でできています。



図3-3: コマンドの部品。「cargo に、release のやり方で run してもらい、Hanako を渡す」

まず動かす最小コード

この単元の「最小コード」は、Rust ではなく PowerShell のコマンドです。Windows のスタートメニューで「ターミナル」または「PowerShell」と検索して開き、1行ずつ打って Enter キーを押してください。

PowerShell の基本コマンドを打ってみる

> **▼ このコマンドがやること（先に日本語で）**：今いるフォルダを確かめてから、練習用のフォルダ `shell-practice` を作って中に入ります。メモのファイルを作り、一覧で確かめ、中身を表示し、コピーして、コピーを消し、最後に元のフォルダへ戻ります。

```
# 今いるフォルダを表示する
Get-Location
# shell-practice フォルダを作る
mkdir shell-practice
# その中へ移動する
cd shell-practice
# memo.txt に1行書き込む（文字コードは UTF-8）
Set-Content memo.txt "はじめての PowerShell" -Encoding utf8
# フォルダの中身を一覧する
ls
# ファイルの中身を表示する
cat memo.txt
# コピーを作って、一覧で確かめる
Copy-Item memo.txt memo-backup.txt
ls
# コピーを消す
Remove-Item memo-backup.txt
# 1つ上のフォルダへ戻る
cd ..
# Rust の道具の版を確かめる
cargo --version
rustc --version
```

▼ **実行結果**（プロンプトとコマンドも含めて、画面に出たとおりに載せています）

```
PS C:\Users\you> Get-Location

Path
----
C:\Users\you


PS C:\Users\you> mkdir shell-practice


    ディレクトリ: C:\Users\you


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        2026/10/17     11:30                shell-practice


PS C:\Users\you> cd shell-practice
PS C:\Users\you\shell-practice> Set-Content memo.txt "はじめての PowerShell" -Encoding
utf8
PS C:\Users\you\shell-practice> ls


    ディレクトリ: C:\Users\you\shell-practice


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        2026/10/17     11:30             31 memo.txt


PS C:\Users\you\shell-practice> cat memo.txt
はじめての PowerShell
PS C:\Users\you\shell-practice> Copy-Item memo.txt memo-backup.txt
PS C:\Users\you\shell-practice> ls


    ディレクトリ: C:\Users\you\shell-practice


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        2026/10/17     11:30             31 memo-backup.txt
-a----        2026/10/17     11:30             31 memo.txt


PS C:\Users\you\shell-practice> Remove-Item memo-backup.txt
PS C:\Users\you\shell-practice> cd ..
```

```
PS C:\Users\you> cargo --version
cargo 1.99.1 (5f94df478 2026-08-27)
PS C:\Users\you> rustc --version
rustc 1.99.1 (58cf8f9c3 2026-10-13)
```

`cargo --version` と `rustc --version` は、Rust をインストールしていれば版が表示されます。まだの人は、この2行だけエラーになります（下の「よくあるエラー」を見てください）。

▼ コマンドを1つずつ分解して解説

解説1: 今いる場所を知る・移動する

```
Get-Location
cd shell-practice
cd ..
```

- `Get-Location`（ゲット・ロケーション）は「今いるフォルダ」を表示します。プロンプトにも同じものが出ていますが、長くて省略されるときに便利です。短く `pwd` と打っても同じです。
- `cd`（change directory）は移動です。`cd shell-practice` は相対パス（UNIT 02）なので、「今いるフォルダの中の `shell-practice` へ」という意味です。
- `cd ..` は「1つ上のフォルダへ」です。

解説2: 作る・見る

```
mkdir shell-practice
Set-Content memo.txt "はじめての PowerShell" -Encoding utf8
ls
cat memo.txt
```

- `mkdir`（make directory）はフォルダを作ります。
- `Set-Content`（セット・コンテント）はファイルに文字を書き込みます。`-Encoding utf8` は「文字を UTF-8 という方式で保存して」というオプションです。付けないとどうなるかは「よくあるエラー」で見ます。
- `ls`（list）はフォルダの中身の一覧です。`Mode` 列の `d` はフォルダ（directory）、`a` は普通のファイルの印です。`Length` はファイルの大きさ（バイト数）です。
- `cat`（キャット）はファイルの中身を表示します。

解説3: コピーする・消す

```
Copy-Item memo.txt memo-backup.txt
Remove-Item memo-backup.txt
```

- `Copy-Item 元 先` でコピーします。
- `Remove-Item` で消します。**ごみ箱には入らず、すぐ消えます。**消すものを確かめてから打ちましょう。

> **ヒント:** コマンドを途中まで打って **Tab キー**を押すと、ファイル名やコマンド名を補完してくれます（ `memo` まで打って Tab → `memo.txt` ）。また **↑キー**で、前に打ったコマンドを呼び戻せます。打ち間違いが減るので、早めに慣れておきましょう。

解説4: PowerShell と bash の対応表

PowerShell のコマンドには `Get-Location` のような正式名と、`pwd`・`ls`・`cat` のような短い別名（**エイリアス**）があります。短い別名は bash に似せてあるので、WSL や macOS で bash を使うときにも役立ちます。

やりたいこと	PowerShell（正式名）	PowerShell（別名）	bash（WSL・macOS・Linux）
今いるフォルダを表示	`Get-Location`	`pwd`	`pwd`
移動	`Set-Location フォルダ`	`cd フォルダ`	`cd フォルダ`
一覧	`Get-ChildItem`	`ls`・`dir`	`ls`
フォルダを作る	`New-Item -ItemType Directory 名前`	`mkdir 名前`	`mkdir 名前`
中身を表示	`Get-Content ファイル`	`cat ファイル`	`cat ファイル`
コピー	`Copy-Item 元 先`	`cp 元 先`	`cp 元 先`
削除	`Remove-Item ファイル`	`rm ファイル`	`rm ファイル`
環境変数を設定	`$env:名前 = "値"`	—	`export 名前=値`
直前の終了コード	`$LASTEXITCODE`	—	`$?`

もう一歩: 実務に近い形

コマンドからプログラムに値を渡す方法は、主に2つあります。

- **コマンドライン引数** … コマンドの後ろに並べる値（ `cargo run -- Rust` ）。そのとき限りの値を渡すのに使います。
- **環境変数** … シェルが覚えておいて、そこから起動したプログラムすべてに見せる「名前付きの値」。設定や秘密の鍵など、**コードに書き込みたくない値**を渡すのによく使います。

> **用語: 環境変数** … OS やシェルが持っている「名前＝値」のメモ。たとえば `PATH` という環境変数には「コマンドのプログラムを探しに行くフォルダの一覧」が入っています。`cargo` と打つだけで動くのは、`PATH` に `C:\Users\you\.cargo\bin` が入っているからです。

引数と環境変数を受け取る

> **▼ このコードがやること（先に日本語で）**：起動するときに渡された値（引数）を、番号つきで全部表示します。そのあと、環境変数 `GREETING` の値を読み、設定されていなければ「こんにちは」を使って表示します。

```rust
use std::env;

fn main() {
    // コマンドラインから渡された値を順番に表示する（0 番目はプログラム自身）
    for (i, arg) in env::args().enumerate() {
        println!("引数 {i}: {arg}");
    }

    // 環境変数 GREETING を読む（設定されていなければ「こんにちは」を使う）
    let greeting = env::var("GREETING").unwrap_or(String::from("こんにちは"));
    println!("あいさつ: {greeting}");
}
```

PowerShell で、次の順に実行します。

```
cargo run -q -- Rust "Web アプリ"
$env:GREETING = "おはよう"
cargo run -q
.\target\debug\practice.exe 1 2 3
Remove-Item Env:GREETING
cargo run -q
```

▼ 実行結果

```
PS C:\Users\you\rust-study\practice> cargo run -q -- Rust "Web アプリ"
引数 0: target\debug\practice.exe
引数 1: Rust
引数 2: Web アプリ
あいさつ: こんにちは
PS C:\Users\you\rust-study\practice> $env:GREETING = "おはよう"
PS C:\Users\you\rust-study\practice> cargo run -q
引数 0: target\debug\practice.exe
あいさつ: おはよう
PS C:\Users\you\rust-study\practice> .\target\debug\practice.exe 1 2 3
引数 0: C:\Users\you\rust-study\practice\target\debug\practice.exe
引数 1: 1
引数 2: 2
引数 3: 3
あいさつ: おはよう
PS C:\Users\you\rust-study\practice> Remove-Item Env:GREETING
PS C:\Users\you\rust-study\practice> cargo run -q
引数 0: target\debug\practice.exe
あいさつ: こんにちは
```

▼ コードを1つずつ分解して解説

解説1: 引数を番号つきで取り出す

```rust
for (i, arg) in env::args().enumerate() {
    println!("引数 {i}: {arg}");
}
```

- `env::args()` は引数の並びです（UNIT 01 でも使いました）。
- `.enumerate()` は、並びの1つずつに `0, 1, 2, …` の番号を付けてくれます。`(i, arg)` で「番号」と「値」を同時に受け取っています。
- 0番目はプログラム自身のパスです。`cargo run` 経由だと `target\debug\practice.exe`、直接起動すると絶対パスになっています。どう起動されたかによって変わる、ということです。
- `"Web アプリ"` のように**空白を含む値は `"` で囲む**と、1つの引数として渡せます。囲まないと `Web` と `アプリ` の2つに分かれます。

解説2: 環境変数を読む

```rust
let greeting = env::var("GREETING").unwrap_or(String::from("こんにちは"));
```

- `env::var("GREETING")` は環境変数 `GREETING` を読みます。設定されていないこともあるので、結果は成功か失敗のどちらかです。
- `.unwrap_or(既定値)` は「失敗だったら既定値を使う」という意味です。

- PowerShell で `$env:GREETING = "おはよう"` と設定すると、**その PowerShell の窓から起動したプログラム**にだけ見えます。窓を閉じると消え、他の窓には影響しません。
- `Remove-Item Env:GREETING` で設定を消せます。

> **補足（bash の場合）**: WSL や macOS の bash では、`GREETING=おはよう cargo run -q -- Rust` のように、コマンドの前に `名前=値` を書くと「そのコマンドにだけ」環境変数を渡せます。ずっと設定しておくなら `export GREETING=おはよう` です。

よくあるエラーと読み方

コマンドの名前を打ち間違えた

```
PS C:\Users\you> carg run
carg : 用語 'carg' は、コマンドレット、関数、スクリプト ファイル、または操作可能なプログラムの名前
として認識されません。名前が正しく記述されていることを確認し、パスが含まれている場合は
そのパスが正しいことを確認してから、再試行してください。
発生場所 行:1 文字:1
+ carg run
+ ~~~~
    + CategoryInfo          : ObjectNotFound: (carg:String) [],
CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

何が起きたか: シェル（係員）が `carg` という担当を探しましたが、見つかりませんでした。`発生場所 行:1 文字:1` と `~~~~` の印が、「1行目の1文字目からの `carg` の部分がおかしい」と指しています。

直し方: 綴りを直して `cargo run` と打ちます。綴りが正しいのに同じエラーになる場合は、**そのプログラムがインストールされていない**か、`PATH` にフォルダが入っていません。Rust をインストールした直後なら、**PowerShell の窓を開き直す**と直ることがよくあります（`PATH` の変更は、新しく開いた窓から有効になるためです）。

bash の書き方を PowerShell で打った

```
PS C:\Users\you> export GREETING=hello
export : 用語 'export' は、コマンドレット、関数、スクリプト ファイル、または操作可能なプログラムの
名前として認識されません。名前が正しく記述されていることを確認し、パスが含まれてい
る場合はそのパスが正しいことを確認してから、再試行してください。
発生場所 行:1 文字:1
+ export GREETING=hello
+ ~~~~~~
    + CategoryInfo          : ObjectNotFound: (export:String) [],
CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

何が起きたか: `export` は bash の言葉なので、PowerShell の係員には通じません。インターネットの記事をコピーして貼ったときによく起きます。

直し方: PowerShell では `$env:GREETING = "hello"` と書きます。記事のコマンドを使うときは、**それが PowerShell 向けか bash 向けか**を最初に確かめましょう。この教材では、PowerShell 向けのコードブロックには `powershell`、bash 向けには `bash` の印を付けています。

`curl` が思った動きをしない

Web のやりとりを見る道具 `curl`（カール）は UNIT 05 で使いますが、Windows PowerShell 5.1 には落とし穴があります。

```
PS C:\Users\you> Get-Command curl

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------
Alias           curl -> Invoke-WebRequest


PS C:\Users\you> Get-Command curl.exe

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------
Application     curl.exe                                           8.21.0.0
C:\Windows\system32\curl.exe
```

何が起きたか: PowerShell 5.1 では、`curl` は **`Invoke-WebRequest` という別のコマンドの別名**になっています。そのため、インターネットの記事どおりに `curl -I ...` などと打っても、オプションの意味が違って、エラーになったり入力待ちで止まったりします。

直し方: 本物の curl を使うときは、**`curl.exe` と拡張子まで書きます**。この教材ではいつも `curl.exe` と書きます。

PowerShell で作ったファイルを Rust で読めない

`-Encoding utf8` を付けずに `Set-Content` で作ったファイルを、Rust で読んでみます。

```rust
use std::fs;

fn main() -> std::io::Result<()> {
    // memo.txt を UTF-8 の文字列として読み込む
    let text = fs::read_to_string("memo.txt")?;
    println!("memo.txt の中身: {}", text.trim_end());
    Ok(())
}
```

```
Set-Content memo.txt "はじめての PowerShell"
cargo run
```

▼ 実行結果

```
Error: Error { kind: InvalidData, message: "stream did not contain valid UTF-8" }
error: process didn't exit successfully: `target\debug\practice.exe` (exit code: 1)
```

何が起きたか: 文字をファイルに保存するときの「文字と数値の対応表」を**文字コード**と呼びます。Windows PowerShell 5.1 の `Set-Content` は、何も指定しないと日本語を **Shift_JIS（シフトジス）** という古い文字コードで保存します。一方 Rust の `read_to_string` は **UTF-8（ユーティーエフエイト）** しか受け付けないので、「中身が正しい UTF-8 ではない（InvalidData）」と失敗しました。

直し方: `Set-Content` に `-Encoding utf8` を付けるか、ファイルは VS Code などのエディタ（既定で UTF-8）で作ります。

> **注意:** PowerShell 5.1 の `-Encoding utf8` は、ファイルの先頭に **BOM（ボム）** という目に見えない印を付けます。Rust で読むと、その印が文字列の先頭に1文字ぶん残ります。画面上は見えませんが、「先頭の文字を比べたら一致しない」といった不思議な不具合の原因になることがあります。迷ったら、ファイルはエディタで UTF-8（BOM なし）として作るのがいちばん確実です。

RustNote ではここで使う

- RustNote を作る間、`cargo`（ビルド・実行・テスト）、`git`（変更の記録）、`turso`（データベースの操作）、`fly`（公開）といったコマンドを使います。どれも「コマンド名・サブコマンド・オプション・引数」の形です（第1巻でインストールします）。
- Topcoat で作ったサーバーは、環境変数 `HOST` と `PORT` で待ち受ける場所を決めます。手元では何も設定せずに `127.0.0.1:3000`、Fly.io の上では `HOST=0.0.0.0`・`PORT=8080` を設定します（UNIT 06・第12巻）。
- データベースの場所やパスワード類も、コードに直接書かず**環境変数**で渡します。コードを公開しても秘密が漏れないようにするためです（第4巻・第9巻）。

練習問題

問題1: `practice` フォルダにいるとき、`src` フォルダの中身を一覧するコマンドは何でしょうか。また、今いるフォルダを確かめるコマンドを2通り答えてください。

問題2: 「もう一歩」のプログラムで「あいさつ: おはよう」と表示させる手順を、PowerShell と bash の両方で書いてください。

問題3: 引数が何個渡されたか（プログラム自身は数えない）を表示するプログラムを書いてください。`cargo run -q -- a "b c" d` で「渡された値は 3 個です」と表示されれば正解です。

解答と解説

問題1の解答: 一覧は `ls src`（正式名なら `Get-ChildItem src`）です。今いるフォルダの確認は `Get-Location` と `pwd` の2通りです。

```
PS C:\Users\you\rust-study\practice> ls src


    ディレクトリ: C:\Users\you\rust-study\practice\src


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        2026/10/15     20:47            689 main.rs
```

問題2の解答:

```
# PowerShell
$env:GREETING = "おはよう"
cargo run -q
```

```
# bash（WSL・macOS・Linux）
GREETING=おはよう cargo run -q
```

bash で実行した結果です（引数に `Rust` も渡しています）。

```
引数 0: target\debug\practice.exe
引数 1: Rust
あいさつ: おはよう
```

問題3の解答:

```
use std::env;

fn main() {
    // 0 番目（プログラム自身）を skip(1) で飛ばしてから数える
    let count = env::args().skip(1).count();
    println!("渡された値は {count} 個です");
}
```

```
cargo run -q -- a "b c" d
```

▼ 実行結果

```
渡された値は 3 個です
```

`"b c"` は `"` で囲んでいるので1個として数えられます。`skip(1)` は「先頭から1個飛ばす」、`count()` は「残りの個数を数える」という意味です。

まとめ

- 端末は文字の窓、シェルはコマンドを読んで OS にプログラムの起動を頼む係員、コマンドは「名前・サブコマンド・オプション・引数」の形のお願いです。
- プログラムへは**コマンドライン引数**（そのときの値）と**環境変数**（設定・秘密）で値を渡せます。
- PowerShell と bash は方言が違います。記事のコマンドがどちら向けかを確かめ、Windows では `curl.exe` と書き、ファイルは UTF-8 で作りましょう。

UNIT 04: ソースコードから実行まで

この単元のゴール

- **ソースコード・コンパイラ・実行ファイル**の関係を、流れ図で説明できる
- `rustc`（コンパイラ）と `cargo`（プロジェクト管理の道具）の役割の違いが分かる
- `cargo new`・`cargo run`・`cargo build`・`cargo check`・`--release` を使い分けられる
- **コンパイル時の失敗**と**実行時の失敗**を区別できる

たとえ話でつかむ

プログラムが動くまでの流れは、**本の翻訳と出版**にたとえられます。

- **ソースコードは日本語の原稿**です。人間が読み書きしやすい言葉（Rust）で書かれていますが、コンピュータ（CPU）はこのままでは読めません。CPU が読めるのは **機械語**（0 と 1 の並び）だけです。
- **コンパイラ（rustc）は翻訳者**です。原稿を機械語に翻訳します。ただし Rust の翻訳者はとても厳しく、**翻訳する前に原稿を隅々まで校閲**します。「ここで数字と文字を足していますよ」「この変数は書き換え禁止ですよ」と矛盾を1つでも見つけると、翻訳を断って赤ペンの指摘（**コンパイルエラー**）を返します。
- **実行ファイル（.exe）は翻訳済みの本**です。CPU がそのまま読めるので、翻訳者がいなくても何度でも読めます（実行できます）。
- **Cargo（カーゴ）は編集者**です。原稿の置き場所を決め、翻訳者に依頼し、必要な参考資料（他の人が作った部品＝**クレート**）を取り寄せ、できた本を決まった棚にしまってくれます。私たちはふだん編集者の Cargo に頼み、翻訳者の rustc を直接呼ぶことはあまりありません。

> **用語: コンパイル** … ソースコードを、CPU が実行できる形（機械語）に翻訳すること。翻訳する道具を **コンパイラ** と呼びます。Rust のコンパイラの名前は `rustc`（ラストシー）です。
>
> **用語: クレート（crate）** … Rust のプログラムや部品のひとまとまり。「木箱」という意味です。他の人が作ったクレートは crates.io というサイトで公開されていて、Cargo が自動で取り寄せてくれます（Topcoat もクレートの1つです）。

図で理解する

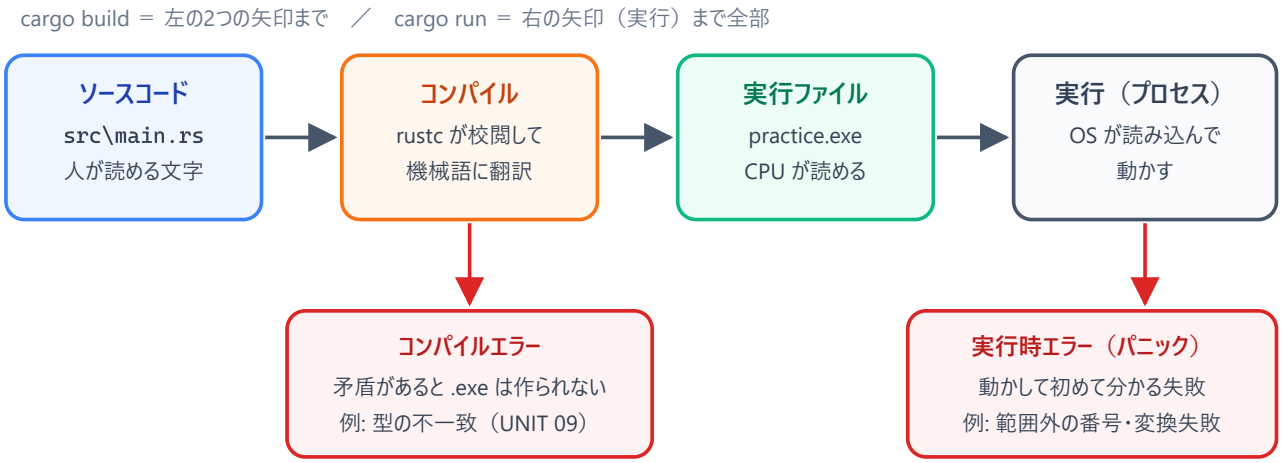


図4-1: ソースコードが実行されるまで。失敗には「コンパイル時」と「実行時」の2種類がある

Cargo のプロジェクトは、次のような決まった形のフォルダになっています。

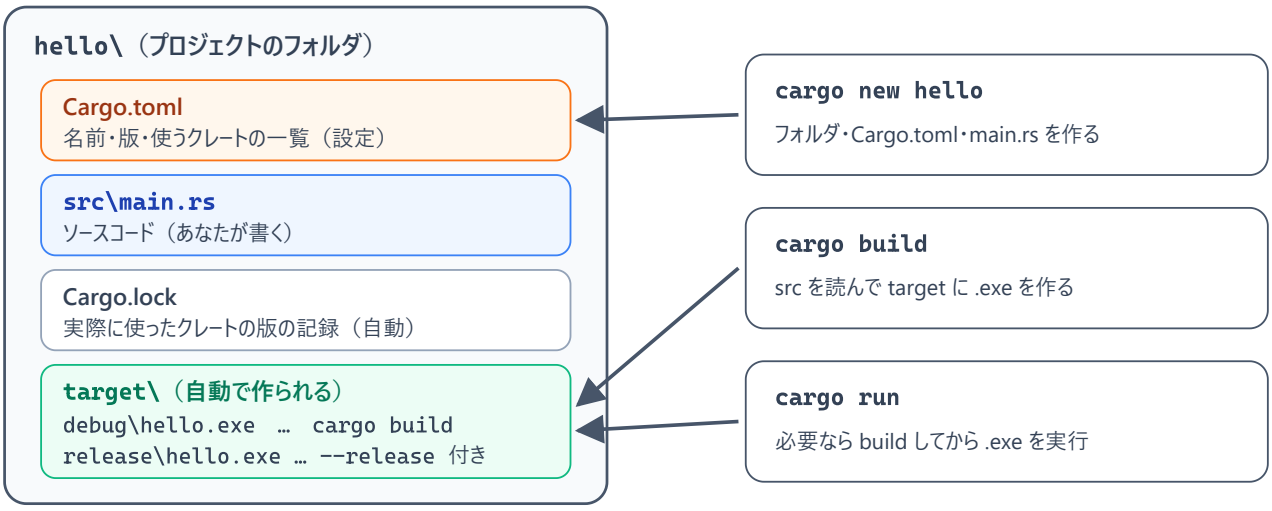


図4-2: Cargo のプロジェクトの形と、主なコマンドが触る場所

まず動かす最小コード

`cargo new` でプロジェクトを作り、`cargo run` で動かす

> **▼ このコマンドがやること（先に日本語で）：** `hello` という名前の Rust プロジェクトを作り、中に入って、最初から用意されているプログラムをコンパイルして実行します。もう一度実行して、2回目は翻訳（コンパイル）が省かれることも確かめます。

UNIT 03 で作った `rust-study` フォルダの中で、次のコマンドを打ちます。

```
cargo new hello
cd hello
cat Cargo.toml
cat src\main.rs
cargo run
cargo run
```

▼ 実行結果

```
PS C:\Users\you\rust-study> cargo new hello
    Creating binary (application) `hello` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html
PS C:\Users\you\rust-study> cd hello
PS C:\Users\you\rust-study\hello> cat Cargo.toml
[package]
name = "hello"
version = "0.1.0"
edition = "2024"

[dependencies]
PS C:\Users\you\rust-study\hello> cat src\main.rs
fn main() {
    println!("Hello, world!");
}
PS C:\Users\you\rust-study\hello> cargo run
   Compiling hello v0.1.0 (C:\Users\you\rust-study\hello)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.30s
     Running `target\debug\hello.exe`
Hello, world!
PS C:\Users\you\rust-study\hello> cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
     Running `target\debug\hello.exe`
Hello, world!
```

▼ 出力を1つずつ分解して解説

解説1: `cargo new` が作ったもの

```
    Creating binary (application) `hello` package
```

- 「`hello` という名前の、**実行できるアプリ**（binary）のパッケージを作りました」という報告です。
- 2行目の `note:` は「`Cargo.toml` に書ける設定の一覧はこのページにあります」という案内で、エラーではありません。
- 作られたのは `Cargo.toml`・`src\main.rs`・`.gitignore`（Git 用の設定）と、隠しフォルダの `.git` です。

解説2: `Cargo.toml` はプロジェクトの「表紙」

```
[package]
name = "hello"
version = "0.1.0"
edition = "2024"

[dependencies]
```

- `[package]` の下はこのプロジェクト自身の情報です。`name` は名前、`version` はこのプログラムの版です。
- `edition = "2024"` は「Rust 2024 年版の文法で書いています」という宣言です。Rust は数年ごとに書き方の細かい決まりを見直していて、このシリーズは最新の 2024 年版を使います。
- `[dependencies]`（依存関係）の下には、使いたいクレートを書きます。今は空です。第6巻では、ここに Topcoat が加わります。

> **用語: TOML（トムル）** … 設定ファイルの書き方の1つ。`名前 = 値` を並べ、`[見出し]` でまとまりを作ります。

解説3: `cargo run` の3行の報告

```
   Compiling hello v0.1.0 (C:\Users\you\rust-study\hello)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.30s
     Running `target\debug\hello.exe`
```

- `Compiling` …「`hello` の 0.1.0 版をコンパイルしています」。
- `Finished` …「終わりました」。`dev` プロファイルは**開発用の設定**で、`unoptimized`（最適化なし）・`debuginfo`（調査用の情報つき）という意味です。かかった時間は 0.30 秒でした。
- `Running` …「できた `target\debug\hello.exe` を実行します」。この後の `Hello, world!` が、プログラム自身の出力です。
- 2回目は `Compiling` の行がありません。ソースコードが変わっていないので、Cargo が「翻訳し直す必要なし」と判断し、前回の `.exe` をそのまま使ったからです。

解説4: できた実行ファイルを確かめる

`cargo run` の後に一覧を見ると、`target` フォルダと `Cargo.lock` が増えています。

```
PS C:\Users\you\rust-study\hello> ls


    ディレクトリ: C:\Users\you\rust-study\hello


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        2026/10/17     11:30                src
d-----        2026/10/17     11:30                target
-a----        2026/10/17     11:30              8 .gitignore
-a----        2026/10/17     11:30            149 Cargo.lock
-a----        2026/10/17     11:30             76 Cargo.toml


PS C:\Users\you\rust-study\hello> .\target\debug\hello.exe
Hello, world!
```

- `target\debug\hello.exe` が翻訳済みの本です。**Cargo を通さずに直接実行しても動きます。**
- 実行ファイルは、そのまま別の Windows PC にコピーしても動きます（同じ種類の CPU なら）。Rust を入れていない PC でも動くのが、コンパイルする言語の特徴です。
- `target` フォルダは何度でも作り直せるので、Git での記録の対象外です（`.gitignore` にそう書かれています）。

もう一歩: 実務に近い形

開発用（debug）と本番用（release）の速さを比べる

> **▼ このコードがやること（先に日本語で）**：1 から 2 億までの数について「7 で割った余り」を足し合わせ、かかった時間を表示します。同じコードを、開発用の設定と本番用の設定でコンパイルして、速さを比べます。

```rust
use std::hint::black_box;
use std::time::Instant;

fn main() {
    // 計算を始めた時刻を覚えておく
    let start = Instant::now();

    // 2 億回のくり返しで、1 から順に「7 で割った余り」を足していく
    let mut total: u64 = 0;
    for n in 1..=200_000_000u64 {
        // black_box は「この計算を省略しないで」とコンパイラに伝えるための道具
        total += black_box(n) % 7;
    }

    // かかった時間を表示する
    println!("合計: {total}");
    println!("かかった時間: {:?}", start.elapsed());
}
```

```
cargo run
cargo run --release
```

▼ **実行結果**（`Compiling` などの行も載せています）

```
   Compiling practice v0.1.0 (C:\Users\you\rust-study\practice)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.35s
     Running `target\debug\practice.exe`
合計: 599999998
かかった時間: 1.3268229s
   Compiling practice v0.1.0 (C:\Users\you\rust-study\practice)
    Finished `release` profile [optimized] target(s) in 0.71s
     Running `target\release\practice.exe`
合計: 599999998
かかった時間: 545.5482ms
```

答えは同じですが、`--release` 付きは2倍以上速くなりました（時間は PC や、そのとき他に動いているアプリによって変わります）。

▼ コードを1つずつ分解して解説

解説1: 時間を計る

```
let start = Instant::now();
// ...
println!("かかった時間: {:?}", start.elapsed());
```

- `Instant::now()` は「今この瞬間」を記録します。`start.elapsed()` は「`start` から何秒たったか」です。
- `{:?}` で表示すると、`1.3268229s`（秒）や `545.5482ms`（ミリ秒＝1000分の1秒）のように単位つきで出ます。

解説2: 大きな数と型

```
let mut total: u64 = 0;
for n in 1..=200_000_000u64 {
```

- `u64` は「0 以上の整数で、とても大きな数まで入る型」です。合計が約 6 億になるので、大きめの型を選んでいます。型は第2巻で学びます。
- `200_000_000` の `_` は読みやすくするための区切りで、`200000000` と同じです。

解説3: debug と release の違い

	`cargo build`（debug）	`cargo build --release`
置き場所	`target\debug\`	`target\release\`
コンパイルの速さ	速い	遅め（最適化に時間をかける）
実行の速さ	遅め	速い
使う場面	開発中、何度も直して試すとき	本番に出すとき、速さを測るとき

- **最適化**とは、コンパイラが「同じ結果になる、もっと速いやり方」に機械語を作り変えることです。
- `black_box` は、計算を丸ごと省略される（最適化されすぎる）のを防ぐための道具です。速さ比べの実験でだけ使います。

コンパイラを直接呼ぶ: `rustc`

Cargo は裏で `rustc` を呼んでいます。1ファイルだけなら、`rustc` を直接使うこともできます。

```
fn main() {
    println!("rustc だけでコンパイルしました");
}
```

この内容で `main.rs` を作ったフォルダで、次を実行します。

```
rustc main.rs
ls
.\main.exe
```

▼ 実行結果

```
PS C:\Users\you\rust-study\rustc-only> rustc main.rs
PS C:\Users\you\rust-study\rustc-only> ls


    ディレクトリ: C:\Users\you\rust-study\rustc-only


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         2026/10/17     11:30         131072 main.exe
-a----         2026/10/17     11:30        1396736 main.pdb
-a----         2026/10/15     20:38             74 main.rs


PS C:\Users\you\rust-study\rustc-only> .\main.exe
rustc だけでコンパイルしました
```

`rustc` は**何も表示せずに**終わり、`main.exe` ができました。成功したときに黙っているのは、多くのコマンドに共通する習慣です。`main.pdb` は、問題を調べるときに使う補助のファイルです。実際の開発では、クレートの取り寄せや設定の管理をしてくれる Cargo を使います。

よく使う Cargo のコマンド

コマンド	すること	いつ使うか
`cargo new 名前`	新しいプロジェクトを作る	始めるとき
`cargo run`	コンパイルして実行する	動かして確かめるとき
`cargo build`	コンパイルだけする	実行ファイルだけ欲しいとき
`cargo check`	校閲（エラー探し）だけして、`.exe` は作らない	書きながら何度も確かめるとき（いちばん速い）
`cargo test`	テストを実行する	直したら毎回（UNIT 10）
`cargo fmt`	コードの見た目（字下げなど）を整える	保存する前
`cargo clippy`	「もっと良い書き方」を助言してもらう	区切りのいいところで

よくあるエラーと読み方

プロジェクトの外で `cargo run` した

```
PS C:\Users\you\rust-study> cargo run
error: could not find `Cargo.toml` in `C:\Users\you\rust-study` or any parent directory
```

何が起きたか:「`C:\Users\you\rust-study` にも、その親のフォルダにも `Cargo.toml` が見つからない」というエラーです。Cargo は `Cargo.toml` のある場所をプロジェクトとみなすので、プロジェクトの**外**にいると何をコンパイルすればよいか分かりません。

直し方: `cd hello` のように、`Cargo.toml` のあるフォルダへ移動してから実行します。プロンプトで今いる場所を確かめましょう（UNIT 02・03）。

コンパイルは通ったのに、実行して止まった

> **▼ このコードがやること（先に日本語で）:** 文字の `"42"` と `"4x2"` を、それぞれ数値に変換しようとします。1つ目は成功しますが、2つ目は数値ではないので、実行中に失敗します。

```
fn main() {
    // 文字列を数値に変換する。どちらもコンパイルは通る
    let ok: i32 = "42".parse().unwrap();
    println!("1 つ目: {ok}");

    // "4x2" は数値ではないので、実行してみて初めて失敗が分かる
    let ng: i32 = "4x2".parse().unwrap();
    println!("2 つ目: {ng}");
}
```

▼ 実行結果

```
1 つ目: 42

thread 'main' (53536) panicked at src\main.rs:7:33:
called `Result::unwrap()` on an `Err` value: ParseIntError { kind: InvalidDigit }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
error: process didn't exit successfully: `target\debug\practice.exe` (exit code: 101)
```

何が起きたか:

- コンパイルは成功し、1つ目の `println!` までは実行されました（`1 つ目: 42`）。

- 7行目で、`"4x2"` を数値にしようとして失敗しました。`.parse()` は「成功（`Ok`）か失敗（`Err`）」を返し、`.unwrap()` は「成功なら中身を取り出す、**失敗ならパニックする**」という道具です。
- `ParseIntError { kind: InvalidDigit }` は「整数への変換エラー、種類は『数字でない文字がある』」という意味です。

直し方: 「失敗するかもしれない」と分かっている処理では、`unwrap()` を使わず、成功と失敗の両方の場合を書きます（練習問題3）。フォームに入力された値などは、利用者が何を打つか分からないので、必ずこの書き方をします。

> **つまずきポイント:** 「コンパイルが通った ＝ 正しく動く」ではありません。コンパイラが見つけるのは**書き方の矛盾**（型が合わない、変数が無い など）で、**実行してみないと分からない値の問題**（ファイルが無い、入力が数字でない など）は、実行時の失敗になります。Rust はコンパイル時に見つけられる問題を最大限見つけてくれるので、実行時の失敗はかなり少なくなります。

RustNote ではここで使う

- 開発中は `cargo run` で RustNote のサーバーを起動し、ブラウザで `http://127.0.0.1:3000` を開いて確かめます。Topcoat には、ファイルを保存するたびに自動でビルドし直してくれる `topcoat dev` というコマンドもあります（第6巻）。
- `Cargo.toml` の `[dependencies]` に、Topcoat・データベース用のクレート・パスワード処理用のクレートなどを書きます。どのクレートのどの版を使うかは第9巻で決めます。
- 公開するときは `cargo build --release` で本番用の実行ファイルを作り、Docker という箱に入れて Fly.io に送ります（第12巻）。

練習問題

問題1: `cargo new greet` を実行すると、どんなファイルとフォルダが作られるでしょうか。また、`cargo run` の後に増えるものは何でしょうか。

問題2: 次の場面では、`cargo build`・`cargo run`・`cargo check` のどれを使うのがよいでしょうか。

- a. コードを書いている途中で、エラーが無いかだけを素早く確かめたい
- b. 書き終えたので、動かして結果を見たい
- c. 実行ファイルだけを作って、あとで別の PC で動かしたい

問題3: 「よくあるエラー」の `"4x2"` の例を、パニックせずに「変換できません」と表示して最後まで動くように書き直してください。

解答と解説

問題1の解答: `greet` フォルダの中に `Cargo.toml`・`src\main.rs`・`.gitignore`（と隠しフォルダ `.git`）が作られます。`cargo run` の後には、コンパイル結果の入った `target` フォルダと、使ったクレートの版を記録する `Cargo.lock` が増えます。

問題2の解答: a は `cargo check`（`.exe` を作らないぶん最も速い）、b は `cargo run`、c は `cargo build`（本番用なら `cargo build --release`）。

問題3の解答:

```
fn main() {
    for text in ["42", "4x2"] {
        // parse の結果は「成功(Ok)」か「失敗(Err)」のどちらか
        match text.parse::<i32>() {
            Ok(n) => println!("{text} -> 数値 {n}"),
            Err(e) => println!("{text} -> 変換できません（{e}）"),
        }
    }
}
```

▼ 実行結果

```
42 -> 数値 42
4x2 -> 変換できません（invalid digit found in string）
```

`match` で `Ok` と `Err` の両方の場合を書いたので、失敗しても止まらずに最後まで進みます。`{e}` には、エラーの説明文「invalid digit found in string（文字列の中に数字でない文字があった）」が入ります。`parse::<i32>()` の `::<i32>` は「`i32`（整数）に変換して」という指定です。

まとめ

- ソースコードは rustc が**校閲してから**機械語に翻訳し、実行ファイルができます。Cargo はその作業全体を取り仕切る編集者です。
- `cargo run` は「必要ならコンパイル → 実行」、`cargo check` は校閲だけ、`--release` は速い本番用です。
- 失敗には**コンパイル時**（書き方の矛盾、`.exe` が作られない）と**実行時**（パニックなど、動かして初めて分かる）の2種類があります。

UNIT 05: ブラウザ・URL・HTTP

この単元のゴール

- URL を「スキーム・ホスト・ポート・パス・クエリ」の部品に分けて読める
- ブラウザに URL を入れてから画面が出るまでに起きることを、順番に説明できる
- HTTP の**リクエスト**と**レスポンス**の中身（メソッド・パス・ヘッダー・本文・ステータスコード）を読める
- `curl.exe` を使って、ブラウザの裏で行われているやりとりを自分の目で見られる

たとえ話でつかむ

Web のしくみは、**決まった書式の注文書を郵送して、返事をもらう**ことにそっくりです。

- **URL は宛先**です。「どの会社（ホスト）の、どの窓口（ポート）の、どの担当（パス）に、どんな条件（クエリ）で」お願いするかが全部書いてあります。
- **DNS（ディーエヌエス）は電話帳**です。`example.com` のような会社名を、郵便で実際に使う番地（**IP アドレス**、例 `172.66.147.243`）に引き直してくれます。
- **HTTP リクエストは注文書**です。1行目に「何をしてほしいか（例: `GET /notes` ＝ ノートの一覧を見せて）」、続けて添え状（**ヘッダー**）、必要なら中身（**本文**）を書きます。書式が決まっているので、世界中のどのサーバーにも通じます。
- **HTTP レスポンスは返事**です。1行目に結果のスタンプ（**ステータスコード**、例 `200 OK` ＝ 成功、`404 Not Found` ＝ 見つからない）、続けて添え状、最後に中身（多くは **HTML**）が入っています。
- **ブラウザは秘書**です。注文書を書いて送り、届いた HTML を読んで、文字や色や画像をきれいに並べた画面にしてくれます。

> **用語: HTTP（エイチティーティーピー）** … HyperText Transfer Protocol の略。ブラウザとサーバーがやりとりするときの「注文書と返事の書き方の決まり（**プロトコル**）」です。**HTTPS** は、そのやりとりを暗号化して、途中で盗み見や書き換えをされないようにしたものです。
>
> **用語: HTML（エイチティーエムエル）** … Web ページの中身を書く言葉。`<h1>見出し</h1>` のように、タグで文章の役割を示します。

図で理解する

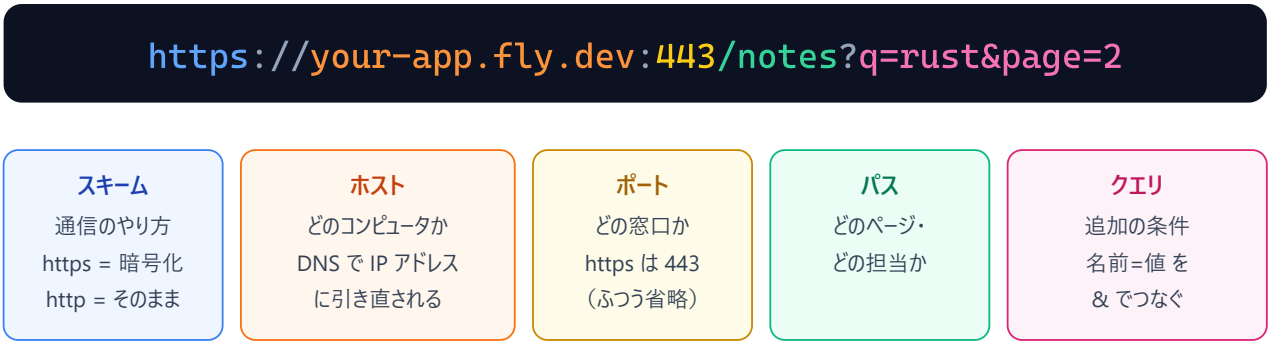


図5-1: URL の分解。「rust を含むノートの2ページ目を見せて」というお願いになっている



図5-2: URL を入れてから画面が出るまで。不具合はこの①〜⑥のどこで起きたかで切り分ける

HTTP のリクエストとレスポンスは、どちらも「1行目・ヘッダー・空行・本文」という同じ形をしています。

リクエスト（ブラウザ → サーバー）

```
POST /notes HTTP/1.1
Host: your-app.fly.dev
Cookie: session=...
Content-Type: application/
x-www-form-urlencoded
（空行 = ヘッダーの終わり）
title=牛乳を買う&body=
```

1行目: メソッド・パス・HTTP の版
本文: フォームに入力した内容

レスポンス（サーバー → ブラウザ）

```
HTTP/1.1 200 OK
Content-Type: text/html;
charset=utf-8
Content-Length: 1834
（空行 = ヘッダーの終わり）
<!doctype html><html>...
```

1行目: HTTP の版・ステータスコード
本文: 画面に描く HTML

図5-3: HTTP のリクエストとレスポンスの形。どちらも「1行目・ヘッダー・空行・本文」（値は例）

メソッドは「何をしてほしいか」を表す動詞です。RustNote で使うのは主に次の2つです。

メソッド	意味	例
`GET`	見せて（データを変えない）	`GET /notes` ノート一覧を見せて
`POST`	送るので処理して（データが変わる）	`POST /notes` このノートを作って

ステータスコードは返事の結果を表す3桁の数字で、最初の桁でおおまかな意味が分かります。

コード	名前	意味	RustNote での例
`200`	OK	成功	一覧ページを返した
`303`	See Other	「処理したので、こちらの URL を見に行って」	ノート作成後に一覧へ移動させる
`400`	Bad Request	リクエストの形がおかしい	送られてきた値が読めない
`401` / `403`	Unauthorized / Forbidden	ログインが必要／権限がない	（UNIT 08 で説明）
`404`	Not Found	見つからない	存在しない、または**他人の**ノート
`500`	Internal Server Error	サーバー側の失敗	プログラムの不具合

2xx は成功、3xx は「別の場所へ」、4xx は**頼んだ側**の問題、5xx は**サーバー側**の問題、と覚えておくと便利です。

まず動かす最小コード

`curl.exe` でブラウザの裏側をのぞく

curl（カール） は、ブラウザの代わりに HTTP のリクエストを送り、返ってきたものを**そのまま文字で見せてくれる**道具です。Windows 10 以降には最初から入っています（UNIT 03 で見たとおり、PowerShell では必ず `curl.exe` と書きます）。

> **▼ このコマンドがやること（先に日本語で）**：説明用に用意されている公式のサイト `example.com` に HTTP のリクエストを送り、送った注文書（リクエスト）と、返ってきた返事（レスポンス）のヘッダーを表示します。本文の HTML は表示せずに捨てます。

```
curl.exe -4 -sv http://example.com/ -o NUL
```

▼ 実行結果（あなたの PC のアドレスの部分は伏せています）

```
* Host example.com:80 was resolved.
* IPv6: (none)
* IPv4: 172.66.147.243, 104.20.23.154
*   Trying 172.66.147.243:80...
* Established connection to example.com (172.66.147.243 port 80) from 192.168.x.x port
56035
* using HTTP/1.x
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 15 Oct 2026 11:38:59 GMT
< Content-Type: text/html; charset=utf-8
< Transfer-Encoding: chunked
< Connection: keep-alive
< Age: 9
< Server: cloudflare
< Last-Modified: Tue, 13 Oct 2026 22:05:48 GMT
< Allow: GET, HEAD
< Accept-Ranges: bytes
< cf-cache-status: HIT
< CF-RAY: a4ae8a66ff859649-KIX
< alt-svc: h3=":443"; ma=86400
<
{ [589 bytes data]
* Connection #0 to host example.com:80 left intact
```

▼ 出力を1つずつ分解して解説

解説1: コマンドのオプション

```
curl.exe -4 -sv http://example.com/ -o NUL
```

- `-v`（verbose: 詳しく）は、やりとりの中身を表示するオプションです。`-s`（silent）は進み具合の表示を消します。2つまとめて `-sv` と書けます。
- `-4` は「IPv4 という形式のアドレスで接続して」という指定です。表示を短くするために付けています。
- `-o NUL` は「本文は `NUL`（何も残らない特別な場所）に捨てて」という意味です。bash では `-o /dev/null` と書きます。

解説2: `*` の行 — 名前を引いて接続する（図5-2 の①②）

```
* Host example.com:80 was resolved.
* IPv4: 172.66.147.243, 104.20.23.154
* Established connection to example.com (172.66.147.243 port 80) from 192.168.x.x port
56035
```

- `*` の行は curl 自身の報告です。
- `was resolved` は「DNS で名前を引けた」という意味で、`example.com` の IP アドレスが2つ見つかりました。
- `Established connection` は「接続できた」。`port 80` は、`http` の URL でポートを省略したときに使われる番号です（`https` なら 443）。

解説3: `>` の行 — 送った注文書（図5-2 の③）

```
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.21.0
> Accept: */*
>
```

- `>` の行は、**curl がサーバーへ送ったリクエスト**そのものです。
- 1行目 `GET / HTTP/1.1` は「HTTP/1.1 の決まりで、パス `/`（トップページ）を GET（見せて）」。
- `Host:` はどのサイト宛てか、`User-Agent:` は送り主のソフトの名前、`Accept: */*` は「どんな種類の中身でも受け取ります」というヘッダーです。
- 最後の `>` だけの行が**空行**で、「ヘッダーはここまで」の合図です。GET なので本文はありません。

解説4: `<` の行 — 返ってきた返事（図5-2 の⑤）

```
< HTTP/1.1 200 OK
< Content-Type: text/html; charset=utf-8
< Server: cloudflare
<
{ [589 bytes data]
```

- `<` の行は、**サーバーから返ってきたレスポンス**です。
- 1行目の `200 OK` が結果のスタンプで、「成功」です。
- `Content-Type: text/html; charset=utf-8` は「本文は HTML で、文字コードは UTF-8 です」という添え状です。ブラウザはこれを見て、本文を HTML として描きます。
- 空行の後の `{ [589 bytes data]` は、「本文を 589 バイト受け取った（が `-o NUL` で捨てた）」という報告です。
- `-v` の代わりに `-i` を付けると、ヘッダーと本文の HTML をそのまま表示できます。

URL を部品に分けるプログラム

> **▼ このコードがやること（先に日本語で）**：図5-1 の URL を、文字の区切り（ `://` ・ `/` ・ `?` ・ `&` ・ `=` ）を手がかりにして、スキーム・ホスト・パス・クエリに分けて表示します。URL が「決まった区切りを持つ、ただの文字」であることを確かめます。

```
fn main() {
    let url = "https://your-app.fly.dev/notes?q=rust&page=2";
    println!("URL 全体: {url}");

    // 「://」の前がスキーム（通信のやり方）
    let (scheme, rest) = url.split_once("://").unwrap();
    println!("スキーム: {scheme}");

    // 最初の「/」の前がホスト（どのコンピュータか）
    let (host, path_and_query) = match rest.find('/') {
        Some(i) => (&rest[..i], &rest[i..]),
        None => (rest, "/"),
    };
    println!("ホスト   : {host}");

    // 「?」の前がパス（どのページか）、後ろがクエリ（追加の条件）
    let (path, query) = path_and_query
        .split_once('?')
        .unwrap_or((path_and_query, ""));
    println!("パス     : {path}");

    // クエリは「名前=値」が & でつながっている
    for pair in query.split('&') {
        if let Some((name, value)) = pair.split_once('=') {
            println!("クエリ   : {name} = {value}");
        }
    }
}
```

▼ 実行結果

```
URL 全体: https://your-app.fly.dev/notes?q=rust&page=2
スキーム: https
ホスト   : your-app.fly.dev
パス     : /notes
クエリ   : q = rust
クエリ   : page = 2
```

▼ コードを1つずつ分解して解説

解説1: 区切りで2つに分ける `split_once`

```
let (scheme, rest) = url.split_once("://").unwrap();
```

- `split_once("://")` は、文字列を**最初に見つかった** `://` の前後の2つに分けます。結果は `("https", "your-app.fly.dev/notes?q=rust&page=2")` です。
- 区切りが見つからないこともあるので結果は `Option` で、`.unwrap()` で中身を取り出しています（見つからなければパニックします。練習用の割り切りです）。
- `let (scheme, rest) = ...` は、2つ組の値を2つの変数に一度に受け取る書き方です。

解説2: 位置を探して切り出す

```
let (host, path_and_query) = match rest.find('/') {
    Some(i) => (&rest[..i], &rest[i..]),
    None => (rest, "/"),
};
```

- `rest.find('/')` は、最初の `/` が**何文字目（何バイト目）にあるか**を探します。
- `&rest[..i]` は「先頭から i の手前まで」、`&rest[i..]` は「i から最後まで」の切り出しです。
- `/` が無い URL（`https://example.com` など）のときは、パスを `/` とみなしています。

解説3: クエリを1組ずつ読む

```
for pair in query.split('&') {
    if let Some((name, value)) = pair.split_once('=') {
        println!("クエリ  : {name} = {value}");
    }
}
```

- `query.split('&')` は `&` で区切った部品を1つずつ取り出します（`q=rust`、`page=2`）。
- `if let Some(...) = ...` は「区切りが見つかったときだけ中を実行する」という書き方です。

> **注意:** 本物の URL には、日本語や記号を `%E7%89%9B` のような形に置き換える決まり（**URL エンコード**）などもあります。実際のアプリでは、こうした細かい決まりまで正しく扱ってくれる部品（Topcoat など）に任せます。ここでは「URL は区切りの決まった文字列だ」と分かれば十分です。

もう一歩: 実務に近い形

HTTP の注文書を、自分の手で書いて送る

ブラウザや curl がしていることは、結局「決まった形の文字を送って、文字を受け取る」だけです。それを Rust で直接やってみます。

> **▼ このコードがやること（先に日本語で）：** `example.com` の 80 番ポートに接続し、HTTP のリクエストを**文字として**組み立てて送ります。返ってきた文字を全部受け取り、空行を境にヘッダーと本文に分けて表示します。

```
use std::io::{Read, Write};
use std::net::TcpStream;

fn main() -> std::io::Result<()> {
    // example.com の 80 番ポート（http の窓口）に接続する
    let mut stream = TcpStream::connect("example.com:80")?;

    // HTTP リクエストは「決まった形の文字」。1 行ずつ \r\n で区切り、空行で終わる
    let request = "GET / HTTP/1.1\r\n\
                   Host: example.com\r\n\
                   User-Agent: rust-study\r\n\
                   Connection: close\r\n\
                   \r\n";
    stream.write_all(request.as_bytes())?;

    // サーバーからの返事（レスポンス）を最後まで受け取る
    let mut response = String::new();
    stream.read_to_string(&mut response)?;

    // 空行（\r\n\r\n）の前がヘッダー、後ろが本文
    let (head, body) = response.split_once("\r\n\r\n").unwrap_or((&response, ""));
    println!("----- ヘッダー部分 -----");
    println!("{head}");
    println!("----- 本文の先頭 60 文字 -----");
    let preview: String = body.chars().take(60).collect();
    println!("{preview}");
    Ok(())
}
```

▼ 実行結果

```
----- ヘッダー部分 -----
HTTP/1.1 200 OK
Date: Sat, 17 Oct 2026 02:30:02 GMT
Content-Type: text/html; charset=utf-8
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Tue, 13 Oct 2026 20:08:28 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 11679
cf-cache-status: HIT
CF-RAY: a4bbe10c7ef7440f-KIX
alt-svc: h3=":443"; ma=86400
----- 本文の先頭 60 文字 -----
241
<!doctype html><html lang=en><head><meta charset=utf-8>
```

▼ コードを1つずつ分解して解説

解説1: 接続する `TcpStream::connect`

```
let mut stream = TcpStream::connect("example.com:80")?;
```

- **TCP（ティーシーピー）** は、2台のコンピュータの間に「文字を順番どおり確実に届ける管」をつなぐしくみです。HTTP はこの管の上を流れます。
- `connect("example.com:80")` は、名前を引いて（DNS）、80 番ポートに接続します。図5-2 の①②を1行でやっています。
- 接続できたら、`stream` に書けば相手に届き、`stream` から読めば相手からの文字が受け取れます。

解説2: 注文書を文字で組み立てる

```
let request = "GET / HTTP/1.1\r\n\
               Host: example.com\r\n\
               User-Agent: rust-study\r\n\
               Connection: close\r\n\
               \r\n";
```

- HTTP の決まりでは、行の区切りは `\r\n`（**CR LF**：復帰と改行の2文字）です。
- 最後の `\r\n` だけの行が空行で、「ヘッダーはここまで」の合図です。これを忘れると、サーバーは続きを待ち続けます。
- 行末の `\` は「ソースコード上の改行と、次の行の先頭の空白を無視して1つの文字列につなげて」という Rust の書き方です。

- `Connection: close` は「返事を送ったら接続を切ってください」というお願いです。これで `read_to_string` が「最後まで読んだ」と分かります。

解説3: 返事を受け取って分ける

```
stream.read_to_string(&mut response)?;
let (head, body) = response.split_once("\r\n\r\n").unwrap_or((&response, ""));
```

- `read_to_string` で、接続が切れるまでの全部を `response` に読み込みます。
- 最初の空行（ `\r\n\r\n` ）でヘッダーと本文を分けています。
- 本文の先頭の `241` は HTML ではありません。ヘッダーに `Transfer-Encoding: chunked` とあるとおり、このサーバーは本文を「塊（chunk）」に分けて送っていて、`241` は「次の塊は 16 進数で 241（＝577）バイト」という目印です。ブラウザや curl は、こうした細かい決まりも自動で処理してくれています。

よくあるエラーと読み方

名前が引けない（curl のエラー 6）

```
PS C:\Users\you> curl.exe -sS http://exampel-typo.invalid/
curl: (6) Could not resolve host: exampel-typo.invalid
```

何が起きたか: 図5-2 の①で失敗しています。DNS（電話帳）に `exampel-typo.invalid` という名前が載っていませんでした。`(6)` は curl のエラー番号です。

直し方: URL の綴りを確かめます。綴りが正しいなら、インターネットにつながっているかを確かめます。

接続できない（curl のエラー 7）

```
PS C:\Users\you> curl.exe -sS http://127.0.0.1:8080/
curl: (7) Failed to connect to 127.0.0.1:8080 after 2030 ms: Could not connect to
server
```

何が起きたか: 図5-2 の②で失敗しています。名前は分かった（ここでは `127.0.0.1` を直接指定）けれど、その番地の 8080 番の窓口で**誰も待っていなかった**のです。サーバーを起動し忘れている、ポート番号を間違えている、のどちらかがほとんどです（UNIT 06 で、このポートで待つサーバーを作ります）。

直し方: サーバーが起動しているか、ポート番号が合っているかを確かめます。

返事は来たが 404

```
PS C:\Users\you> curl.exe -s -i http://example.com/notes-not-here
HTTP/1.1 404 Not Found
Date: Thu, 15 Oct 2026 11:39:00 GMT
Content-Type: text/html; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Server: cloudflare
Age: 9487
cf-cache-status: HIT
```

（本文の HTML は省略しています）

何が起きたか: これは**通信としては成功**しています。サーバーまで届き、サーバーが「そのパスのページはありません（404）」と返事をくれました。①〜⑤のうち④の結果です。

直し方: パスの綴りを確かめます。自分のサーバーなら、そのパスを受け付ける処理（**ルート**）を作ったかを確かめます。

> **つまずきポイント:** 「ページが表示されない」とひとことで言っても、①名前が引けない、②接続できない、④サーバーが 404 や 500 を返した、⑥ HTML の書き方がおかしい、では原因も直し方もまったく違います。`curl.exe -v` で**どこまで進んだか**を確かめるのが、Web の不具合を調べる第一歩です。

RustNote ではここで使う

- RustNote のページは、すべて「メソッド＋パス」の組で決まっています（巻頭の表）。たとえば一覧は `GET /notes`、作成は `POST /notes`、検索は `GET /notes?q=買い物` です。
- フォームの送信（POST）に成功したら、RustNote は HTML を直接返さずに **303 See Other** で「`GET /notes` を見に行って」と返します。こうしておくと、ブラウザの再読み込みボタンで同じノートが二重に作られる事故を防げます。この作法を **PRG（Post/Redirect/Get）** と呼びます（第6巻・第10巻）。
- 他の人のノートの URL を開かれたときは、**404** を返します。理由は UNIT 08 で説明します。

練習問題

問題1: 次の URL を、スキーム・ホスト・ポート・パス・クエリに分けてください。

```
http://127.0.0.1:8080/notes/42/edit?from=list
```

問題2: URL を分けるプログラムを改造して、ホストとポートを分けて表示してください。ポートが書かれていない URL のときは、`http` なら `80`、`https` なら `443` と表示します。

問題3: 次の場面で返すステータスコードとして、最もふさわしいものを選んでください（200・303・404・500）。

- a. ノート一覧のページを表示する
- b. ノートを作成したので、一覧のページへ移動してもらう
- c. `/notes/9999` が開かれたが、そんな番号のノートは無い
- d. プログラムの不具合で、処理の途中で失敗した

解答と解説

問題1の解答: スキームは `http`、ホストは `127.0.0.1`、ポートは `8080`、パスは `/notes/42/edit`、クエリは `from=list` です。ホストに IP アドレスを直接書くこともできます。

問題2の解答:

```
fn main() {
    let url = "http://127.0.0.1:8080/notes/42/edit?from=list";

    let (scheme, rest) = url.split_once("://").unwrap();
    let (host_and_port, path_and_query) = match rest.find('/') {
        Some(i) => (&rest[..i], &rest[i..]),
        None => (rest, "/"),
    };

    // 「ホスト:ポート」を「:」で分ける。ポートが書かれていなければ既定値を使う
    let (host, port) = match host_and_port.split_once(':') {
        Some((host, port)) => (host, port),
        None if scheme == "https" => (host_and_port, "443"),
        None => (host_and_port, "80"),
    };

    let (path, query) = path_and_query
        .split_once('?')
        .unwrap_or((path_and_query, ""));

    println!("スキーム: {scheme}");
    println!("ホスト   : {host}");
    println!("ポート   : {port}");
    println!("パス     : {path}");
    println!("クエリ   : {query}");
}
```

▼ 実行結果

```
スキーム: http
ホスト   : 127.0.0.1
ポート   : 8080
パス     : /notes/42/edit
クエリ   : from=list
```

`None if scheme == "https" =>` は「`None` で、しかも条件が成り立つとき」という書き方です。`match` は上から順に調べるので、https でなければ最後の `None =>` に進みます。

問題3の解答: a は `200`、b は `303`、c は `404`、d は `500`。4xx は「頼み方（URL）の問題」、5xx は「サーバー側の問題」です。

まとめ

- URL は「スキーム・ホスト・ポート・パス・クエリ」からなる宛先で、DNS がホスト名を IP アドレスに引き直します。
- HTTP のリクエストとレスポンスは「1行目・ヘッダー・空行・本文」の決まった形の**文字**で、メソッド（GET・POST）とステータスコード（200・303・404・500 など）が要です。
- 表示されないときは `curl.exe -v` で、名前解決・接続・返事のどこまで進んだかを確かめます。

UNIT 06: サーバー・ポート・localhost

この単元のゴール

- **サーバー**とは「リクエストを待ち続けるプロセス」だと説明できる
- **IP アドレス・ポート・localhost**（**127.0.0.1**）・**0.0.0.0** の違いを説明できる
- Rust の標準ライブラリだけで小さな Web サーバーを作り、`curl.exe` とブラウザからアクセスできる
- 「ポートが使用中」「実行ファイルを上書きできない」というエラーに対処できる

たとえ話でつかむ

ネットワークの住所は、**大きなマンション**で考えると分かりやすくなります。

- **IP アドレスはマンションの住所**です。インターネットにつながったコンピュータ1台1台に付いています（例 `172.66.147.243`）。
- **ポートは部屋番号**です。1つのマンション（1台の PC）には 1〜65535 番までの部屋があり、それぞれの部屋に別々の住人（プロセス）が住めます。同じ住所に届いた荷物も、部屋番号で届け先のプロセスが決まります。
- **サーバーは、部屋で来客を待ち続けている住人**です。「8080 号室で待っています」と管理人（OS）に届け出ておくと、8080 号室宛ての訪問者が来るたびに取り次いでもらえます。この届け出を **bind（バインド）**、待つことを **listen（リッスン）** と呼びます。1つの部屋に住めるのは1人だけです。

- **localhost（ローカルホスト） ＝ 127.0.0.1 は「このマンションの中」**を表す特別な住所です。内線電話のようなもので、外の人はこの番号にかけられません。開発中は、自分の PC の中だけでサーバーとブラウザをやりとりさせるのに使います。
- **0.0.0.0 で待つのは「どの入口から来た客も受け付ける」**という届け出です。127.0.0.1 で待つと内線（同じ PC の中）からしか来客がありませんが、0.0.0.0 で待つと、ネットワークの外からの来客も受け付けます。

> **用語: サーバー** … 頼まれごと（リクエスト）を待ち受けて、返事（レスポンス）を返す側のプログラム、またはそれが動いているコンピュータ。頼む側（ブラウザなど）は **クライアント** と呼びます。
>
> **用語: IP アドレス** … ネットワーク上のコンピュータの住所。`127.0.0.1` のような4つの数字の形（IPv4）と、`::1` のような長い形（IPv6）があります。

図で理解する

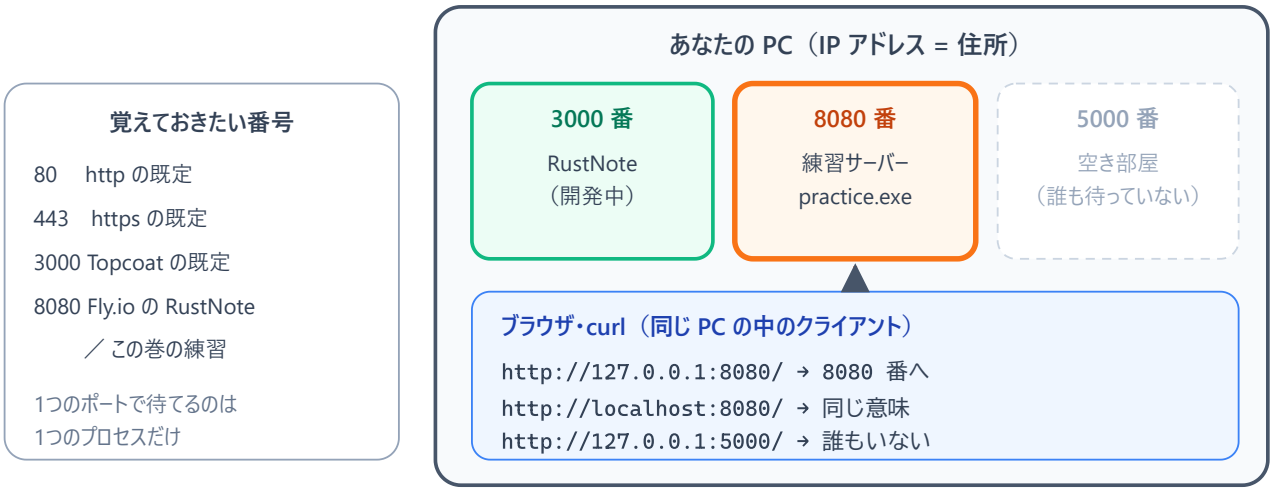


図6-1: ポートは部屋番号。URL の「:8080」で、どのプロセスに届けるかが決まる

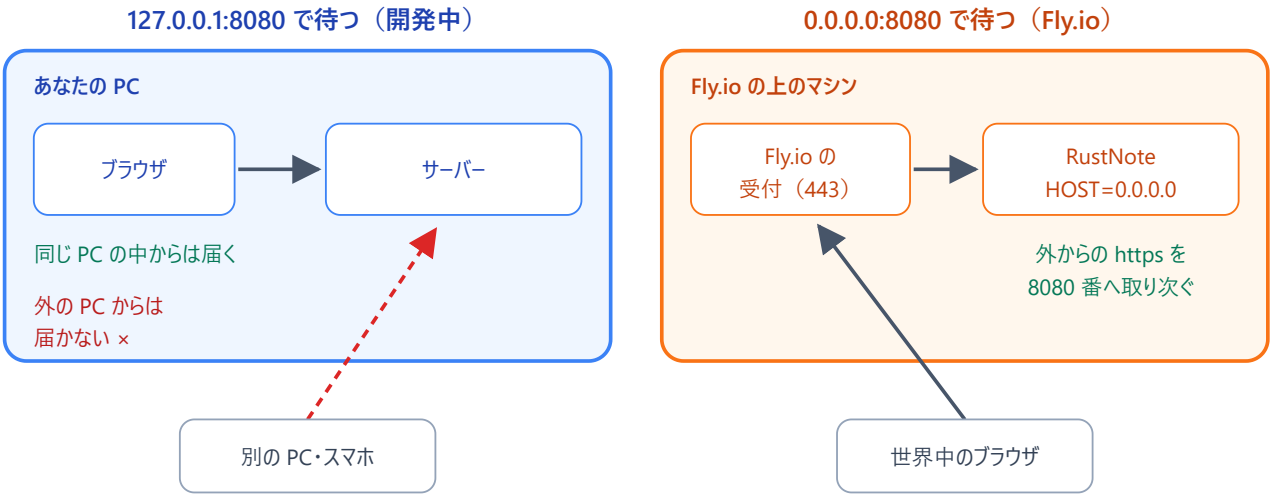


図6-2: 待ち受ける住所の違い。127.0.0.1 は自分の PC の中だけ、0.0.0.0 は外からも受け付ける

> **つまずきポイント:** `0.0.0.0` は「**どの入口でも待つ**」という待ち受け側の指定で、ブラウザに打つ宛先ではありません。開発中のサーバーにアクセスするときは `http://127.0.0.1:3000` や `http://localhost:3000` を使います。

まず動かす最小コード

Rust だけで作る、いちばん小さな Web サーバー

> **▼ このコードがやること（先に日本語で）**：自分の PC（127.0.0.1）の 8080 番ポートで来客を待ちます。リクエストが来るたびに、その1行目を画面に記録し、「Hello from Rust!」という見出しの HTML を HTTP の決まった形で返します。止めるまでずっと待ち続けます。

```rust
use std::io::{BufRead, BufReader, Write};
use std::net::TcpListener;

fn main() -> std::io::Result<()> {
    // 自分の PC（127.0.0.1）の 8080 番ポートで待ち受けを始める
    let listener = TcpListener::bind("127.0.0.1:8080")?;
    println!("http://127.0.0.1:8080 で待っています（止めるには Ctrl+C）");

    // 接続が来るたびに、1 つずつ順番に処理する
    for stream in listener.incoming() {
        let mut stream = stream?;
        let mut reader = BufReader::new(&stream);

        // リクエストの 1 行目（例: GET / HTTP/1.1）を読む
        let mut first_line = String::new();
        reader.read_line(&mut first_line)?;
        println!("受け取った: {}", first_line.trim_end());

        // 残りのヘッダー行を、空行が来るまで読み飛ばす
        loop {
            let mut line = String::new();
            let n = reader.read_line(&mut line)?;
            if n == 0 || line == "\r\n" {
                break;
            }
        }

        // 返事（レスポンス）を HTTP の決まった形で組み立てて送る
        let body = "<h1>Hello from Rust!</h1>\n";
        let response = format!(
            "HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=utf-8\r\nContent-
Length: {}\r\n\r\n{}",
            body.len(),
            body
        );
        stream.write_all(response.as_bytes())?;
    }
    Ok(())
}
```

PowerShell の窓を**2つ**開きます。1つ目の窓でサーバーを起動します。

```
cargo run
```

サーバーは止まらずに待ち続けます。2つ目の窓から、curl でアクセスします。

```
curl.exe -s -i http://127.0.0.1:8080/
curl.exe -s http://localhost:8080/hello
```

▼ **実行結果**（2つ目の窓・curl 側）

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 26

<h1>Hello from Rust!</h1>
<h1>Hello from Rust!</h1>
```

▼ **実行結果**（1つ目の窓・サーバー側の記録）

```
http://127.0.0.1:8080 で待っています（止めるには Ctrl+C）
受け取った: GET / HTTP/1.1
受け取った: GET /hello HTTP/1.1
```

ブラウザで `http://127.0.0.1:8080/` を開いても、同じ見出しが表示されます。止めるときは、サーバーの窓で **Ctrl キーを押しながら C** を押します。

▼ コードを1つずつ分解して解説

解説1: 部屋を借りて待ち始める（bind）

```
let listener = TcpListener::bind("127.0.0.1:8080")?;
```

- `TcpListener::bind` は、OS に「127.0.0.1 の 8080 番で待たせてください」と届け出ます。
- 届け出が通ると `listener`（待ち受け係）が手に入ります。他のプロセスがすでに 8080 番を使っていると、ここで失敗します（「よくあるエラー」で見ます）。

解説2: 来客を1人ずつ迎える

```
for stream in listener.incoming() {
    let mut stream = stream?;
    // ...
}
```

- `listener.incoming()` は「来客（接続）の列」です。`for` で1人ずつ取り出し、来客がいないときは**次の来客まで待ちます**。だからこのプログラムは終わりません。

- `stream` は UNIT 05 の `TcpStream` と同じで、読めば相手からの文字、書けば相手への文字になります。今回はサーバー側なので、まず読んでから書きます。

解説3: 注文書を読む

```
let mut first_line = String::new();
reader.read_line(&mut first_line)?;
// ...
loop {
    let mut line = String::new();
    let n = reader.read_line(&mut line)?;
    if n == 0 || line == "\r\n" {
        break;
    }
}
```

- `BufReader` は、届いた文字を1行ずつ読みやすくする道具です。
- 1行目（`GET / HTTP/1.1`）を読んで記録したあと、空行（`"\r\n"`）が来るまでヘッダーを読み飛ばします。
- ヘッダーを読まずに返事をして接続を閉じると、相手側で「接続が途中で切られた」というエラーになることがあるため、最後まで読んでいます。
- `n == 0` は「相手が何も送らずに接続を閉じた」場合です。

解説4: 返事を書く

```
let response = format!(
    "HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=utf-8\r\nContent-Length:
{}\r\n\r\n{}",
    body.len(),
    body
);
stream.write_all(response.as_bytes())?;
```

- `format!` は、`{}` の場所に値を埋め込んだ文字列を作ります（`println!` の「表示せずに文字列を作る」版）。
- UNIT 05 の図5-3 と同じ「1行目・ヘッダー・空行・本文」の形です。
- `Content-Length` は本文の**バイト数**です。`body.len()` はバイト数を返すので、ここに使えます（文字数とバイト数の違いは UNIT 10 で見ます）。
- `write_all` で相手に送ります。`stream` の処理が終わると接続は自動で閉じられます。

もう一歩: 実務に近い形

本物の Web アプリは、URL のパスによって返す内容を変えます。この振り分けを **ルーティング** と呼びます。

パスで返事を変える（ルーティング）

> ▼ **このコードがやること（先に日本語で）**：リクエストの1行目からメソッドとパスを取り出し、`GET /` ならトップページ、`GET /health` なら `ok`、それ以外なら「404 ページが見つかりません」を返します。どのリクエストに何を返したかを、サーバーの画面に記録します。

```rust
use std::io::{BufRead, BufReader, Write};
use std::net::{TcpListener, TcpStream};

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080")?;
    println!("http://127.0.0.1:8080 で待っています（止めるには Ctrl+C）");

    for stream in listener.incoming() {
        handle(stream?)?;
    }
    Ok(())
}

// 1 つの接続（1 回のリクエスト）を処理する
fn handle(mut stream: TcpStream) -> std::io::Result<()> {
    let mut reader = BufReader::new(&stream);

    // 1 行目「GET /health HTTP/1.1」を、空白で 3 つに分ける
    let mut first_line = String::new();
    reader.read_line(&mut first_line)?;
    let mut parts = first_line.split_whitespace();
    let method = parts.next().unwrap_or("");
    let path = parts.next().unwrap_or("");

    // 残りのヘッダー行は空行まで読み飛ばす
    loop {
        let mut line = String::new();
        let n = reader.read_line(&mut line)?;
        if n == 0 || line == "\r\n" {
            break;
        }
    }

    // メソッドとパスの組み合わせで、返す内容を決める（ルーティング）
    let (status, body) = match (method, path) {
        ("GET", "/") => ("200 OK", "<h1>トップページ</h1>\n"),
        ("GET", "/health") => ("200 OK", "ok\n"),
        _ => ("404 Not Found", "<h1>ページが見つかりません</h1>\n"),
    };
    println!("{method} {path} -> {status}");

    let response = format!(
        "HTTP/1.1 {status}\r\nContent-Type: text/html; charset=utf-8\r\nContent-Length:
{}\r\n\r\n{body}",
        body.len()
    );
    stream.write_all(response.as_bytes())?;
```

```
    Ok(())
}
```

サーバーを起動したまま、別の窓から curl で3回アクセスし、最後にブラウザで `http://127.0.0.1:8080/` を開きます。

```
curl.exe -s http://127.0.0.1:8080/health
curl.exe -s -i http://127.0.0.1:8080/notes
curl.exe -s -i -X POST http://127.0.0.1:8080/health
```

▼ 実行結果（curl 側）

```
ok
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 43

<h1>ページが見つかりません</h1>

HTTP/1.1 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 43

<h1>ページが見つかりません</h1>
```

▼ 実行結果（サーバー側の記録）

```
http://127.0.0.1:8080 で待っています（止めるには Ctrl+C）
GET /health -> 200 OK
GET /notes -> 404 Not Found
POST /health -> 404 Not Found
GET / -> 200 OK
GET /favicon.ico -> 404 Not Found
  -> 404 Not Found
```

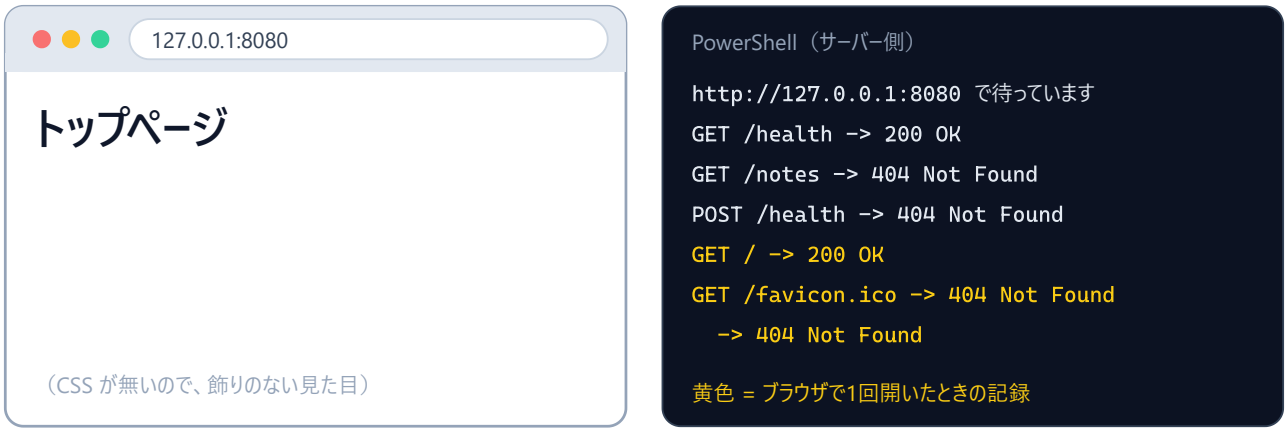


図6-3: ブラウザで開いた画面（左）と、そのときのサーバーの記録（右）

▼ コードを1つずつ分解して解説

解説1: 処理を関数に分ける

```
for stream in listener.incoming() {
    handle(stream?)?;
}
```

- 1回分のリクエストの処理を `handle` という関数に切り出しました。`main` は「待って、来たら `handle` に任せる」だけになり、読みやすくなります。
- `fn handle(mut stream: TcpStream) -> std::io::Result<()>` は、「`TcpStream` を1つ受け取り、成功か失敗を返す関数」という意味です。

解説2: メソッドとパスを取り出す

```
let mut parts = first_line.split_whitespace();
let method = parts.next().unwrap_or("");
let path = parts.next().unwrap_or("");
```

- `split_whitespace()` は空白で区切って、部品を1つずつ取り出せるようにします。`GET /health HTTP/1.1` なら `GET`、`/health`、`HTTP/1.1` の順です。
- `.next()` で次の部品を1つ取り出します。無ければ `unwrap_or("")` で空文字にします。

解説3: `match` で振り分ける

```
let (status, body) = match (method, path) {
    ("GET", "/") => ("200 OK", "<h1>トップページ</h1>\n"),
    ("GET", "/health") => ("200 OK", "ok\n"),
    _ => ("404 Not Found", "<h1>ページが見つかりません</h1>\n"),
};
```

- `(method, path)` の組を、上から順に比べます。`_` は「それ以外ぜんぶ」です。
- `POST /health` が 404 になったのは、`("GET", "/health")` とメソッドが違うからです。**同じパスでもメソッドが違えば別の注文**として扱われます。
- これが **ルーティング** の正体です。Topcoat などのフレームワークは、これをもっと便利で安全な形で用意してくれます。

解説4: ブラウザは頼んでいないものも頼む

- ブラウザで1回開いただけなのに、記録が3行増えました。
- `GET /favicon.ico` は、ブラウザがタブに表示する小さなアイコン（**ファビコン**）を自動で取りに来たものです。
- 最後の `-> 404 Not Found`（メソッドもパスも空）は、ブラウザが「次に使うかもしれない」と先回りして開いた予備の接続が、何も送らずに閉じられたものです（解説3 の `n == 0` の場合）。
- サーバーの記録には、こうした「人が頼んでいないリクエスト」も混ざります。記録を読むときに慌てないようにしましょう。

よくあるエラーと読み方

ポートがすでに使われている

サーバーを動かしたまま、同じプログラムをもう1つ起動すると、次のエラーになります。

```
Error: Os { code: 10048, kind: AddrInUse, message: "通常、各ソケット アドレスに対してプロトコル、ネットワーク アドレス、またはポートのどれか 1 つのみを使用できます。" }
```

何が起きたか: `bind` で 8080 番を借りようとしましたが、すでに別のプロセスが住んでいました。`AddrInUse`（アドレス使用中）、Windows のエラー番号は 10048 です。

直し方: 先に動いているサーバーを止めます（その窓で Ctrl+C）。どのプロセスが使っているか分からないときは、次のコマンドで調べられます。

```
Get-NetTCPConnection -LocalPort 8080 -State Listen | Select-Object LocalAddress, LocalPort, State, OwningProcess
Get-Process -Id (Get-NetTCPConnection -LocalPort 8080 -State Listen).OwningProcess
netstat -ano | findstr :8080
```

▼ 実行結果

```
LocalAddress LocalPort  State OwningProcess
------------ ---------  ----- -------------
127.0.0.1         8080 Listen         54588


Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
     67       6      864       6396       0.00  54588   1 practice


  TCP    127.0.0.1:8080         0.0.0.0:0              LISTENING       54588
```

- 1つ目のコマンドは「8080 番で待ち受けている（`Listen`）接続」を探し、持ち主のプロセス番号（`OwningProcess`）を表示します。
- 2つ目は、その番号のプロセスの情報を表示します。`ProcessName` が `practice` なので、さっき起動した自分のサーバーだと分かります。
- 3つ目の `netstat` は昔からあるコマンドで、最後の数字が同じプロセス番号です。

自分のサーバーなら、`Stop-Process -Id 54588` で止めることもできます。

サーバーを動かしたまま `cargo build` した

```
   Compiling practice v0.1.0 (C:\Users\you\rust-study\practice)
error: failed to remove file `C:\Users\you\rust-
study\practice\target\debug\practice.exe`

Caused by:
  アクセスが拒否されました。 (os error 5)
```

何が起きたか: Cargo は新しい `practice.exe` を作る前に古いファイルを消そうとしますが、Windows では**実行中のプログラムのファイルは消せません**。そのため「アクセスが拒否されました（OS のエラー 5）」になりました。`Caused by:`（原因は）の下に、OS からの本当の理由が書かれています。

直し方: サーバーを Ctrl+C で止めてから、もう一度 `cargo build`（または `cargo run`）します。サーバーを作っているときは「直す → 止める → 起動し直す」の順番を習慣にしましょう。

RustNote ではここで使う

- RustNote は Topcoat の `topcoat::start` でサーバーを起動します。待ち受ける場所は環境変数 `HOST` と `PORT` で決まり、何も設定しなければ `127.0.0.1:3000` です。開発中はブラウザで `http://127.0.0.1:3000` を開きます。この巻の練習サーバーが 8080 番なのは、RustNote とぶつからないようにするためです。
- Fly.io の上では、`HOST=0.0.0.0`・`PORT=8080` を設定して起動します。0.0.0.0 にしないと、Fly.io の受付からの取り次ぎを受けられません（第12巻）。

- RustNote にも、`ok` と返すだけの `GET /health` を作ります。Fly.io はこの URL に定期的にアクセスして、「サーバーが生きているか」を確かめます（**ヘルスチェック**、第9巻・第12巻）。

練習問題

問題1: 「もう一歩」のサーバーに、`GET /about` で「このサーバーについて」という見出しを返すルートを追加し、curl で確かめてください。

問題2: 次の説明のうち正しいものをすべて選んでください。

- a. `127.0.0.1:8080` で待っているサーバーには、同じ Wi-Fi の別の PC からアクセスできる
- b. `localhost` と `127.0.0.1` は、どちらも自分の PC を指す
- c. 1つのポートでは、1つのプロセスしか待ち受けられない
- d. ブラウザで `http://0.0.0.0:3000` を開くのが、Fly.io にアクセスする正しい方法である

問題3: `cargo run` したら `AddrInUse` のエラーが出ました。調べて解決する手順を書いてください。

解答と解説

問題1の解答: `match` に1行追加します（他の部分は「もう一歩」のコードと同じです）。

```
    let (status, body) = match (method, path) {
        ("GET", "/") => ("200 OK", "<h1>トップページ</h1>\n"),
        ("GET", "/health") => ("200 OK", "ok\n"),
        ("GET", "/about") => ("200 OK", "<h1>このサーバーについて</h1>\n"),
        _ => ("404 Not Found", "<h1>ページが見つかりません</h1>\n"),
    };
```

```
curl.exe -s -i http://127.0.0.1:8080/about
```

▼ 実行結果

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 40

<h1>このサーバーについて</h1>
```

サーバー側には `GET /about -> 200 OK` と記録されます。追加する場所は `_ =>` の行より**上**です。`_` は「それ以外ぜんぶ」なので、その下に書いた行には決して届きません。

問題2の解答: 正しいのは **b と c** です。a は誤りで、127.0.0.1 は自分の PC の中だけです。d も誤りで、0.0.0.0 は待ち受け側の指定です。Fly.io のアプリには `https://アプリ名.fly.dev` の URL でアクセスします。

問題3の解答:

1. `Get-NetTCPConnection -LocalPort ポート番号 -State Listen` で、そのポートを使っているプロセスの番号（OwningProcess）を調べる
2. `Get-Process -Id 番号` で、それが何のプログラムかを確かめる
3. 自分が前に起動したサーバーなら、その窓で Ctrl+C するか `Stop-Process -Id 番号` で止める。別の大事なアプリなら、自分のサーバーのポート番号のほうを変える

まとめ

- サーバーは、OS に届け出たポート（部屋番号）で来客を待ち続けるプロセスです。1つのポートで待てるのは1つのプロセスだけです。
- `127.0.0.1`（localhost）は自分の PC の中だけ、`0.0.0.0` で待つと外からも受け付けます。RustNote は手元で `127.0.0.1:3000`、Fly.io で `0.0.0.0:8080` です。
- パスとメソッドで処理を振り分けるのがルーティングです。ポートの使用中・実行ファイルのロックは「先に動いているものを止める」で解決します。

UNIT 07: データベースと永続化

この単元のゴール

- 変数の値はプロセスが終わると消え、ファイルやデータベースの中身は残ること（**永続化**）を体験する
- データベースの **テーブル（表）・行・列・主キー・制約** を説明できる
- 「ファイルに書くだけ」では足りなくなる理由と、データベースが引き受けてくれる仕事が分かる
- SQL の文と、制約違反のエラーを読める

たとえ話でつかむ

データの置き場所を、**会社の事務所**にたとえます。

- **変数（メモリ）はホワイトボード**です。書くのも消すのも速いですが、退社するとき（プロセスの終了）に消されてしまい、停電（電源オフ）でも消えます。
- **ファイルはノートのメモ**です。閉じても残りますが、書き方は自分しだいです。何人もが同時に書き込むと字が重なり、目当ての1行を探すにも最初から読むしかありません。
- **データベース（DB）は台帳を管理する経理係**です。データを決まった形の**表**で整理し、「この列は必ず埋める」「メールアドレスは同じものを2つ登録しない」といった**ルール（制約）**を守らない記入は受け付けません。何人から同時に頼まれても順番に正しく処理し、「Hanako さんのノートだけ」のような探し物もすばやく見つけてくれます。

> **用語: 永続化（えいぞくか）** … プログラムが終わっても、PC を再起動しても、データが残るように保存すること。
>
> **用語: データベース（DB）** … データを整理して保存し、探したり書き換えたりする仕事を引き受ける専用のプログラム（とそのデータ）。RustNote は **Turso（ターソ）** というサービスの **libSQL（リブエスキューエル）** というデータベースを使います。libSQL は、世界中で使われている **SQLite（エスキューライト）** の仲間です。

図で理解する

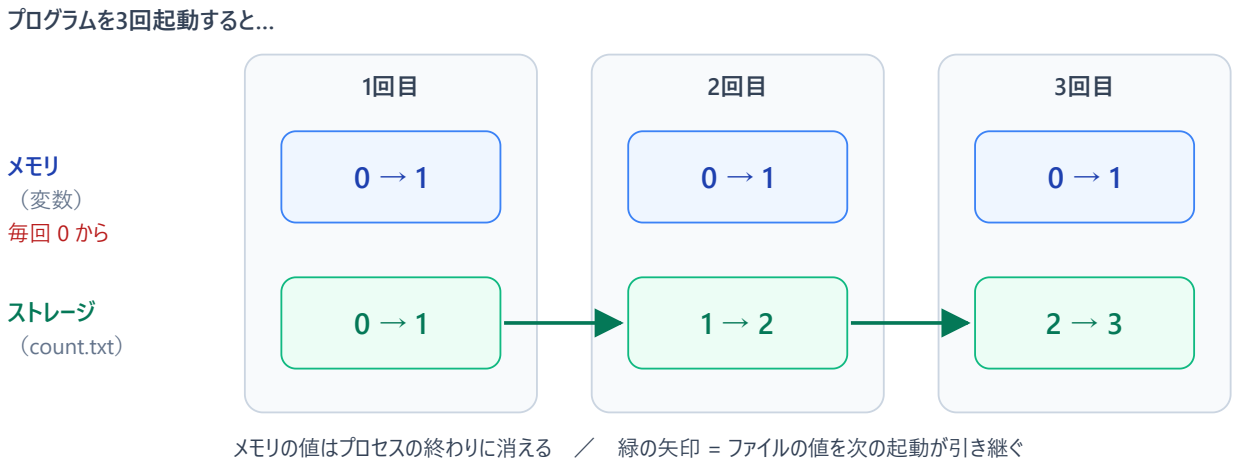


図7-1: メモリの値はプロセスが終わると消え、ストレージに書いた値は次の起動に引き継がれる

データベースは、データを **テーブル（表）** の形でしまいます。表計算ソフトのシートによく似ています。

notes テーブル（一部の列だけ）

id	user_id	title	done	updated_at
1	1	牛乳を買う	0	2026-10-15 09:12
2	1	Rust の本を読む	1	2026-10-14 21:40
3	2	歯医者の予約	0	2026-10-13 18:05

行（レコード）
ノート1件ぶんのデータ
（橙の枠がその1件）

列（カラム）
項目。型とルールを持つ
（青の枠が title 列）

主キー（id）
1件を特定する番号
同じ値は2つとない

図7-2: テーブルの行と列。done は 0 = 未済み、1 = 済み（値は例）

テーブルどうしは、列の値でつながります。`notes` の `user_id` 列には「そのノートの持ち主の `users.id`」が入っています。

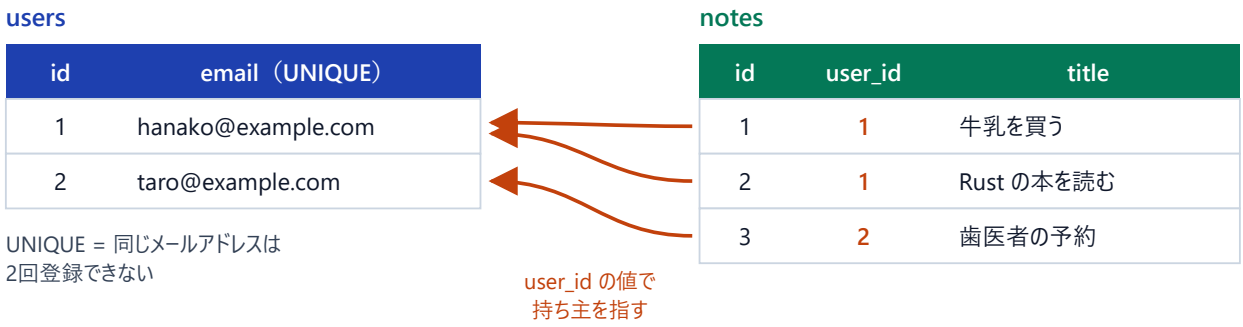


図7-3: users と notes のつながり。user_id が「持ち主」を表す

まず動かす最小コード

消えるカウンターと、消えないカウンター

> **▼ このコードがやること（先に日本語で）**：2つのカウンター（数を数える仕組み）を同時に1増やします。1つはメモリの中だけ、もう1つは `count.txt` というファイルに保存します。プログラムを何回か起動して、どちらが前回の続きから数えるかを比べます。

```
use std::fs;

fn main() {
    // (1) メモリの中だけのカウンター。プログラムが始まるたびに 0 から
    let mut in_memory = 0;
    in_memory += 1;

    // (2) ファイルに保存するカウンター。前回の値を読み、1 足して書き戻す
    //     count.txt が無い（初回）ときは "0" として扱う
    let saved = fs::read_to_string("count.txt").unwrap_or(String::from("0"));
    let mut on_disk: u32 = saved.trim().parse().unwrap();
    on_disk += 1;
    fs::write("count.txt", on_disk.to_string()).unwrap();

    println!("メモリのカウンター  : {in_memory}");
    println!("ファイルのカウンター: {on_disk}");
}
```

```
cargo run -q
cargo run -q
cargo run -q
cat count.txt
```

▼ 実行結果

```
メモリのカウンター　: 1
ファイルのカウンター: 1
メモリのカウンター　: 1
ファイルのカウンター: 2
メモリのカウンター　: 1
ファイルのカウンター: 3
3
```

メモリのカウンターは毎回 1 に戻り、ファイルのカウンターは 1→2→3 と増えました。PC を再起動してからもう一度実行しても、ファイルのカウンターは 4 になります。

▼ コードを1つずつ分解して解説

解説1: メモリのカウンター

```
let mut in_memory = 0;
in_memory += 1;
```

- 変数 `in_memory` はメモリ上にあり、**プロセスが終わると OS に片付けられます**。次に起動したときは、また `0` から始まります。

解説2: 前回の値を読む

```
let saved = fs::read_to_string("count.txt").unwrap_or(String::from("0"));
let mut on_disk: u32 = saved.trim().parse().unwrap();
```

- `count.txt` を読みます。初回はファイルが無くて失敗するので、`unwrap_or` で `"0"` を使います。
- ファイルの中身は**文字**の `"3"` なので、`.parse()` で**数値**の `3` に変換します。`.trim()` は前後の空白や改行を取り除きます。
- `: u32` は「0 以上の整数として扱う」という型の指定です。これがあるので、`parse` は何に変換すればよいか分かります。

解説3: 新しい値を書き戻す

```
on_disk += 1;
fs::write("count.txt", on_disk.to_string()).unwrap();
```

- 1 増やした数を `.to_string()` で文字に戻して、ファイルに上書きします。
- 「読む → 変える → 書き戻す」は、データを保存するプログラムの基本の形です。データベースを使っても、この流れは変わりません。

もう一歩: 実務に近い形

ファイルで「表」を作ってみる

ノートの一覧を、1行1件・タブ区切りのファイル（**TSV**）で保存してみます。データベースの表を、手作りで真似したものです。

> **▼ このコードがやること（先に日本語で）**：起動時にタイトルが渡されたら、`notes.tsv` の最後に「id・持ち主の番号・タイトル・済みかどうか」を1行追加します。そのあと、ファイルの全行を表の形で表示します。

```
use std::env;
use std::fs::{self, OpenOptions};
use std::io::Write;

fn main() -> std::io::Result<()> {
    let file = "notes.tsv";

    // 引数が渡されたら、それをタイトルとして 1 行（1 件）追加する
    if let Some(title) = env::args().nth(1) {
        // 今ある行数 + 1 を、新しい行の id にする
        let existing = fs::read_to_string(file).unwrap_or_default();
        let next_id = existing.lines().count() + 1;

        // ファイルの末尾に追記する（無ければ作る）
        let mut f = OpenOptions::new().create(true).append(true).open(file)?;
        // 列の順番: id, user_id, title, done（タブ文字 \t で区切る）
        writeln!(f, "{next_id}\t1\t{title}\tfalse")?;
    }

    // ファイルの全行を、表（テーブル）の形で表示する
    let text = fs::read_to_string(file).unwrap_or_default();
    println!("id | user_id | done  | title");
    for line in text.lines() {
        let cols: Vec<&str> = line.split('\t').collect();
        println!(
            "{:<2} | {:<7} | {:<5} | {}",
            cols[0], cols[1], cols[3], cols[2]
        );
    }
    Ok(())
}
```

```
cargo run -q -- 牛乳を買う
cargo run -q -- "Rust の本を読む"
cat notes.tsv
```

▼ 実行結果

```
id | user_id | done  | title
1  | 1       | false | 牛乳を買う
id | user_id | done  | title
1  | 1       | false | 牛乳を買う
2  | 1       | false | Rust の本を読む
1   1   牛乳を買う   false
2   1   Rust の本を読む   false
```

最後の2行が `notes.tsv` の中身そのものです。列の間の空白に見える部分は、タブ文字です。

▼ コードを1つずつ分解して解説

解説1: 追記モードで開く

```
let mut f = OpenOptions::new().create(true).append(true).open(file)?;
writeln!(f, "{next_id}\t1\t{title}\tfalse")?;
```

- `fs::write` は上書きでしたが、`OpenOptions` で `append(true)`（末尾に追記）を指定すると、前の行を残したまま書き足せます。`create(true)` は「無ければ作る」です。
- `writeln!` は、`println!` の「画面の代わりにファイルへ書く」版です。`\t` はタブ文字です。

解説2: 1行を列に分ける

```
let cols: Vec<&str> = line.split('\t').collect();
```

- `split('\t')` でタブごとに区切り、`.collect()` で部品を並べた `Vec`（**ベクタ：伸び縮みする並び**）にまとめます。
- `cols[0]` が id、`cols[2]` がタイトルのように、番号で列を取り出せます。

解説3: このやり方の限界

手作りの表は動きますが、本物のアプリにするには次の問題があります。

困りごと	手作りファイルでは	データベースなら
2人が同時に追加した	id が重なったり、行が混ざったりする	順番に処理して、id も自動で重ならない
タイトルにタブが含まれていた	列がずれて表が壊れる	値の区切りを DB が管理するので壊れない
「Hanako のノートだけ」を探す	全行を読んで1行ずつ調べる	条件を書くだけで、索引を使って速く探す
空のタイトル、同じメールの二重登録	自分で全部チェックする	**制約**として書いておけば DB が断る
途中で電源が切れた	書きかけの半端な行が残る	「全部やるか、何もしないか」を保証できる（**トランザクション**）

本物のデータベースなら: SQL の雰囲気

データベースには、**SQL（エスキューエル）** という言葉でお願いします。同じことを SQL で書くと次のようになります。ここは**見るだけ**で大丈夫です（第7巻で、Turso の道具を使って実際に動かします）。

```
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    email TEXT NOT NULL UNIQUE
);
CREATE TABLE notes (
    id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id),
    title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 80),
    done INTEGER NOT NULL DEFAULT 0
);
INSERT INTO users (email) VALUES ('hanako@example.com');
INSERT INTO users (email) VALUES ('taro@example.com');
INSERT INTO notes (user_id, title) VALUES (1, '牛乳を買う');
INSERT INTO notes (user_id, title) VALUES (1, 'Rust の本を読む');
INSERT INTO notes (user_id, title) VALUES (2, '歯医者の予約');
SELECT id, title, done FROM notes WHERE user_id = 1;
```

▼ **実行結果**（SQLite 3.50 のコマンドラインツールで実行し、列を `|` で区切って表示したもの）

```
id | title | done
1 | 牛乳を買う | 0
2 | Rust の本を読む | 0
```

- `CREATE TABLE` は表を作る文です。列の名前・型（`INTEGER` は整数、`TEXT` は文字）・ルールを書きます。

- `PRIMARY KEY` は主キー、`NOT NULL` は「必ず値を入れる」、`UNIQUE` は「同じ値を2つ入れない」、`CHECK (...)` は「この条件を満たす値だけ入れる」、`REFERENCES users(id)` は「`users` の `id` のどれかを指す」というルールです。
- `INSERT INTO ... VALUES ...` は行を追加する文、`SELECT ... FROM ... WHERE ...` は「条件に合う行を探して見せて」という文です。`WHERE user_id = 1` と書くだけで、Hanako のノートだけが返ってきます。

よくあるエラーと読み方

保存されたデータが壊れていた

最小コードの `count.txt` の中身を、手で `abc` に書き換えてから実行します。

```
Set-Content count.txt "abc"
cargo run
```

▼ 実行結果

```
thread 'main' (9276) panicked at src\main.rs:11:49:
called `Result::unwrap()` on an `Err` value: ParseIntError { kind: InvalidDigit }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
error: process didn't exit successfully: `target\debug\practice.exe` (exit code: 101)
```

何が起きたか: 11行目の `parse().unwrap()` で、`"abc"` を数値に変換できずにパニックしました（UNIT 04 と同じ `ParseIntError`）。保存されたデータは、**いつも自分の思ったとおりの形とは限らない**のです。

直し方: 今回は `count.txt` を消すか、数字に直せば動きます。本物のアプリでは、読み込んだデータが壊れていても止まらずにエラーとして扱う書き方をします。データベースを使うと、そもそも型や制約に合わないデータが入りにくくなります。

データベースが記入を断った（制約違反）

上の SQL の続きで、ルールに違反する記入を試します。

```
INSERT INTO users (email) VALUES ('hanako@example.com');
```

```
Runtime error near line 1: UNIQUE constraint failed: users.email (19)
```

```
INSERT INTO notes (user_id, title) VALUES (1, '');
```

```
Runtime error near line 1: CHECK constraint failed: length(title) BETWEEN 1 AND 80 (19)
```

何が起きたか:

- 1つ目は「`users.email` の **UNIQUE 制約**に失敗」。同じメールアドレスがすでに登録されているので断られました。
- 2つ目は「**CHECK 制約** `length(title) BETWEEN 1 AND 80` に失敗」。空のタイトル（0文字）は 1〜80 文字の範囲外です。
- 末尾の `(19)` は、SQLite の「制約違反」を表すエラー番号です。

直し方: これは「DB がちゃんと仕事をした」結果です。アプリ側では、この失敗を受け取って「そのメールアドレスはすでに登録されています」「タイトルは1〜80文字で入力してください」のような、利用者に分かる言葉に直して画面に表示します。

> **つまずきポイント:** 「アプリ側でチェックしているから、DB の制約はいらない」と考えがちですが、両方に書くのが基本です。アプリのチェックは**利用者に親切なメッセージを出すため**、DB の制約は**どんな経路からでも間違ったデータを入れさせない最後の砦**です。

RustNote ではここで使う

- RustNote のデータは、次の4つのテーブルにしまいます（第7巻・第9巻で作ります）。

テーブル	中身	主な列
`users`	登録した人	`id`、`email`（UNIQUE）、`password_hash`、`created_at`
`notes`	ノート	`id`、`user_id`（→ `users.id`）、`title`（1〜80文字）、`body`、`done`、`version`、`created_at`、`updated_at`
`sessions`	ログイン中の状態	`token_hash`、`user_id`、`expires_at`、`created_at`
`schema_migrations`	表の作り変えの記録	`version`、`applied_at`

- 開発中はデータベースを**自分の PC のファイル**として持ち、公開後はインターネット上の **Turso** を使います。プログラムからの使い方はほぼ同じです。
- テーブルの形を後から変えるときは、変更の手順を SQL のファイル（**マイグレーション**）として残し、`schema_migrations` に「どこまで適用したか」を記録します（第7巻）。

練習問題

問題1: 最小コードを改造して、`cargo run -q -- reset` と実行したときは `count.txt` を消して「カウンターを 0 に戻しました」と表示するようにしてください。

問題2: 「本物のデータベースなら」の SQL の表に対して、次の SQL を実行すると何が返ってくるでしょうか。

```
SELECT title FROM notes WHERE user_id = 2;
```

問題3: RustNote で次のデータを保存するとき、「メモリだけでよい」か「データベースに保存すべき」かを答えてください。

- a. ユーザーのメールアドレス
- b. 今まさに処理しているリクエストで送られてきたタイトルの文字列
- c. ノートの「済み（done）」の状態
- d. ログイン中であることの記録（セッション）

解答と解説

問題1の解答:

```
use std::env;
use std::fs;

fn main() {
    // 1 つ目の引数が "reset" なら、ファイルを消して 0 に戻す
    if env::args().nth(1).as_deref() == Some("reset") {
        // ファイルが無い場合のエラーは気にしないので、結果を捨てる
        let _ = fs::remove_file("count.txt");
        println!("カウンターを 0 に戻しました");
        return;
    }

    let saved = fs::read_to_string("count.txt").unwrap_or(String::from("0"));
    let mut on_disk: u32 = saved.trim().parse().unwrap();
    on_disk += 1;
    fs::write("count.txt", on_disk.to_string()).unwrap();
    println!("ファイルのカウンター: {on_disk}");
}
```

```
cargo run -q
cargo run -q
cargo run -q -- reset
cargo run -q
```

▼ 実行結果

```
ファイルのカウンター: 1
ファイルのカウンター: 2
カウンターを 0 に戻しました
ファイルのカウンター: 1
```

`.as_deref()` は、「`Option<String>`（あるかもしれない文字列）」を、`"reset"` と比べられる形（`Option<&str>`）に変える道具です。`return` で `main` を途中で終えています。`let _ = ...` は「結果を使わ

ずに捨てる」という意味です。

問題2の解答: `user_id` が 2 の行は Taro の「歯医者の予約」だけなので、次の1行が返ってきます。

```
歯医者の予約
```

問題3の解答: a・c・d は**データベース**です。サーバーが再起動しても残っていないと困るからです（d が消えると、全員がログアウトしてしまいます）。b は**メモリだけでよい**です。検査してデータベースに書き込んだら、その文字列はもう不要です。

まとめ

- 変数（メモリ）の値はプロセスの終了で消えます。残したいデータはストレージに**永続化**します。
- データベースはデータを**テーブル**（**行と列**）で整理し、主キー・UNIQUE・CHECK などの**制約**で間違ったデータを断ってくれます。
- 手作りのファイルでは、同時書き込み・検索・壊れたデータに弱いので、Web アプリではデータベースを使います。

UNIT 08: 認証と認可

この単元のゴール

- **認証**（あなたは誰？）と**認可**（それをしてよい？）の違いを説明できる
- ログインしてからページを見るまでの流れ（パスワードの確認 → **セッション** → **Cookie**）を説明できる
- パスワードをそのまま保存してはいけない理由と、**ハッシュ**の考え方が分かる
- 「他人のデータが見えてしまう」不具合と、それを防ぐ書き方が分かる

たとえ話でつかむ

ログインのしくみは、**ホテル**とそっくりです。

- **認証はフロントでの本人確認**です。名前と身分証で「あなたが予約した Hanako さんですね」と確かめます。Web アプリでは、メールアドレスとパスワードで確かめます。
- **セッションとカードキー**。本人確認が済むと、フロントは**カードキー**を渡し、台帳に「このカードキーは Hanako さんのもの、明日の 11 時まで有効」と記録します。それ以降、毎回身分証を見せる必要はありません。Web アプリでは、このカードキーにあたる**合言葉**（**トークン**）をブラウザの **Cookie**（**クッキー**） にしまい、ブラウザはリクエストのたびにそれを添えて送ります。
- **認可は「どの部屋を開けられるか」**です。カードキーはホテルの客だという証明にはなりますが、**開けられるのは自分の部屋だけ**です。Web アプリでも、ログインしている人が**他の人のノート**を見ようとしたら断らなければなりません。
- **ログアウトはチェックアウト**です。台帳からカードキーの記録を消すので、そのカードキーではもう扉が開きません。

用語: 認証（にんしょう、Authentication） … 相手が誰なのかを確かめること。ログインがその代表です。

用語: 認可（にんか、Authorization） … 確かめた相手に、その操作をしてよいかを判断すること。

用語: Cookie（クッキー） … サーバーがブラウザに預けておく小さなメモ。ブラウザは、同じサイトへのリクエストのたびに自動でそれを送り返します。

図で理解する

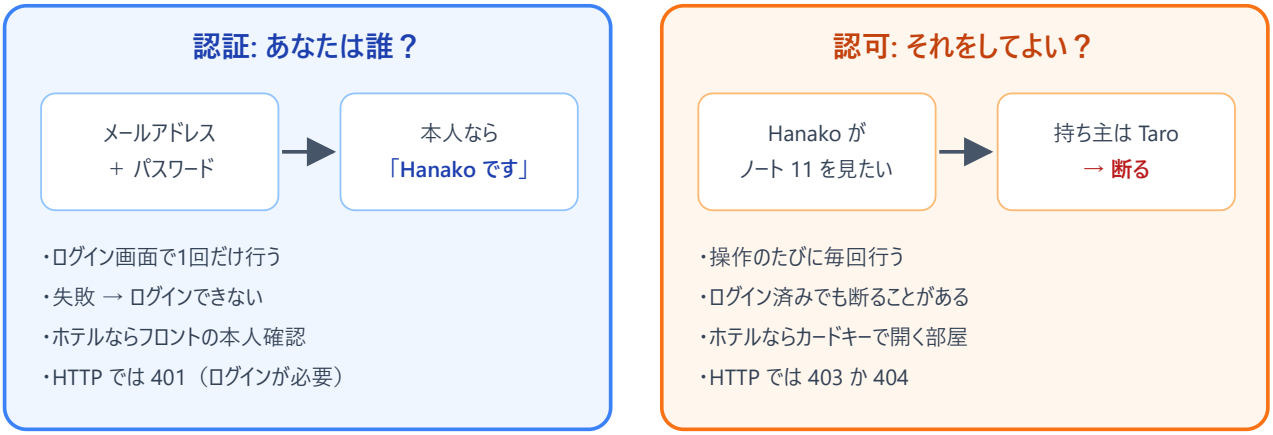


図8-1: 認証と認可は別の質問。ログインできても、他人のノートは見せない

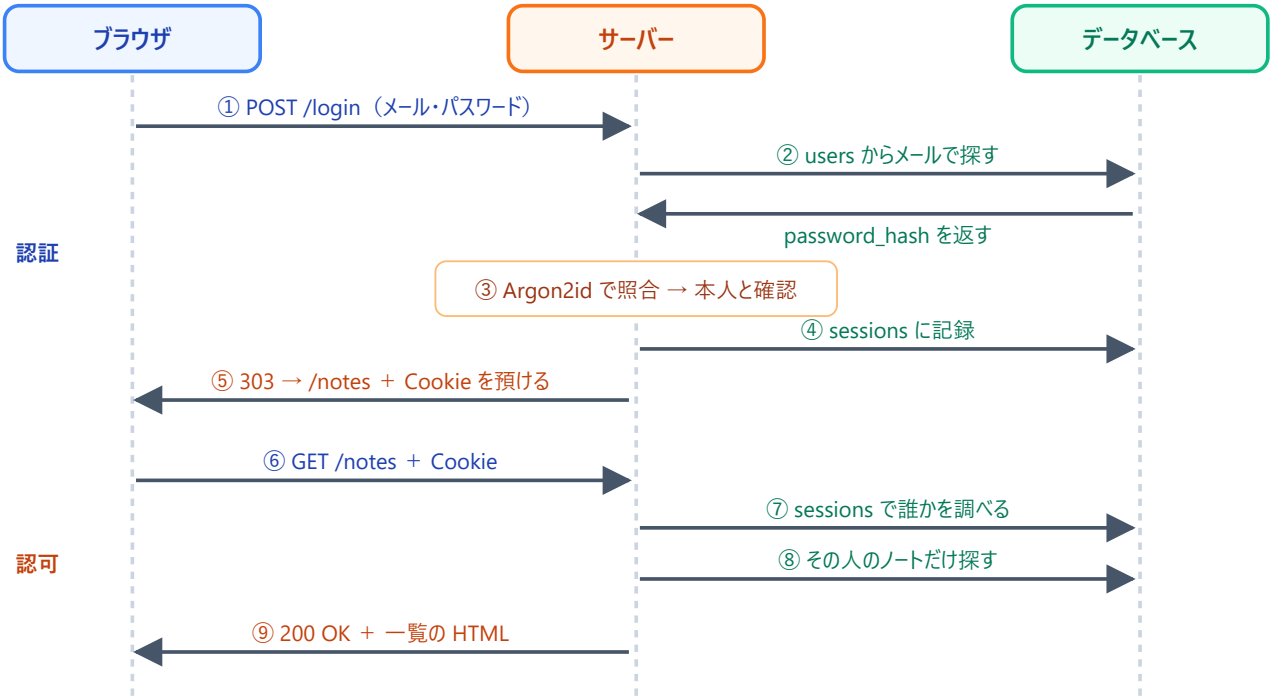


図8-2: ログインしてから一覧を見るまで。①〜⑤が認証、⑥〜⑨で毎回「誰か」を調べて認可する

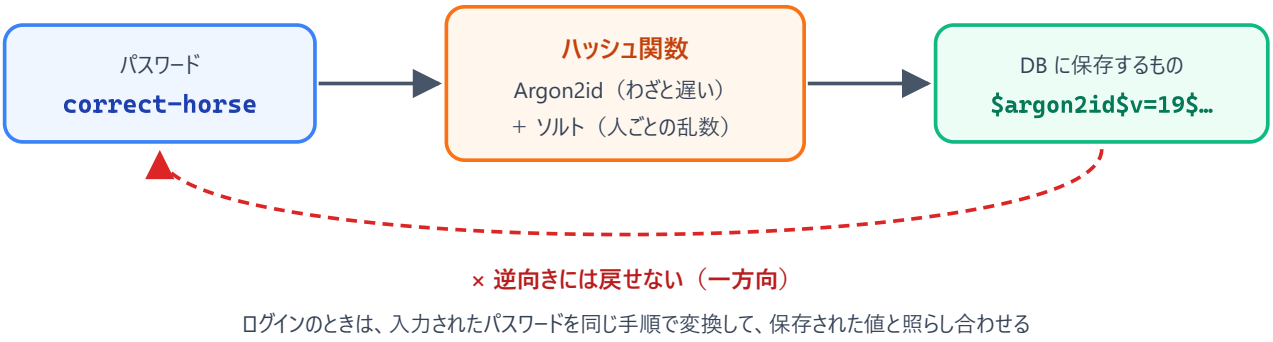


図8-3: パスワードはハッシュにして保存する。DB が盗まれても、元のパスワードは分からない

まず動かす最小コード

ログインと「持ち主チェック」のミニ版

本物のログイン機能は第8巻で作ります。ここでは、認証と認可の**考え方だけ**を、メモリの中のデータで確かめます。

> **▼ このコードがやること（先に日本語で）**：2人のユーザー（Hanako と Taro）と、それぞれのノートを1つずつ用意します。まず Hanako としてログインを試し（間違ったパスワードと正しいパスワード）、次にログインした Hanako が「自分のノート」と「Taro のノート」を見ようとしたときの結果を表示します。

```rust
// 登録した人（ユーザー）
struct User {
    id: u32,
    email: String,
    // 注意: 学習用にパスワードをそのまま持っている。本物はハッシュだけを保存する
    password: String,
}

// ノート（誰のものかを user_id で覚えている）
struct Note {
    id: u32,
    user_id: u32,
    title: String,
}

// 認証: メールアドレスとパスワードが合っていれば、その人の id を返す
fn login(users: &[User], email: &str, password: &str) -> Option<u32> {
    for user in users {
        if user.email == email && user.password == password {
            return Some(user.id);
        }
    }
    None
}

// 認可: ログイン中の人（current_user）が持ち主であるノートだけを返す
fn find_note(notes: &[Note], current_user: u32, note_id: u32) -> Option<&Note> {
    notes
        .iter()
        .find(|note| note.id == note_id && note.user_id == current_user)
}

fn main() {
    let users = vec![
        User {
            id: 1,
            email: String::from("hanako@example.com"),
            password: String::from("correct-horse"),
        },
        User {
            id: 2,
            email: String::from("taro@example.com"),
            password: String::from("battery-staple"),
        },
    ];
    let notes = vec![
        Note {
            id: 10,
            user_id: 1,
```

```
            title: String::from("牛乳を買う"),
        },
        Note {
            id: 11,
            user_id: 2,
            title: String::from("歯医者の予約"),
        },
    ];

    // 1. 認証（あなたは誰？）
    let wrong = login(&users, "hanako@example.com", "wrong-password");
    println!("[認証] 間違ったパスワード -> {wrong:?}");
    let hanako = login(&users, "hanako@example.com", "correct-horse");
    println!("[認証] 正しいパスワード   -> {hanako:?}");

    // 2. 認可（そのノートを見てよい？）
    let Some(current_user) = hanako else {
        println!("ログインできなかったので、ここで終わり");
        return;
    };
    for note_id in [10, 11] {
        match find_note(&notes, current_user, note_id) {
            Some(note) => println!("[認可] ノート {note_id} -> 200 OK「{}」",
note.title),
            None => println!("[認可] ノート {note_id} -> 404 Not Found"),
        }
    }
}
```

▼ 実行結果

```
[認証] 間違ったパスワード -> None
[認証] 正しいパスワード   -> Some(1)
[認可] ノート 10 -> 200 OK「牛乳を買う」
[認可] ノート 11 -> 404 Not Found
```

ログインに成功した Hanako でも、Taro のノート 11 は「見つからない（404）」になりました。

▼ コードを1つずつ分解して解説

解説1: データの形を決める `struct`

```
struct Note {
    id: u32,
    user_id: u32,
    title: String,
}
```

- `struct`（ストラクト、**構造体**）は、関係のある値をひとまとめにした「自分だけの型」を作る書き方です。データベースのテーブルの1行と同じ形にしています。
- `String` は、あとから長さが変わってもよい文字列の型です（第2巻）。
- `user_id` が「このノートの持ち主」です。認可のかなめになる列です。

解説2: 認証する関数 `login`

```
fn login(users: &[User], email: &str, password: &str) -> Option<u32> {
    for user in users {
        if user.email == email && user.password == password {
            return Some(user.id);
        }
    }
    None
}
```

- `users: &[User]` は「ユーザーの並びを借りて受け取る」、`-> Option<u32>` は「見つかれば `Some(id)`、見つからなければ `None` を返す」という意味です。
- `&&` は「かつ」です。メールアドレスとパスワードの**両方**が合ったときだけ、その人の `id` を返します。
- 最後の `None` は「全員調べても合わなかった」ときの返り値です。Rust では、最後に書いた値（`;` を付けない）がそのまま関数の返り値になります。

> **注意:** この例はパスワードをそのまま比べていますが、**本物のアプリでは絶対にこう書きません。**パスワードをそのまま保存すると、データベースが盗まれたとき、全員のパスワードがそのまま漏れてしまいます。正しいやり方は「もう一歩」と第8巻で扱います。

解説3: 認可する関数 `find_note`

```
fn find_note(notes: &[Note], current_user: u32, note_id: u32) -> Option<&Note> {
    notes
        .iter()
        .find(|note| note.id == note_id && note.user_id == current_user)
}
```

- `.iter().find(...)` は、並びの中から条件に合う最初の1つを探します。見つからなければ `None` です。
- `|note| ...` は **クロージャ**（その場で作る小さな関数）で、「1件ずつ `note` として受け取り、条件を満たすか」を書いています。
- 条件は「番号が一致する **かつ** 持ち主がログイン中の人」です。他人のノートは「存在しない」のと同じ扱いになります。これが認可です。

解説4: ログインできなかったら先へ進まない

```
let Some(current_user) = hanako else {
    println!("ログインできなかったので、ここで終わり");
    return;
};
```

- `let Some(x) = 値 else { ... };` は、「値が `Some` なら中身を `x` に取り出して先へ進む。`None` なら `else` の中を実行する」という書き方です。
- `else` の中は必ず `return` などで抜けなければなりません。「ログインしていないのに、ノートを探す処理へ進んでしまう」ことが、文法の上で起こりえない形になっています。

もう一歩: 実務に近い形

ハッシュの性質を確かめる（しくみを見るためのおもちゃ）

パスワードの代わりに保存する**ハッシュ**の性質を、Rust の標準ライブラリにある `DefaultHasher` で確かめます。

> **▼ このコードがやること（先に日本語で）**：いくつかの文字列から、それぞれの「指紋」のような数値（ハッシュ値）を作って表示します。同じ文字列なら同じ値になること、1文字違うだけでまったく違う値になることを確かめます。

```
use std::collections::hash_map::DefaultHasher;
use std::hash::{Hash, Hasher};

// 文字列から「指紋」のような数値を作る（しくみを見るためのおもちゃ）
fn toy_hash(text: &str) -> u64 {
    let mut hasher = DefaultHasher::new();
    text.hash(&mut hasher);
    hasher.finish()
}

fn main() {
    let inputs = ["correct-horse", "correct-horse", "correct-horsf", "a"];
    for input in inputs {
        // {:016x} は「16 けたの 16 進数で表示する」という指定
        println!("{input:<14} -> {:016x}", toy_hash(input));
    }
}
```

▼ 実行結果

```
correct-horse  -> f96723216a3e141a
correct-horse  -> f96723216a3e141a
correct-horsf  -> db8dd628cfe17851
a              -> 719b50b9a4f0e9f3
```

（数値そのものは、Rust の版によって変わることがあります。）

▼ コードを1つずつ分解して解説

解説1: ハッシュ値の作り方

```
let mut hasher = DefaultHasher::new();
text.hash(&mut hasher);
hasher.finish()
```

- `DefaultHasher` は、データから決まった長さの数値（ハッシュ値）を作る道具です。
- `text.hash(&mut hasher)` で文字列を流し込み、`finish()` で結果の数値を取り出します。

解説2: 結果から分かる性質

- **同じ入力からは、いつも同じ値**ができます（1行目と2行目）。だから、ログインのときに入力されたパスワードを同じ手順で変換すれば、保存しておいた値と照らし合わせられます。
- **1文字違うだけで、まったく違う値**になります（`horse` と `horsf`）。似ているかどうかも分かりません。
- 入力が1文字でも長い文でも、**結果は同じ長さ**です。結果から元の文字列を計算で戻すことはできません（**一方向**）。

解説3: なぜ本物には Argon2id を使うのか

> **注意:** `DefaultHasher` は、データを素早く仕分けるための道具で、**パスワードの保存には絶対に使ってはいけません。**速すぎるので、盗んだ人が「よくあるパスワード」を1秒に何億通りも試して、元のパスワードを当ててしまえるからです。

パスワード専用のハッシュ関数 **Argon2id（アルゴンツー・アイディー）** は、次の工夫で当てられにくくしてあります。

- **わざと遅く、わざとメモリをたくさん使う**ので、総当たりで試すのにとても時間がかかる
- 人ごとに違う乱数（**ソルト**）を混ぜるので、同じパスワードの人がいても保存される値は別々になる

RustNote は `users.password_hash` 列に Argon2id の結果だけを保存し、元のパスワードはどこにも保存しません（第8巻）。

よくあるエラーと読み方

持ち主チェックを書き忘れた（コンパイラの警告がヒント）

最小コードの `find_note` で、持ち主の条件を消してしまったとします（抜粋。他の部分は同じです）。

```
// 認可: ログイン中の人（current_user）が持ち主であるノートだけを返す
fn find_note(notes: &[Note], current_user: u32, note_id: u32) -> Option<&Note> {
    notes.iter().find(|note| note.id == note_id)
}
```

▼ **実行結果**（コンパイル時の警告と、実行結果）

```
   Compiling practice v0.1.0 (C:\Users\you\rust-study\practice)
warning: unused variable: `current_user`
  --> src\main.rs:27:30
   |
27 | fn find_note(notes: &[Note], current_user: u32, note_id: u32) -> Option<&Note> {
   |                              ^^^^^^^^^^^^ help: if this is intentional, prefix it
with an underscore: `_current_user`
   |
   = note: `#[warn(unused_variables)]` (part of `#[warn(unused)]`) on by default

warning: field `user_id` is never read
  --> src\main.rs:12:5
   |
10 | struct Note {
   |        ---- field in this struct
11 |     id: u32,
12 |     user_id: u32,
   |     ^^^^^^^
   |
   = note: `#[warn(dead_code)]` (part of `#[warn(unused)]`) on by default

warning: `practice` (bin "practice") generated 2 warnings (run `cargo fix --bin
"practice" -p practice` to apply 1 suggestion)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.29s
     Running `target\debug\practice.exe`
[認証] 間違ったパスワード -> None
[認証] 正しいパスワード   -> Some(1)
[認可] ノート 10 -> 200 OK「牛乳を買う」
[認可] ノート 11 -> 200 OK「歯医者の予約」
```

何が起きたか:

- 実行結果の最後の行で、Hanako が **Taro のノート「歯医者の予約」を見られてしまいました。**URL の番号を書き換えるだけで他人のデータが見えてしまうこの種の不具合は **IDOR（アイドア）** と呼ばれ、実際の Web サービスでもよく起きる、深刻な問題です。
- コンパイルは成功しましたが、コンパイラは **warning（警告）** を2つ出しています。
- `unused variable: current_user` …「引数 `current_user` を受け取ったのに、一度も使っていない」
- `field user_id is never read` …「`Note` の `user_id` を、どこからも読んでいない」
- どちらも「持ち主を調べる処理が無い」ことを、遠回しに教えてくれています。

直し方: 条件に `&& note.user_id == current_user` を戻します。警告は「エラーではないから無視してよいもの」ではなく、**バグの手がかり**です。警告が出たら、必ず理由を確かめましょう。

> **ヒント:** `help: if this is intentional, prefix it with an underscore` は「わざと使わないのなら、名前の先頭に `_` を付けてください（そうすれば警告を消せます）」という助言です。今回は**わざとではない**ので、`_` を付けて黙らせるのは間違った直し方です。

401・403・404 の使い分け

状況	返すもの	理由
ログインしていない人が `/notes` を開いた	ログイン画面へ移動させる（または 401）	まず認証が必要
ログインしているが、権限のない操作をした	403 Forbidden	認可で断った
ログインしているが、**他人のノート**を開こうとした	**404 Not Found**	「そのノートは存在する」ことすら教えないため

他人のノートに 403 を返すと、「その番号のノートは存在する（が、あなたのではない）」と相手に分かってしまいます。404 なら、存在しないノートと区別がつきません。RustNote はこの理由で 404 を返します。

RustNote ではここで使う

- 登録・ログイン・ログアウトは `GET|POST /signup`・`GET|POST /login`・`POST /logout` です。パスワードは **Argon2id** のハッシュにして `users.password_hash` に保存します。
- ログインに成功すると、推測できない長い乱数（**セッショントークン**）を作り、ブラウザには Cookie として渡し、データベースの `sessions` テーブルには**トークンのハッシュ**（`token_hash`）と有効期限を保存します。トークンそのものを保存しないのは、パスワードと同じく、DB が漏れても悪用されないようにするためです。
- Topcoat のセッション機能が使う Cookie には、`Secure`（https のときだけ送る）・`HttpOnly`（ページ内のプログラムから読めない）・`SameSite=Lax`（他のサイトからの送信に勝手に付かない）といった安全のための印が付きます（第8巻）。
- ノートを探す SQL には、必ず `WHERE user_id = ?`（ログイン中の人）を付けます。他人のノートは 404 です。

練習問題

問題1: 最小コードの `main` を書き換えて、Taro（`taro@example.com`・`battery-staple`）でログインし、ノート 10 と 11 を見ようとした結果を表示してください。

問題2: 次の a〜d は、認証と認可のどちらでしょうか。

- a. ログイン画面で、入力されたパスワードが正しいか確かめる
- b. 編集ボタンが押されたとき、そのノートの持ち主がログイン中の人か確かめる
- c. Cookie のセッショントークンから、ログイン中の人が誰かを調べる
- d. 管理者だけが、全ユーザーの一覧ページを開ける

問題3: パスワードを「そのまま」ではなく「ハッシュにして」保存する理由を、1〜2文で説明してください。

解答と解説

問題1の解答: `main` 関数の「1. 認証」から後ろを、次のように置き換えます（`struct` と関数、`users`・`notes` の用意は同じです）。

```
    // Taro（taro@example.com）でログインする
    let taro = login(&users, "taro@example.com", "battery-staple");
    println!("[認証] Taro -> {taro:?}");

    let Some(current_user) = taro else {
        println!("ログインできなかったので、ここで終わり");
        return;
    };
    for note_id in [10, 11] {
        match find_note(&notes, current_user, note_id) {
            Some(note) => println!("[認可] ノート {note_id} -> 200 OK「{}」",
note.title),
            None => println!("[認可] ノート {note_id} -> 404 Not Found"),
        }
    }
}
```

▼ 実行結果

```
[認証] Taro -> Some(2)
[認可] ノート 10 -> 404 Not Found
[認可] ノート 11 -> 200 OK「歯医者の予約」
```

Hanako のときとちょうど逆になりました。同じ `find_note` でも、ログイン中の人によって結果が変わるのが認可です。

問題2の解答: a は**認証**、b は**認可**、c は**認証**（セッションで「誰か」を確かめている）、d は**認可**（誰かは分かったうえで、してよいかを判断している）。

問題3の解答: データベースが盗まれたり、内部の人がのぞいたりしても、元のパスワードが分からないようにするためです。多くの人は同じパスワードを他のサービスでも使っているので、漏れると被害がそのサービスだけでは済みません。

まとめ

- **認証**は「あなたは誰？」（ログイン）、**認可**は「それをしてよい？」（持ち主チェック）で、別々に毎回考えます。
- ログイン後はセッショントークンを Cookie で受け渡し、サーバーは毎回それで「誰か」を調べます。パスワードもトークンも、DB には**ハッシュ**で保存します。
- 他人のデータには 404 を返します。コンパイラの「使っていない」という警告は、持ち主チェック漏れの手がかりになります。

UNIT 09: コンパイラエラーとの付き合い方

この単元のゴール

- Rust のエラーメッセージを、**エラー番号・場所・印**（`^^^`）・`note:`・`help:` の部品に分けて読める
- エラーが複数出たときに、**上から1つずつ**直す理由が分かる
- `rustc --explain` で、エラー番号の詳しい説明を読める
- **エラー**（error）と**警告**（warning）の違いが分かり、`cargo clippy` の助言を活かせる

たとえ話でつかむ

Rust のコンパイラのエラーメッセージは、**とても親切な先生の添削**（**赤ペン**）です。

- 「×」だけ付けて返す先生ではありません。**どのページの何行目の何文字目か、何が問題か、なぜそう判断したか**、そして多くの場合**どう直せばよいか**まで書いてくれます。
- 添削が10か所あっても、最初の1つの書き間違いが原因で、後ろがまとめて×になっていることがあります。**最初の赤ペンから順に**直すのが近道です。
- 先生の説明が短すぎて分からないときは、**教科書の該当ページ**（`rustc --explain`）を開けば、例つきの長い説明が読めます。

最初はエラーが出るたびに「怒られた」と感じるかもしれません。でも Rust のコンパイラは、**動かしてから壊れるはずだった問題を、動かす前に見つけてくれる**相棒です。エラーをたくさん見た人ほど、Rust が上手になります。

> **用語: エラー（error）** … これを直さないとコンパイルが終わらない（実行ファイルが作られない）問題。
>
> **用語: 警告（warning）** … コンパイルは終わるが、「たぶん間違いでは？」「もっと良い書き方がある」という指摘。UNIT 08 で見たとおり、バグの手がかりになることがあります。

図で理解する

次のエラーメッセージは「もう一歩」で実際に出すものです。部品ごとに意味があります。

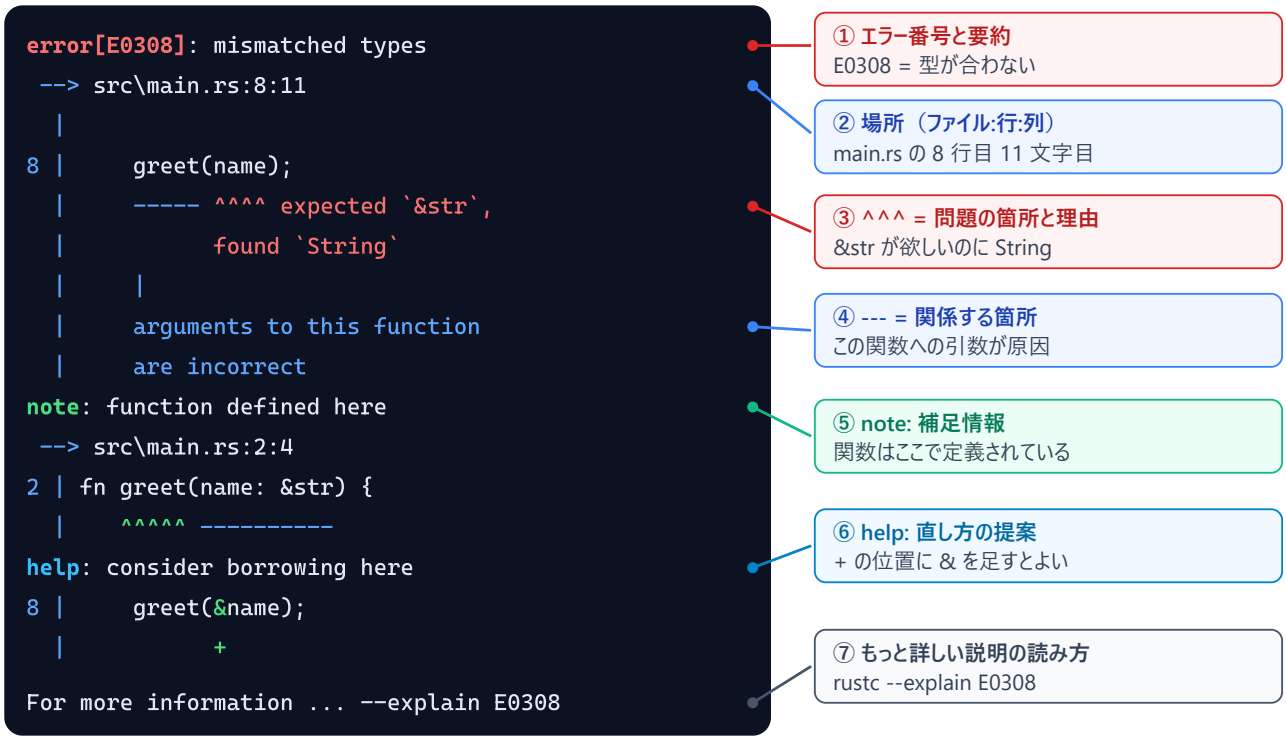


図9-1: エラーメッセージの部品（紙面に合わせて長い行は折り返している。右端の点がその行）。①→②→③の順に読み、⑥の提案を検討する

エラーが出たときの進め方は、次の流れです。

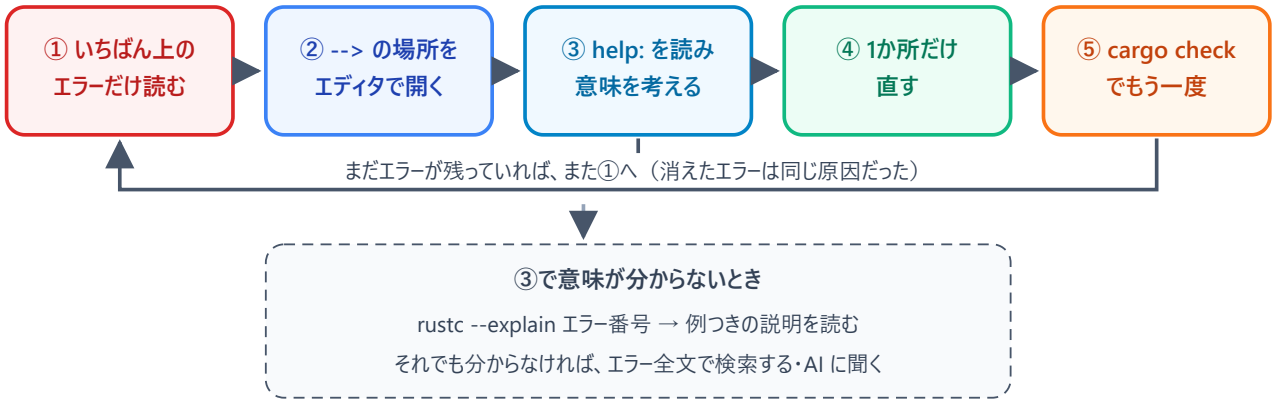


図9-2: エラーとの付き合い方。上から1つずつ、1か所直したらすぐ確かめる

まず動かす最小コード

この単元の最小コードは、**わざと間違えたコード**です。1つずつ `src\main.rs` に貼り付けて `cargo run` し、エラーを自分の目で見てから直してください。

例1: 型が合わない（E0308）

> **▼ このコードがやること（先に日本語で）**：年齢を数値（ `u32` ）の変数に入れたいのに、うっかり `"20"` と引用符を付けて**文字**として書いてしまったコードです。

```
fn main() {
    // 年齢を数値（u32）として持ちたい
    let age: u32 = "20";
    println!("年齢: {age}");
}
```

▼ 実行結果

```
   Compiling practice v0.1.0 (C:\Users\you\rust-study\practice)
error[E0308]: mismatched types
 --> src\main.rs:3:20
  |
3 |     let age: u32 = "20";
  |              ---   ^^^^ expected `u32`, found `&str`
  |              |
  |              expected due to this

For more information about this error, try `rustc --explain E0308`.
error: could not compile `practice` (bin "practice") due to 1 previous error
```

▼ 出力を1つずつ分解して解説

解説1: 1行目は「番号」と「要約」

```
error[E0308]: mismatched types
```

- `error` … 直さないとコンパイルが終わらない問題です。
- `[E0308]` … **エラー番号**です。同じ番号のエラーは同じ種類の問題なので、番号で検索すると説明が見つかります。
- `mismatched types` … 要約で、「型が食い違っている」という意味です。

解説2: `-->` は場所

```
 --> src\main.rs:3:20
```

- `src\main.rs` ファイルの **3行目・20文字目**です。VS Code では、`Ctrl + G` で行番号を指定して移動できます。ターミナルのこの部分を Ctrl を押しながらクリックすると、その場所が開くこともあります。

解説3: `^^^` と `---` の印

```
3 |     let age: u32 = "20";
  |              ---   ^^^^ expected `u32`, found `&str`
  |              |
  |              expected due to this
```

- `^^^^` は**問題の箇所**です。`"20"` の下に付いていて、「`u32` を期待していたのに、`&str`（文字列）が見つかった」と書かれています。
- `---` は**関係する箇所**です。`u32` の下に付いていて、「このせいで `u32` を期待した（expected due to this）」と、判断の根拠を示しています。
- 直し方は、引用符を外して `let age: u32 = 20;` とすることです。

解説4: 最後の2行

```
For more information about this error, try `rustc --explain E0308`.
error: could not compile `practice` (bin "practice") due to 1 previous error
```

- 1行目は「詳しい説明は `rustc --explain E0308` で読めます」という案内です（「もう一歩」で使います）。
- 2行目は Cargo のまとめで、「前に出たエラー1個のせいで、`practice` をコンパイルできなかった」という意味です。エラーの中身はこの行ではなく、**上のほう**にあります。

例2: 名前の打ち間違い（E0425）

ここから先のエラーの出力では、最初の `Compiling ...` の行を省きます。例2・例3では、最後の2行（例1の解説4と同じ形の案内とまとめ）も省いています。

```
fn main() {
    let message = "こんにちは";
    println!("{}", mesage);
}
```

▼ 実行結果

```
error[E0425]: cannot find value `mesage` in this scope
 --> src\main.rs:3:20
  |
3 |     println!("{}", mesage);
  |                    ^^^^^^
  |
help: a local variable with a similar name exists
  |
3 |     println!("{}", message);
  |                        +
```

- 「この範囲（scope）に `mesage` という値は見つからない」というエラーです。
- `help:` で「似た名前のローカル変数があります」と、`message` を提案してくれています。下の `+` は「ここに1文字足す」という印です。
- 打ち間違いは、Rust で最もよく出るエラーの1つです。`help:` の提案どおりに直せば動きます。

例3: 書き換え禁止の変数を書き換えた（E0384）

```rust
fn main() {
    let count = 0;
    count = count + 1;
    println!("count = {count}");
}
```

▼ 実行結果

```
error[E0384]: cannot assign twice to immutable variable `count`
 --> src\main.rs:3:5
  |
2 |     let count = 0;
  |         ----- first assignment to `count`
3 |     count = count + 1;
  |     ^^^^^^^^^^^^^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
2 |     let mut count = 0;
  |         +++
```

- 「書き換え禁止（immutable）の変数 `count` に、2回目の代入はできない」というエラーです。
- `---` が「最初の代入はここ（2行目）」、`^^^` が「2回目の代入はここ（3行目）」を指しています。**問題の行と原因の行が別**であることに注意してください。

- `help:` は「この変数を書き換え可能（mutable）にしては？」と、2行目に `mut` を足す（`+++`）提案をしています。UNIT 01 で学んだ `let mut` です。

3つとも、`help:` や印のとおりに直すと動きます。

```
年齢: 20
こんにちは
count = 1
```

（例1〜3を直したものを、それぞれ実行した結果です。）

もう一歩: 実務に近い形

`note:` と `help:` の両方が出るエラー

> **▼ このコードがやること（先に日本語で）**：名前を受け取ってあいさつする関数 `greet` を作り、`String` 型の変数を渡して呼び出そうとしたコードです。関数が受け取る型と、渡した型が少しだけ違っています。

```rust
// name さんにあいさつする関数
fn greet(name: &str) {
    println!("こんにちは、{name} さん");
}

fn main() {
    let name = String::from("Hanako");
    greet(name);
}
```

▼ 実行結果

```
error[E0308]: mismatched types
 --> src\main.rs:8:11
  |
8 |     greet(name);
  |     ----- ^^^^ expected `&str`, found `String`
  |     |
  |     arguments to this function are incorrect
  |
note: function defined here
 --> src\main.rs:2:4
  |
2 | fn greet(name: &str) {
  |    ^^^^^ ----------
help: consider borrowing here
  |
8 |     greet(&name);
  |           +

For more information about this error, try `rustc --explain E0308`.
error: could not compile `practice` (bin "practice") due to 1 previous error
```

▼ 出力を1つずつ分解して解説

解説1: 問題の箇所と、関係する箇所

- `^^^^` は8行目の `name` を指し、「`&str` が欲しいのに `String` が来た」と言っています。
- `-----` は `greet` を指し、「この関数への引数が正しくない（arguments to this function are incorrect）」と言っています。

解説2: `note:` は別の場所の補足

```
note: function defined here
 --> src\main.rs:2:4
```

- `note:` は「参考までに」という補足です。ここでは「その関数はここ（2行目）で定義されていて、`name: &str` という引数を受け取ると書いてある」と、**関数の側の場所**まで見せてくれています。
- エラーの原因が**2か所にまたがる**とき（呼び出す側と、定義する側）に、両方を見比べられるようにしてくれているのです。

解説3: `help:` は直し方の提案

```
help: consider borrowing here
  |
8 |     greet(&name);
  |           +
```

- 「ここで借りる（borrow）ことを検討してください」という提案で、`name` の前に `&` を足した形を見せてくれています。
- 提案どおり `greet(&name);` に直すと、`こんにちは、Hanako さん` と表示されます。
- `&` の意味（借用）は第3巻のテーマです。今は「help が具体的な直し方まで見せてくれる」ことが分かれば十分です。

> **注意:** `help:` は**多くの場合正しい**提案ですが、いつもあなたの意図どおりとは限りません。UNIT 08 の「`_` を付けて警告を消す」提案のように、意味を考えずに従うと問題を隠してしまうこともあります。「なぜその直し方で直るのか」を一言で説明できるか、自分に聞いてから直しましょう。

同じ原因のエラーは、まとめて消える

```
fn main() {
    // 在庫の数
    let mut stock = HashMap::new();
    stock.insert("りんご", 3);

    // 値段
    let mut prices = HashMap::new();
    prices.insert("りんご", 120);

    println!("{stock:?} {prices:?}");
}
```

▼ 実行結果

```
error[E0433]: cannot find type `HashMap` in this scope
 --> src\main.rs:3:21
  |
3 |     let mut stock = HashMap::new();
  |                     ^^^^^^^ use of undeclared type `HashMap`
  |
help: consider importing this struct
  |
1 + use std::collections::HashMap;
  |

error[E0433]: cannot find type `HashMap` in this scope
 --> src\main.rs:7:22
  |
7 |     let mut prices = HashMap::new();
  |                      ^^^^^^^ use of undeclared type `HashMap`
  |
help: consider importing this struct
  |
1 + use std::collections::HashMap;
  |

For more information about this error, try `rustc --explain E0433`.
error: could not compile `practice` (bin "practice") due to 2 previous errors
```

- エラーは2つですが、原因は「`HashMap` を `use` していない」の1つだけです。
- `help:` の `1 + use std::collections::HashMap;` は、「1行目にこの行を**追加**して」という意味です（`+` は追加の印）。1行足せば、**2つのエラーが同時に消えます**。
- エラーが20個出ても慌てないでください。**最初の1つを直して、もう一度コンパイル**すると、半分以下に減ることがよくあります。

`rustc --explain` で教科書を開く

```
rustc --explain E0384
```

▼ **実行結果**（先頭の部分）

```
An immutable variable was reassigned.

Erroneous code example:

```
fn main() {
    let x = 3;
    x = 5; // error, reassignment of immutable variable
}
```

By default, variables in Rust are immutable. To fix this error, add the keyword
`mut` after the keyword `let` when declaring the variable. For example:

```
fn main() {
    let mut x = 3;
    x = 5;
}
```
```

- 1行目が要約（「書き換え禁止の変数に再代入した」）、続いて**間違った例**（Erroneous code example）と、**直し方の説明と正しい例**が載っています。
- 英語ですが、コードの例は英語が読めなくても比べられます。翻訳ツールにかけるのも良い方法です。

エラーではないが、より良い書き方: `cargo clippy`

例3を `let mut count = 0;` に直したコードに `cargo clippy` をかけると、次の助言が出ます。

```
cargo clippy
```

▼ **実行結果**（助言の部分）

```
warning: manual implementation of an assign operation
 --> src\main.rs:3:5
  |
3 |     count = count + 1;
  |     ^^^^^^^^^^^^^^^^^ help: replace it with: `count += 1`
  |
  = help: for further information visit https://rust-lang.github.io/rust-clippy/rust-
1.99.0/index.html#assign_op_pattern
  = note: `#[warn(clippy::assign_op_pattern)]` on by default
```

Clippy（クリッピー） は、コンパイラより一歩踏み込んで「動くけれど、こう書くほうが Rust らしい」という助言をくれる道具です。ここでは「`count = count + 1` は `count += 1` と書けます」と教えてくれています。エラーと同じ読み方で読めます。

よくあるエラーと読み方

使い終わった値を、また使おうとした（E0382）

```
fn main() {
    let title = String::from("牛乳を買う");
    let saved = title;
    println!("保存したタイトル: {saved}");
    println!("元のタイトル: {title}");
}
```

▼ 実行結果

```
error[E0382]: borrow of moved value: `title`
 --> src\main.rs:5:24
  |
2 |     let title = String::from("牛乳を買う");
  |         ----- move occurs because `title` has type `String`, which does not
implement the `Copy` trait
3 |     let saved = title;
  |                 ----- value moved here
4 |     println!("保存したタイトル: {saved}");
5 |     println!("元のタイトル: {title}");
  |                              ^^^^^ value borrowed here after move
  |
help: consider cloning the value if the performance cost is acceptable
  |
3 |     let saved = title.clone();
  |                      ++++++++
```

何が起きたか: ラベルを上から読むと、話の筋が分かります。

1. 2行目: `title` は `String` 型で、コピーされない（`Copy` ではない）型なので、渡すと**移動**（**move**）する
2. 3行目: `title` の中身が `saved` に**移動した**（value moved here）
3. 5行目: 移動した後の `title` を使おうとした（borrowed here after move）

Rust では、文字列などの値には**持ち主がちょうど1人**いて、`let saved = title;` で持ち主が `saved` に引っ越します。引っ越した後の `title` は空き家なので、使えません。これが Rust の中心にある**所有権**というルールで、第3巻で詳しく学びます。

直し方: `help:` の提案どおり `title.clone()` で複製を作って渡すか、`saved` だけを使うように書き直します（練習問題2）。

> **ヒント:** 日本語の文字が入った行では、`^^^^^` の印が少しずれて見えることがあります。日本語の文字が画面上で2文字分の幅を取るためです。印の位置より、`-->` の行番号と列番号を信じましょう。

`;` を忘れた

```
fn main() {
    let price = 120
    let count = 3;
    println!("合計: {}", price * count);
}
```

▼ 実行結果

```
error: expected `;`, found keyword `let`
 --> src\main.rs:2:20
  |
2 |     let price = 120
  |                     ^ help: add `;` here
3 |     let count = 3;
  |     --- unexpected token

error: could not compile `practice` (bin "practice") due to 1 previous error
```

何が起きたか: 「`;` があるはずの所で、`let` というキーワードが見つかった」というエラーです。このように、文法の間違いにはエラー番号が付かないこともあります。

直し方: `help: add ; here` のとおり、2行目の終わり（`^` の位置）に `;` を足します。コンパイラは3行目の `let` を見て初めて「おかしい」と気づくので、**エラーの印が、本当に直すべき場所の次の行に付く**ことがある、と覚えておきましょう。

RustNote ではここで使う

- RustNote では Topcoat の `view!` や `#[page]` のような**マクロ**（コードを自動で書き足してくれる仕組み）を使います。マクロの中で間違えると、少し長めのエラーが出ることがありますが、読み方は同じです。まず `-->` の場所と最初の `^^^` を見ます。
- 作業の区切りごとに `cargo check`・`cargo clippy`・`cargo test` を通す習慣を付けます。第11巻では、警告が1つでもあれば失敗にする設定（`-D warnings`）で、品質を保つしくみも作ります。

練習問題

問題1: 次のコードのエラーを読み、直してください。税込み価格（120 円の 1.08 倍）を表示したいコードです。

```
fn main() {
    let price: u32 = 120;
    // 税込み価格（1.08 倍）を計算したい
    let with_tax = price * 1.08;
    println!("税込み: {with_tax}");
}
```

```
error[E0277]: cannot multiply `u32` by `{float}`
 --> src\main.rs:4:26
  |
4 |     let with_tax = price * 1.08;
  |                          ^ no implementation for `u32 * {float}`
  |
  = help: the trait `Mul<{float}>` is not implemented for `u32`
  = help: the following other types implement trait `Mul<Rhs>`:
            `&u32` implements `Mul<u32>`
            `&u32` implements `Mul`
            `u32` implements `Mul<&u32>`
            `u32` implements `Mul<Duration>`
            `u32` implements `Mul`

For more information about this error, try `rustc --explain E0277`.
error: could not compile `practice` (bin "practice") due to 1 previous error
```

問題2: 「よくあるエラー」の E0382 のコードを、`help:` の提案を使って直してください。

問題3: `rustc --explain E0425` を実行して、最初の1行に書かれている要約を日本語にしてください。

解答と解説

問題1の解答: エラーの要約は「`u32`（整数）に `{float}`（小数）を掛けることはできない」です。Rust は、整数と小数を**勝手に変換して計算しません。**どちらかに型をそろえます。

```rust
fn main() {
    let price: u32 = 120;

    // 方法 1: 整数のまま「108 を掛けて 100 で割る」（小数点以下は切り捨て）
    let with_tax = price * 108 / 100;
    println!("税込み（整数で計算）: {with_tax}");

    // 方法 2: f64（小数の型）に変換してから 1.08 倍する
    let with_tax_f64 = f64::from(price) * 1.08;
    println!("税込み（小数で計算）: {with_tax_f64}");
}
```

▼ 実行結果

```
税込み（整数で計算）: 129
税込み（小数で計算）: 129.60000000000002
```

小数で計算すると `129.60000000000002` という半端な結果になりました。コンピュータは小数を2進数で近似して持つため、こうした小さな誤差が出ます。お金の計算は、方法1のように**整数（円）で扱う**のが基本です。

`= help:` のあとの長い一覧（`Mul` を持っている型の一覧）は、今は読み飛ばして大丈夫です。エラーメッセージは、**分かる部分から読めばよい**のです。

問題2の解答:

```rust
fn main() {
    let title = String::from("牛乳を買う");
    // clone() で「複製」を作って saved に渡す。title は手元に残る
    let saved = title.clone();
    println!("保存したタイトル: {saved}");
    println!("元のタイトル: {title}");
}
```

▼ 実行結果

```
保存したタイトル: 牛乳を買う
元のタイトル: 牛乳を買う
```

問題3の解答: 最初の行は `An unresolved name was used.` で、「解決できない（見つからない）名前が使われた」という意味です。例2の `mesage` がまさにこれでした。

```
An unresolved name was used.

Erroneous code examples:
```

まとめ

- エラーメッセージは「① 番号と要約 → ② `-->` の場所 → ③ `^^^` の理由 → `note:` の補足 → `help:` の提案」の順に読みます。
- 複数のエラーは**上から1つずつ**。1か所直したらすぐ `cargo check` します。分からなければ `rustc --explain 番号`。
- 警告や Clippy の助言も読み方は同じです。`help:` は「なぜ直るのか」を考えてから使いましょう。

UNIT 10: 学習サイクルと完成像

この単元のゴール

- 「**予想** → **実行** → **観察** → **1か所変える** → **説明・記録**」の学習サイクルを、自分で回せる
- `cargo test` で**テスト**を動かし、失敗したテストの出力（`left` と `right`）を読める
- 文字列の「**バイト数**」と「**文字数**」の違いが分かる（RustNote のタイトル検査につながる）
- この巻の10単元が、RustNote の1回のリクエストのどこにつながっているかを説明できる

たとえ話でつかむ

プログラミングの上達は、**料理の練習**に似ています。

- レシピ（教材）を何冊読んでも、料理は上手になりません。上達する人は、作る前に「**たぶんこんな味になる**」と予想し（予想）、作って（実行）、味見をして（観察）、「**塩をひとつまみだけ**増やしたらどうなるか」を試し（1か所変える）、違いを言葉にして**レシピノートに書き留めます**（説明・記録）。
- 一度に塩と砂糖と火加減を全部変えると、何が効いたのか分かりません。**変えるのは1か所ずつ**です。
- 書き留めたレシピノートが、プログラミングでは**テスト**です。「このタイトルは正しい」「空のタイトルはだめ」を自動で確かめるコードを書いておくと、後でコードを変えたときに、前にできていたことが壊れていないかを**一瞬で確かめられます**。

> **用語: テスト** … プログラムが期待どおりに動くかを確かめるための、別の小さなプログラム。Rust では `#[test]` という印を付けた関数がテストになり、`cargo test` で全部まとめて実行できます。

図で理解する

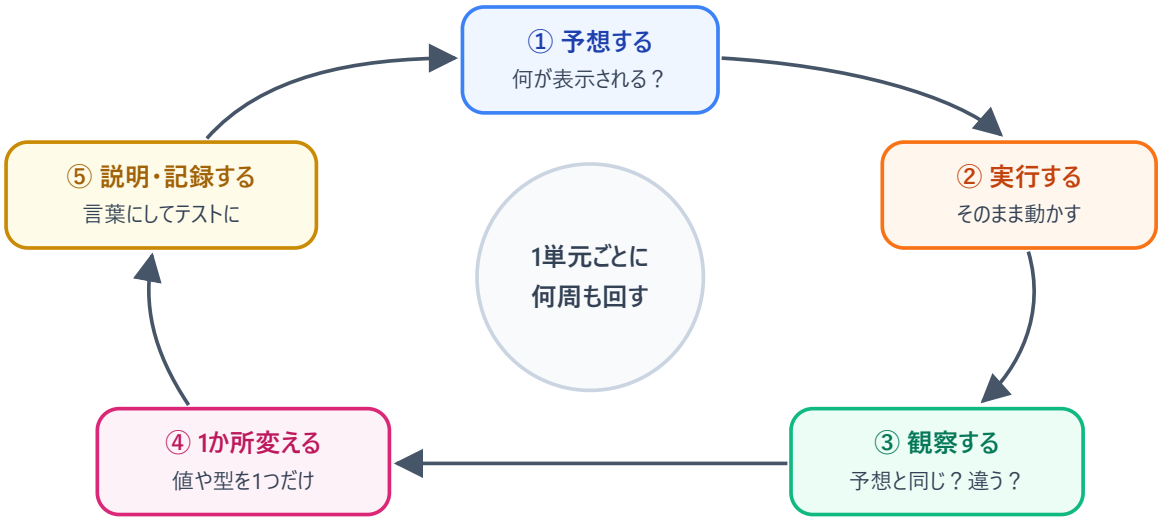


図10-1: 学習サイクル。読んだ量ではなく、このサイクルを何周回したかで進み具合を測る

この単元のコードで扱う「文字数」の落とし穴を、先に図で見ておきます。Rust の文字列は **UTF-8** という形でメモリに並んでいて、日本語の文字は1文字が **3バイト** になります。



図10-2: バイト数と文字数。「80文字まで」を len() で数えると、日本語は 27 文字目で上限を超えてしまう

まず動かす最小コード

予想してから実行する: タイトルの文字数チェック（バグあり）

RustNote のノートのタイトルは「1文字以上80文字以下」と決めています。その検査をする関数を書いてみました。ただし、この版には**バグ**が1つ入っています。

> **▼ このコードがやること（先に日本語で）**：タイトルの文字数を数える関数と、「1〜80文字なら正しい」と判定する関数を作り、3つのタイトルで試して結果を表示します。さらに、正しく動いているかを自動で確かめる**テスト**を4つ用意します。

```rust
// タイトルの文字数を数える（この版にはバグがある）
fn title_length(title: &str) -> usize {
    title.len()
}

// 1 文字以上 80 文字以下なら true（正しいタイトル）
fn is_valid_title(title: &str) -> bool {
    let length = title_length(title);
    (1..=80).contains(&length)
}

fn main() {
    for title in ["Rust", "牛乳を買う", ""] {
        let length = title_length(title);
        let valid = is_valid_title(title);
        println!("{title:?}: 文字数 {length}, 正しい? {valid}");
    }
}

// ここから下は cargo test のときだけ使われる「テスト」
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn counts_japanese_characters() {
        assert_eq!(title_length("牛乳を買う"), 5);
    }

    #[test]
    fn empty_title_is_invalid() {
        assert!(!is_valid_title(""));
    }

    #[test]
    fn eighty_characters_is_valid() {
        // 「あ」を 80 個並べた文字列を作る
        let title = "あ".repeat(80);
        assert!(is_valid_title(&title));
    }

    #[test]
    fn eighty_one_characters_is_invalid() {
        let title = "あ".repeat(81);
        assert!(!is_valid_title(&title));
    }
}
```

① **予想:** `cargo run` すると、3行それぞれに何と表示されるでしょうか。特に `"牛乳を買う"` の文字数を予想してから、次に進んでください。

② **実行:**

```
cargo run -q
```

▼ 実行結果

```
"Rust": 文字数 4, 正しい? true
"牛乳を買う": 文字数 15, 正しい? true
"": 文字数 0, 正しい? false
```

③ **観察:** `"牛乳を買う"` は5文字なのに、**15** と表示されました。予想と違ったところが、学びのチャンスです。続けてテストも実行してみます。

```
cargo test
```

▼ 実行結果

```
   Compiling practice v0.1.0 (C:\Users\you\rust-study\practice)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.46s
     Running unittests src\main.rs (target\debug\deps\practice-7d4a8a329333a0ec.exe)

running 4 tests
test tests::eighty_one_characters_is_invalid ... ok
test tests::empty_title_is_invalid ... ok
test tests::eighty_characters_is_valid ... FAILED
test tests::counts_japanese_characters ... FAILED

failures:

---- tests::eighty_characters_is_valid stdout ----

thread 'tests::eighty_characters_is_valid' (54260) panicked at src\main.rs:39:9:
assertion failed: is_valid_title(&title)

---- tests::counts_japanese_characters stdout ----

thread 'tests::counts_japanese_characters' (3680) panicked at src\main.rs:27:9:
assertion `left == right` failed
  left: 15
 right: 5
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    tests::counts_japanese_characters
    tests::eighty_characters_is_valid

test result: FAILED. 2 passed; 2 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--bin practice`
```

▼ コードと出力を1つずつ分解して解説

解説1: バグの正体は `len()`

```
fn title_length(title: &str) -> usize {
    title.len()
}
```

- `len()` は文字列の **バイト数** を返します（図10-2）。英数字だけなら文字数と同じなので、`"Rust"` では正しく見えます。

- 日本語は1文字3バイトなので、`"牛乳を買う"` は 15 になりました。この関数のままだと、日本語のタイトルは **26文字（78バイト）までしか**付けられません。
- `usize` は「0 以上の整数で、長さや個数を表すのに使う型」です。

解説2: テストの書き方

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn counts_japanese_characters() {
        assert_eq!(title_length("牛乳を買う"), 5);
    }
    // ...
}
```

- `#[cfg(test)]` は「`cargo test` のときだけコンパイルする」という印です。ふだんの `cargo run` では、テストは実行ファイルに含まれません。
- `mod tests { ... }` はテストをまとめる入れ物（**モジュール**）、`use super::*;` は「外側で作った関数をここで使えるようにする」という宣言です。
- `#[test]` を付けた関数が、テスト1つ分です。
- `assert_eq!(実際の値, 期待する値)` は「2つが等しいはず。違ったらテスト失敗」、`assert!(条件)` は「条件が true のはず」という確認です。`!` は「〜でない」です。

解説3: テストの結果の読み方

```
test tests::eighty_characters_is_valid ... FAILED
test tests::counts_japanese_characters ... FAILED
```

- テストごとに `ok`（成功）か `FAILED`（失敗）が並びます。実行の順番は毎回変わることがあります（いくつかのテストを同時に動かしているため）。
- 最後の `test result: FAILED. 2 passed; 2 failed;` が集計です。

```
thread 'tests::counts_japanese_characters' (3680) panicked at src\main.rs:27:9:
assertion `left == right` failed
  left: 15
 right: 5
```

- 失敗したテストの詳しい理由です。`src\main.rs:27:9` が失敗した `assert_eq!` の場所です。
- `left` は `assert_eq!` の**左側（実際の値）**、`right` は**右側（期待した値）**です。「15 になったが、5 を期待していた」と読みます。

- `eighty_characters_is_valid` は `assertion failed: is_valid_title(&title)`、つまり「80文字のタイトルが正しいと判定されなかった」です（240バイトなので 80 を超えた）。

解説4: うまくいったテストにも意味がある

- `eighty_one_characters_is_invalid`（81文字はだめ）は、バグがあっても**たまたま**成功しています（243バイトも 80 を超えるので）。
- テストは「成功した＝正しい」とは限りません。**失敗するはずのことが、ちゃんと失敗するか**も確かめるために、境目（0文字・1文字・80文字・81文字）をいくつも用意しておくのがコツです。

もう一歩: 実務に近い形

④ 1か所だけ変えて、⑤ テストで確かめる

直すのは `title_length` の**1行だけ**です。

```
// タイトルの文字数を数える（文字の個数で数える）
fn title_length(title: &str) -> usize {
    title.chars().count()
}
```

ほかの部分（`is_valid_title`・`main`・テスト）は、そのままにします。

```
cargo run -q
cargo test
```

▼ 実行結果

```
"Rust": 文字数 4, 正しい? true
"牛乳を買う": 文字数 5, 正しい? true
"": 文字数 0, 正しい? false
```

```
   Compiling practice v0.1.0 (C:\Users\you\rust-study\practice)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.28s
     Running unittests src\main.rs (target\debug\deps\practice-7d4a8a329333a0ec.exe)

running 4 tests
test tests::counts_japanese_characters ... ok
test tests::empty_title_is_invalid ... ok
test tests::eighty_one_characters_is_invalid ... ok
test tests::eighty_characters_is_valid ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

▼ コードを1つずつ分解して解説

解説1: `chars().count()` で文字を数える

```
title.chars().count()
```

- `.chars()` は文字列を**1文字ずつ**取り出します（バイトではなく文字単位）。
- `.count()` は、取り出した個数を数えます。`"牛乳を買う"` なら 5 です。
- データベースの SQLite（libSQL）の `length()` も文字数を数えるので、RustNote ではアプリ側も DB 側も「文字数で 1〜80」にそろえています。

解説2: `contains` で範囲を確かめる

```
(1..=80).contains(&length)
```

- `1..=80` は「1 以上 80 以下」の範囲です（UNIT 01 の `for` でも使いました）。
- `.contains(&length)` は「`length` がこの範囲に入っているか」を true / false で返します。`length >= 1 && length <= 80` と同じ意味ですが、範囲が一目で分かります。

解説3: テストは「直したこと」を固定する

- 今回は、テストのおかげで「直ったこと」を**4つの角度から一度に**確かめられました。
- もっと大事なのはこれからです。後で誰かが「`len()` のほうが速いのでは」と書き換えたら、`cargo test` がすぐに `left: 15, right: 5` で知らせてくれます。**一度直したバグが二度と戻らない**ようにするのがテストの役目です。
- 学習でも同じです。分かったことをテストの形で残すと、自分の理解の「記録」にもなります。

よくあるエラーと読み方

テストの失敗を読む

ここまでに見た、テストの失敗の出力をまとめます。

出力	意味	次にすること
`test 名前 ... FAILED`	そのテストが失敗した	下の `---- 名前 stdout ----` を読む
`panicked at src\main.rs:27:9:`	失敗した確認の場所	その行の `assert` を見る
`assertion `left == right` failed` と `left:` / `right:`	`assert_eq!` で値が違った	左（実際）と右（期待）を比べる
`assertion failed: 式`	`assert!` の条件が false だった	その式がなぜ false か考える
`test result: FAILED. 2 passed; 2 failed;`	集計	全部 ok になるまで直す
`error: test failed, to rerun pass `--bin practice``	Cargo のまとめ	気にしなくてよい（再実行の方法の案内）

> **つまずきポイント**: テストが失敗したとき、**テストのほうを書き換えて**通そうとしたくなることがあります。たとえば `assert_eq!(title_length("牛乳を買う"), 15);` に変えれば通りますが、それは「バグを正しいことにしてしまう」直し方です。失敗したら、まず「**期待した値（right）のほうが正しいか**」を考え、正しければ**コードのほう**を直します。

RustNote ではここで使う

- RustNote では、タイトルを `String` のままではなく `NoteTitle` という専用の型にして、作るときに必ず `NoteTitle::parse` で「1〜80文字か」を検査します（上限は `MAX_TITLE_CHARS = 80` という定数）。文字数は、この単元と同じ `chars().count()` で数えます。検査を通らないタイトルは、そもそも `NoteTitle` として存在できないので、「検査を忘れる」事故が起こりません（第4巻・第9巻）。
- この単元のようなテストを、第11巻ではたくさん書きます。関数のテスト、HTTP のリクエストを送ってレスポンスを確かめるテスト、「他人のノートは 404」を確かめるセキュリティのテストまで用意します。

この巻の10単元は、RustNote の**1回のリクエスト**の中に全部登場します。



図10-3: RustNote の1回のノート作成と、この巻の単元の対応

練習問題

問題1（予想してから実行）：「もう一歩」で直した版に、次のテストを追加して `cargo test` すると、成功するでしょうか、失敗するでしょうか。予想してから実行してください。

```
#[test]
fn spaces_only_title_is_invalid() {
    assert!(!is_valid_title("   "));
}
```

問題2: 問題1で失敗した場合は、`title_length` の1行だけを変えて、5つのテストがすべて成功するようにしてください。

問題3: 図10-1 の学習サイクルを、この単元のどの作業が①〜⑤のどれに当たるか、対応づけて説明してください。

解答と解説

問題1の解答: 失敗します。空白3つの `" "` は、`chars().count()` で数えると3文字なので、「1〜80文字」の範囲に入り、正しいタイトルと判定されてしまうからです（出力の最初の3行と最後の1行は省略しています）。

```
running 5 tests
test tests::counts_japanese_characters ... ok
test tests::eighty_characters_is_valid ... ok
test tests::eighty_one_characters_is_invalid ... ok
test tests::empty_title_is_invalid ... ok
test tests::spaces_only_title_is_invalid ... FAILED

failures:

---- tests::spaces_only_title_is_invalid stdout ----

thread 'tests::spaces_only_title_is_invalid' (53800) panicked at src\main.rs:50:9:
assertion failed: !is_valid_title("   ")
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    tests::spaces_only_title_is_invalid

test result: FAILED. 4 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

このように、**先に「こうあるべき」というテストを書いて、失敗するのを確かめてから直す**やり方は、実務でもよく使われます。

問題2の解答: 数える前に、前後の空白を `trim()` で取り除きます。

```
// タイトルの文字数を数える（前後の空白を取り除いてから数える）
fn title_length(title: &str) -> usize {
    title.trim().chars().count()
}
```

▼ 実行結果（`cargo test`）

```
running 5 tests
test tests::counts_japanese_characters ... ok
test tests::eighty_characters_is_valid ... ok
test tests::eighty_one_characters_is_invalid ... ok
test tests::empty_title_is_invalid ... ok
test tests::spaces_only_title_is_invalid ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

なお、空白だけのタイトルを認めるかどうかは、アプリの**決めごと（仕様）**です。ここでは「認めない」と決めた場合の練習です。RustNote での扱いは第9巻で要件として決めます。

問題3の解答（例）：①「`"牛乳を買う"` は 5 と出るはず」と予想した → ② `cargo run` と `cargo test` を実行した → ③ 15 と表示され、テストが2つ失敗したのを見た → ④ `len()` を `chars().count()` の1か所だけ変えた → ⑤「`len()` はバイト数、`chars().count()` は文字数」と説明できるようになり、テストとして残した。問題1・2は、2周目のサイクルです。

まとめ

- 「予想 → 実行 → 観察 → 1か所変える → 説明・記録」を回すほど、理解が深くなります。
- `len()` は**バイト数**、`chars().count()` は**文字数**です。日本語を扱うときは特に注意します。
- テストは、分かったこと・直したことを**固定する記録**です。失敗したら `left`（実際）と `right`（期待）を比べ、まずコードを疑います。

この先の学び方

巻の順番

基本は第0巻から第13巻まで、**番号順**に読みます。それぞれの巻が、前の巻の知識を使うように作られているからです。

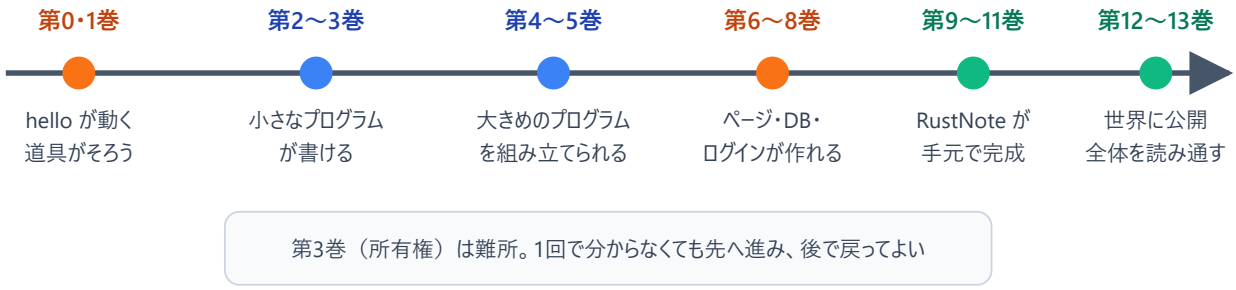


図10-4: 全14巻のマイルストーン。各段階で「できるようになること」

- **第1巻（環境構築）は飛ばさない**でください。道具の版がそろっていないと、教材と違うエラーが出て、原因探しに時間を取られます。
- **第3巻（所有権・借用）は、多くの人がつまずく難所**です。1回で全部分からなくても大丈夫です。第4巻以降で何度も出てくるうちに分かってくるので、7割分かったら先へ進み、後で戻ってきましょう。
- 第13巻には RustNote の**完全なコード**が載っています。第9〜10巻で手が止まったら、答え合わせに使ってかまいません。ただし、まず自分で書いてから見るほうが、ずっと身に付きます。

つまずいたときの調べ方

困ったときは、次の順番で調べると、たいていの問題は自分で解決できます。

1. **エラーメッセージを最初から読む** … UNIT 09 の読み方で、`-->` の場所と `help:` を確かめる。
2. **今いる場所と版を確かめる** … `Get-Location`（UNIT 02・03）、`cargo --version`。教材と同じ版か。

3. **最小の形にする** … 動かないコードから、関係なさそうな行を1つずつ消して、エラーが出る最小のコードを作る。これだけで原因が見えることが多い。
4. **公式の説明を読む** … `rustc --explain エラー番号`、Rust の公式の入門書「The Rust Programming Language」（日本語訳もあります）、クレートの説明サイト docs.rs、Topcoat の公式ドキュメント。
5. **エラー全文で検索する** … エラーの1行目（`error[E0308]: mismatched types` など）をそのまま検索する。自分のユーザー名などは消してから。
6. **人や AI に聞く** … 下の「AI の上手な使い方」のとおりに聞く。

> **ヒント:** 調べて解決したことは、短いメモに残しておきましょう。「何をしたら・どんなエラーが出て・何が原因で・どう直したか」の4行で十分です。同じ問題に2回目に出会ったとき、そのメモが一番の味方になります。

Claude Code など AI の上手な使い方

Claude Code（クロード コード）のような AI のアシスタントは、Rust の学習でもとても頼りになります。ただし、**使い方しだいで、上達を大きく助けることも、逆に妨げることもあります。**

上手な聞き方: 次の4つをそろえて伝えると、的確な答えが返ってきやすくなります。

```
【やりたいこと】RustNote のタイトルを 1〜80 文字に制限したい
【したこと】title.len() で長さを調べて、80 以下なら OK にした
【起きたこと】日本語のタイトルが 27 文字で「長すぎる」と言われる
【聞きたいこと】原因と、正しい数え方を、理由も含めて教えてください
```

- **エラーは全文を貼る** … 「エラーが出ました」だけでは、AI も推測するしかありません。`error[...]` から最後の行まで、そのまま貼ります。
- **「直して」より「なぜ？」と聞く** … 直ったコードだけをもらうと、次に同じ問題が起きたとき自分で直せません。「なぜこのエラーになるのか」「なぜその直し方でよいのか」を説明してもらいましょう。
- **答えは必ず自分で動かして確かめる** … AI の答えは、もっともらしいのに間違っていることがあります。特に Topcoat のような新しいフレームワークは、古い版の書き方を答えてしまうことがあります。`cargo check`・`cargo test` で確かめ、教材や公式ドキュメントと見比べます。
- **まず自分で考えてから聞く** … UNIT 10 のサイクルの①予想と③観察を自分でしてから聞くと、答えの意味がよく分かります。
- **秘密を貼らない** … パスワード、API キー、`.env` ファイルの中身、本物の利用者のデータは、AI にも検索にも貼りません。

> **注意:** AI に書いてもらったコードを、意味が分からないまま RustNote に入れるのはやめましょう。認証や認可（UNIT 08）のような安全に関わる部分は特にです。「このコードを1行ずつ説明して」と頼み、説明できるようになってから使います。

この巻のあとにすること

この巻はこれで終わりです。次は**第1巻（環境構築）**で、Windows に PowerShell・Git・VS Code・Rust などの道具をそろえます。道具がそろったら、この巻に戻ってきて、各単元のコードを**実際に動かしながら**もう一度読んでみてください。1回目には見えなかったことが、きっと見えてきます。

この巻のまとめ

各単元のゴールを、チェックリストにしました。自分の言葉で説明できるか、手を動かしてできるかを確かめて、できたものに印を付けましょう。説明できないものがあれば、その単元の「たとえ話」と「図」に戻ってください。

コンピュータの基本（UNIT 01〜04）

- ☐ CPU・メモリ・ストレージ・OS の役割を、台所のたとえで説明できる
- ☐ プロセスとは何かを説明でき、プロセス ID（PID）が起動のたびに変わる理由が分かる
- ☐ 標準出力・標準エラー出力・終了コードを区別でき、PowerShell で `$LASTEXITCODE` を確かめられる
- ☐ 絶対パスと相対パスを書き分けられ、`.` と `..` の意味が分かる
- ☐ 「ファイルが見つからない」と言われたら、カレントディレクトリを確かめられる
- ☐ PowerShell で `Get-Location`・`cd`・`ls`・`mkdir`・`cat`・`Copy-Item`・`Remove-Item` が使える
- ☐ コマンドを「コマンド名・サブコマンド・オプション・引数」に分けて読める
- ☐ コマンドライン引数と環境変数で、プログラムに値を渡せる
- ☐ Windows PowerShell 5.1 では `curl.exe` と書く理由、ファイルを UTF-8 で作る理由が分かる
- ☐ `cargo new`・`cargo run`・`cargo build`・`cargo check`・`--release` を使い分けられる
- ☐ コンパイル時の失敗と、実行時の失敗（パニック）を区別できる

Web の基本（UNIT 05〜08）

- ☐ URL をスキーム・ホスト・ポート・パス・クエリに分けられる
- ☐ URL を入れてから画面が出るまでの流れ（DNS → 接続 → リクエスト → 処理 → レスポンス → 描画）を説明できる
- ☐ HTTP のリクエストとレスポンスの形（1行目・ヘッダー・空行・本文）を説明できる
- ☐ GET と POST、200・303・404・500 の意味が分かる
- ☐ `curl.exe -v` で、やりとりがどこまで進んだかを確かめられる
- ☐ ポート・localhost（127.0.0.1）・0.0.0.0 の違いを説明できる
- ☐ Rust の標準ライブラリだけで、パスによって返事を変える小さなサーバーを動かせる
- ☐ 「ポートが使用中」「実行ファイルを消せない」エラーに対処できる
- ☐ メモリとストレージの違いを、カウンターの例で説明できる
- ☐ テーブル・行・列・主キー・UNIQUE・CHECK の意味が分かる
- ☐ 認証と認可の違いを説明でき、他人のノートに 404 を返す理由が分かる

- ☐ パスワードをハッシュ（Argon2id）で保存する理由が分かる

学び方（UNIT 09〜10）

- ☐ エラーメッセージを、エラー番号・`-->`・`^^^`・`note:`・`help:` の部品に分けて読める
- ☐ エラーが複数出ても、上から1つずつ直せる
- ☐ `rustc --explain` でエラー番号の説明を読める
- ☐ `cargo test` を実行し、`left` と `right` を比べて失敗の理由を読める
- ☐ `len()`（バイト数）と `chars().count()`（文字数）の違いが分かる
- ☐「予想 → 実行 → 観察 → 1か所変える → 説明・記録」のサイクルを回せる
- ☐ RustNote の完成像（画面・URL・ブラウザ／サーバー／データベースの構成）と、全14巻の順番を説明できる

用語ミニ辞典

この巻に出てきた用語を、五十音順（数字・英字は最後）に並べました。「初出の単元」は、その用語を最初に説明した UNIT の番号です。

用語	読み	意味	初出の単元
アプリ（プログラム）	アプリ	決まった手順を書いたもの。動かしていない状態のもの	01
暗号化	あんごうか	途中で盗み見や書き換えをされないように、データを変換すること。HTTPS で使う	05
一方向	いっぽうこう	ハッシュ値から元のデータに戻せない性質	08
永続化	えいぞくか	プログラムが終わってもデータが残るように保存すること	07
エイリアス	エイリアス	コマンドの別名。PowerShell の `ls` は `Get-ChildItem` の別名	03
エラー（error）	エラー	直さないとコンパイルが終わらない問題	09
拡張子	かくちょうし	ファイル名の最後の `.` から後ろ（`.rs` など）。中身の種類の目印	02
カレントディレクトリ	カレントディレクトリ	今いるフォルダ。相対パスの出発点	02
環境変数	かんきょうへんすう	OS やシェルが持つ「名前＝値」のメモ。設定や秘密を渡すのに使う	03
機械語	きかいご	CPU がそのまま実行できる命令の並び	04
クライアント	クライアント	サーバーに頼みごとをする側（ブラウザなど）	06
クレート	クレート	Rust のプログラムや部品のひとまとまり	04
警告（warning）	けいこく	コンパイルは通るが、間違いの可能性やより良い書き方を知らせる指摘	08
構造体（struct）	こうぞうたい	関係のある値をまとめた、自分で作る型	08
コマンドライン引数	コマンドラインひきすう	コマンドの後ろに並べて、プログラムに渡す値	01
コンパイル	コンパイル	ソースコードを機械語に翻訳すること。翻訳する道具がコンパイラ（Rust では `rustc`）	04
サーバー	サーバー	リクエストを待ち受けて、レスポンスを返す側のプログラム	06
シェル	シェル	コマンドを読んで、OS にプログラムの起動などを頼むプログラム（PowerShell・bash）	03
終了コード	しゅうりょうコード	プロセスが終わるときに OS へ渡す数。0 は成功	01
主キー	しゅキー	テーブルの1行を特定するための列（`id` など）。同じ値は2つない	07

用語	読み	意味	初出の単元
ステータスコード	ステータスコード	HTTP のレスポンスの結果を表す3桁の数（200・404 など）	05
ストレージ	ストレージ	電源を切ってもデータが残る記憶装置（SSD など）	01
制約	せいやく	データベースの列に付けるルール（NOT NULL・UNIQUE・CHECK など）	07
セッション	セッション	ログインした状態を覚えておくしくみ。トークンを Cookie で受け渡す	08
絶対パス	ぜったいパス	ドライブ名（`C:\`）から書いた、どこからでも同じ場所を指すパス	02
相対パス	そうたいパス	今いるフォルダから見た道順で書いたパス	02
ソルト	ソルト	パスワードのハッシュに混ぜる、人ごとに違う乱数	08
ソースコード	ソースコード	人が読み書きするプログラムの文章	04
端末（ターミナル）	たんまつ	文字を入力して結果を表示する窓	03
テスト	テスト	プログラムが期待どおりに動くかを確かめる小さなプログラム	10
テーブル	テーブル	データベースの表。行（1件）と列（項目）でできている	07
データベース（DB）	データベース	データを整理して保存し、探したり書き換えたりするプログラム	07
認可	にんか	その操作をしてよいかを判断すること	08
認証	にんしょう	相手が誰かを確かめること（ログイン）	08
ハッシュ	ハッシュ	データから決まった長さの値を作る一方向の変換。パスワードの保存に使う	08
バイト	バイト	データの大きさの単位。UTF-8 では英数字が1バイト、日本語がふつう3バイト	10
パス	パス	ファイルやフォルダの場所を表す文字列。URL の中の `/notes` などもパスと呼ぶ	02
パニック	パニック	Rust のプログラムが実行中に問題を見つけて止まること。終了コードは 101	01
標準エラー出力	ひょうじゅんエラーしゅつりょく	エラーの知らせを書く出口（`eprintln!`）	01
標準出力	ひょうじゅんしゅつりょく	ふつうの結果を書く出口（`println!`）	01

用語	読み	意味	初出の単元
標準ライブラリ	ひょうじゅんライブラリ	Rust に最初から付いてくる道具箱（`std`）	01
フォルダ（ディレクトリ）	フォルダ	ファイルやフォルダを入れる入れ物	02
フレームワーク	フレームワーク	アプリでよく使う仕組みをまとめた部品集。Topcoat など	巻頭
プロセス	プロセス	実行中のプログラム	01
プロトコル	プロトコル	やりとりの書き方の決まり。HTTP など	05
プロンプト	プロンプト	コマンドの入力を待っている印（`PS C:\Users\you>`）	03
ポート	ポート	1台のコンピュータの中で、どのプロセスに届けるかを決める番号	06
マイグレーション	マイグレーション	テーブルの形を変える手順を、順番に記録して適用するしくみ	07
メソッド（HTTP）	メソッド	リクエストで「何をしてほしいか」を表す言葉（GET・POST）	05
メモリ	メモリ	作業中のデータを置く、速いが電源を切ると消える記憶装置	01
文字コード	もじコード	文字と数値の対応表。UTF-8・Shift_JIS など	03
リクエスト	リクエスト	クライアントからサーバーへのお願い	05
ルーティング	ルーティング	メソッドとパスによって、処理を振り分けること	06
レスポンス	レスポンス	サーバーからの返事	05
0.0.0.0	ゼロゼロゼロゼロ	「すべての入口で待つ」という待ち受け側の指定	06
127.0.0.1（localhost）	ローカルホスト	自分の PC を指す特別な住所	06
Argon2id	アルゴンツー・アイディー	パスワード専用の、わざと遅いハッシュ関数	08
bind / listen	バインド／リッスン	ポートを借りる届け出／来客を待つこと	06
Cargo	カーゴ	Rust のプロジェクト管理の道具。ビルド・実行・テストなどを取り仕切る	04
Cookie	クッキー	サーバーがブラウザに預ける小さなメモ。リクエストのたびに送り返される	08
CPU	シーピーユー	命令を実行して計算する部品	01

用語	読み	意味	初出の単元
curl	カール	HTTP のリクエストを送り、結果を文字で見せる道具。Windows では `curl.exe`	05
DNS	ディーエヌエス	ホスト名を IP アドレスに引き直すしくみ	05
edition	エディション	Rust の文法の版。このシリーズは 2024	04
HTML	エイチティーエムエル	Web ページの中身を書く言葉	05
HTTP / HTTPS	エイチティーティーピー	ブラウザとサーバーのやりとりの決まり。HTTPS は暗号化版	05
IDOR	アイドア	ID を書き換えるだけで他人のデータが見えてしまう不具合	08
IP アドレス	アイピーアドレス	ネットワーク上のコンピュータの住所	06
OS	オーエス	コンピュータの資源をプロセスに割り振る基本のソフト（Windows など）	01
PowerShell	パワーシェル	Windows のシェル。この教材の主な作業環境	03
PRG	ピーアールジー	POST の後に 303 で GET のページへ移動させる作法	05
SQL	エスキューエル	データベースにお願いするための言葉	07
TCP	ティーシーピー	2台の間で、データを順番どおり確実に届けるしくみ。HTTP はこの上を流れる	05
Turso / libSQL	ターソ／リブエスキューエル	RustNote が使うデータベースのサービスと、その中身のデータベース	07
URL	ユーアールエル	Web 上の宛先。スキーム・ホスト・ポート・パス・クエリでできている	05
UTF-8	ユーティーエフエイト	世界中の文字を表せる文字コード。Rust の文字列はこれ	03